Right on Track: A Hybrid Approach to Travel Time Estimation

A Thesis Proposal

presented to

the Department of Software Technology
College of Computer Studies
De La Salle University

In partial fulfillment
of the requirements for the degree of

Bachelor of Science in Computer Science
Major in Software Technology

by

CANSINO, Joehanna L.
DY, Harmony Claire C.
HERNANDEZ, Christa Ysabel T.
UY, Justine Nicole C.

Briane Paul V. SAMSON, PhD
Adviser

October 7, 2025

Abstract

Accurate estimation of train travel times remains a challenge due to various dynamic factors, including dwell times, real-time station congestion, operational delays, and weather conditions. Existing models, which primarily rely on distance-based calculations, fail to account for these real-world complexities, leading to inaccurate predictions. This study proposes a hybrid deep learning model that integrates Multi-Attention Graph Neural Networks (MAGNNs) and Long Short-Term Memory Networks (LSTMs) to improve travel time estimation in the Philippine Light Rail Transit system. MAGNNs capture the structural dependencies of station networks by assigning weighted delays to each node, allowing for a more precise representation of congestion and delay effects. Meanwhile, LSTMs enhance historical data utilization by identifying temporal patterns in travel time fluctuations. By leveraging real-world operational data, we assess the model's ability to produce accurate travel time predictions under varying conditions. The data processing pipeline begins with data collection, cleaning, handling of missing values, wavelet transformation, and normalization using max-min normalization. The data is then split into a training set (75%) and a test set (25%), followed by instance-based transfer learning using LSTM and MAGNN, task decomposition, parameter updates, model retraining, and finally, prediction and performance evaluation using error rate analysis. This approach offers a data-driven solution for optimizing rail transit operations and improving commuter experience.

Keywords: Train travel time estimation; Multi-head Attention Graph Neural Networks; Long Short-Term Memory Networks; Light Rail Transit

Contents

1 Introduction **1**

1.1 Background of the Study 1

1.2 Research Objectives 4

1.3 Scope and Limitations of the Research 4

1.4 Significance of the Research 5

2 Review of Related Literature **7**

2.1 Travel Time Estimation 7

2.1.1 Conceptual Definition 7

2.1.2 Considerations for TTE Accuracy 8

2.1.3 Data Sources for TTE Applications 9

2.1.4 Recent Developments in TTE 13

2.1.5 Modality 15

2.1.6 Challenges of TTEs 16

2.2 Discussion of Deep Learning and Hybrid Models 18

2.2.1 Conceptual Definition of Deep Learning Models 18

2.2.2 Deep Learning Modelling Techniques 18

2.2.3 Benchmark Models 25

2.2.4 Latest Mechanisms to Improve Model Performance 27

2.3 Assessment Strategies 30

2.3.1 Performance Evaluation Metrics 31

2.3.2 Feature Ablation Studies 33

2.3.3 Comparison to Selected Benchmark Model 34

3 Theoretical Framework **35**

3.1 Graph Theory 35

3.2 Graph Neural Networks (GNNs) 38

3.3 Multi-Attention Graph Neural Networks (MA-GNNs) 39

3.4 Recurrent Neural Network (RNN) 41

3.5 Long Short-Term Memory (LSTM) 42

3.6 Multi-View Graph Construction Module 43

3.7 Multi-Mask Attention Module 44

3.8 Multi-Head Attention Graph Module (Weighted Adjacency Matrix Module) 45

3.9 Spatial and Temporal Attention Modules 48

3.10 Multi-Task Learning Module 50

4 Methodology **52**

4.1 Data Collection 53

4.2 Data Preprocesing 58

4.2.1 Data Separation 60

4.2.2 Consideration of Other Datasets 61

4.3 Software Design and Implementation 62

4.3.1 Model Overview 62

4.3.2 Multi-Attention Graph Neural Network (MA-GNN) Components 63

4.3.3 Long-Short Term Memory(LSTM) Components 79

4.3.4 Multi-Task Learning Module Components 85

4.3.5 Model Training and Refinement 87

4.4 Model Validation 88

4.4.1 Performance Evaluation Metrics 89

4.4.2 Feature Ablation Study 90

4.4.3 Module Ablation Study 91

4.4.4 Benchmark Models with Statistical Tests 91

A Research Ethics Documents **96**

B Turnitin Summary Report **108**

References **110**

List of Figures

3.1 Graph Theory 35

3.2 Types of Graphs 36

4.1 Overview Diagram of Methodology 52

4.2 Canberra Light Rail Map 54

4.3 Canberra Light Rail Stations 54

4.4 A sample adjacency matrix between segments in the train line. . . 75

4.5 An Illustration of the Concatenation of Mask Matrices 79

4.6 Overview of LSTM 81

4.7 LSTM cell 82

4.8 Model Validation Framework. 88

List of Tables

2.1 Comparative analysis of historical data, real-time data, and dynamic features 11

2.2 Summary of the Current State of the Art in Travel Time Estimation 14

2.3 Summary of the Machine Learning Approaches for Travel Time Estimation 21

2.4 Benchmark/Baseline Models vs MA-GNN-LSTM 25

2.5 Summary of the Performance Evaluation Metrics 31

4.1 Description of the Features of the Final Dataset 56

4.2 Feature Ablation Study Configurations 90

4.3 Comparison of benchmark models based on learning paradigms . . 91

Chapter 1

Introduction

1.1 Background of the Study

Railway transportation is essential in order for society to function properly, despite its role, railways continue to face a multitude of issues like train punctuality, provision of safety information to passengers, preparedness of staff to assist travelers, station convenience, accessibility of channels for feedback and complaints, as well as the adequacy of train availability (Fajardo, Hernandez, Passi, Vigonte, & Abante, 2023). Train Systems reduce automobile dependencies which helps lower traffic congestion and emissions. Well-functioning rail systems improve connectivity, reduce travel time, and enhance workforce productivity by providing reliable commuting options (Miller, de Barros, Kattan, & Wirasinghe, 2016). As demonstrated by a predictive model developed by physicists Vincent Verbavatz and Marc Barthelemy, the effectiveness of rail systems depends on well-designed infrastructure, service frequency, and accessibility (Buchanan, 2019). Real-time passenger flow monitoring and predictive analytics are needed to manage peak-hour congestion. Understanding travel demand through data analytics helps optimize train schedules and improve service performance (Welch & Widita, 2019). During peak-hours, train frequency is increased, but congestion at platforms often leads to longer waiting times, as passengers may not be able to board the first arriving train. Factors such as varying waiting times, transfer delays, and operational inefficiencies make it difficult to ensure consistent travel times (Zeng, Fu, Arisona, Erath, & Qu, 2014). Unpredictable external influences, such as severe weather and emergencies, further complicate urban rail operations, making distance-based models less reliable for forecasting. Moreover, passenger flow variations add another layer of uncertainty, as traditional models struggle to incorporate dynamic real-time fluctuations (L. Zhu, Chen, Wang, Yu, & Tang, 2023). Basic distance-

based calculations fail to consider real-world factors like weather, passenger flow fluctuations, and special events, all of which impact travel times (Noursalehi, Koutsopoulos, & Zhao, 2021). This emphasizes the need for more innovative solutions to improve travel time estimations under real-world conditions in order to prevent the uncertainties of travel times to not only hinder commuter convenience but also pose a barrier to economic productivity.

This study examines real-world train operations and focuses on real-world factors that influence travel times along transit routes. By analyzing how these elements interact in live settings, the research aims to develop improved estimation models to enhance the efficiency of urban rail systems globally. Train travel systems face persistent disruptions due to maintenance issues, policy changes, operational inefficiencies, infrastructure failures, adverse weather, and human error (Endo, Edelenbos, & Gianoli, 2024) (Huang, Wen, Fu, Peng, & Li, 2020). These unpredictable and interdependent factors complicate the task of estimating travel times accurately (Taslimi, Sarijaloo, Liu, & Pardalos, 2022). As a result, delays not only disrupt immediate train schedules but also ripple across interconnected transit networks, causing missed transfers and degrading the overall passenger experience (Lapamonpinyo, Derrible, & Corman, 2022). Over time, such inefficiencies lead to broader economic losses, including lost productivity, higher fuel consumption, increased health costs, and missed investment opportunities (Mijares, Regmi, & Yai, 2014).

Improving the accuracy of travel time estimates directly addresses these challenges by enabling better operational planning, minimizing service disruptions, and allowing timely deployment of maintenance and backup resources (R. Shi, Xu, Li, & Li, 2021). Enhanced reliability boosts service quality and dependability, reduces commuter frustration, and strengthens public trust in urban rail systems.

Train delay patterns have been widely studied, with researchers classifying their root causes into categories such as station-related, train-related, operational, and timetable-based issues (W.-H. Lee, Yen, & Chou, 2016). These range from boarding congestion and equipment failures to infrastructure maintenance and inefficient scheduling. Addressing these challenges requires not only operational improvements but also better data utilization.

In response, recent studies have emphasized the importance of predictive analytics and machine learning to anticipate delays and optimize rail performance. Spanninger et al. (2022) (Spanninger, Trivella, Büchel, & Corman, 2022) highlight four key types of data essential for delay prediction including Historic Train Movements (HTM), Infrastructure Information (INI), Operational Information (OPI), and External Factors and Weather (EFW). These categories offer a structured approach to understanding the input variables necessary for accurate and

dynamic modeling of train operations.

These developments reflect a broader shift from reactive to proactive operations in railway systems. However, existing models still struggle to handle the complexity and variability of real-world conditions. Furthermore, research has increasingly shifted toward developing advanced prediction methods that can process these multifactorial inputs. Spanninger et al. (2022) (Spanninger et al., 2022) and Fu et al. (2024) (Q. Fu et al., 2024) distinguish between event-driven and data-driven models. J. Wu et al. (2021) (J. Wu et al., 2021) note that prediction models are generally classified as either deterministic or stochastic, which model uncertainty through probability distributions. The unpredictable interplay between infrastructure constraints, human factors, and environmental influences makes reliable travel time estimation a persistent challenge.

This study contributes to ongoing efforts by focusing on the integration of real-world variables into predictive modeling, aiming to develop a more responsive and robust framework for estimating train travel times in dynamic urban contexts. Traditional travel time estimation methods are no longer sufficient for addressing the operational complexities of modern rail systems. Most existing models depend heavily on scheduled timetables and distance-based calculations. While useful under ideal conditions, these approaches often overlook the unpredictable and interdependent factors that cause real-world delays (Mijares et al., 2014) These oversights can cascade into broader system inefficiencies, leading to unreliable travel time estimates and diminished service quality.

Despite recent advances in machine learning and predictive analytics, many current approaches still fail to capture the full range of dynamic variables that affect train operations. Event-driven models offer insight into operational sequences, while data-driven models can learn from historical patterns. However, each approach alone has limitations in modeling uncertainty and responsiveness to real-time disruptions. There remains a need for more adaptive models that can synthesize multiple data types and account for the multifactorial nature of delay propagation.

In light of these challenges, the study addresses the following research question: How can the accuracy of train travel time prediction be improved through the integration of real-world operational factors using a hybrid modeling approach? This study also seeks to answer the following sub questions. First, how can the complex structural dependencies of station networks be effectively modeled to reflect real-world operational dynamics? Second, in what ways can historical travel time data be used to identify and capture patterns and fluctuations that impact train operations? Third, how can we improve traditional travel time estimation methods in terms of accuracy and reliability under various real-world operating

conditions? These research questions form the basis for the study's objectives, which are to evaluate the overall effectiveness of the hybrid model, to assess the accuracy in estimating travel times.

1.2 Research Objectives

The **general objective** of this study aims to develop an effective hybrid deep learning model to estimate travel times on the Transit system.

The **specific objectives** of this research are as follows:

- Gather, preprocess, and analyze real-world transit data
- Model the structural dependencies of station networks using MAGNNs by assigning weighted delays to each node
- Enhance historical data utilization through LSTMs to identify temporal patterns in travel time fluctuations
- Address error accumulation through the Multi-Task Learning Module by utilizing both local and global predictions
- Assess the accuracy of a hybrid MAGNN + LSTM model applied to train transit systems in estimating travel times

1.3 Scope and Limitations of the Research

Bao et al., 2021 (Bao, Li, Li, Shi, & Ding, 2021) considers the spatial and temporal characteristics of a train transit system as important for accurate train delay predictions and therefore, train travel estimations. This study focuses on the development and evaluation of a hybrid modeling approach that integrates real-world operational factors that affect train travel times. By representing the transit system as a network, the estimation model will be able to use the spatial dependent variables to more accurately predict the delays of the network. Schilde et al., 2014 (Schilde, Doerner, & Hartl, 2014) mentions that travel time estimations fail to be accurate if assumed that they are non-stochastic or time-independent. Along with modeling the transit network, the estimation model must also be able to identify patterns based on temporal, in other words time-dependent, variables to provide a better or more accurate estimate.

The use of real-time data is beyond the current scope due to access and technical constraints. To maintain feasibility, the research will rely on historical data sourced from open-access repositories and and will focus on datasets that include key factors that fall under the four key types of data that Spanninger et al. (2022) (Spanninger et al., 2022) mentions as essential for delay prediction. The dataset used to train the hybrid model will cover the past three years, with a focus on post-pandemic operational conditions to ensure recency and relevance of patterns. As data availability may vary across regions, the scope of the analysis will depend on the completeness and granularity of the datasets retrieved.

The implementation will be limited to an experimental framework within a Python-based notebook environment. Due to the study being limited to such an environment, implementation into a software application or website will not be accomplished by the end of this study. The full deployment into an operational software or mobile application is outside this project's scope. While model implementation will be exploratory and conducted in a controlled research environment, the findings are intended to support future improvements in predictive accuracy for train systems at a broader, more scalable level.

1.4 Significance of the Research

This study contributes to the field of Computer Science, particularly in transportation informatics and intelligent transportation systems (ITS), by exploring a hybrid deep learning modeling approach to improve the accuracy and efficiency of travel time predictions. This approach improves on existing models that often rely too much on fixed schedules and historical data, which do not account for real-time changes such as delays, congestion, or weather conditions. Most current models for estimating travel time are deterministic, meaning they use fixed rules and assumptions that do not easily adjust to real-world conditions.

Existing models often fall short in addressing the variability introduced by factors such as passenger flow fluctuations, infrastructure constraints, weather conditions, and unforeseen disruptions. These shortcomings result in inconsistent travel time estimates that can frustrate commuters and reduce trust in public transportation systems. This study seeks to overcome these limitations by identifying and evaluating a hybrid setup capable of synthesizing multiple data types to produce more reliable predictions under dynamic conditions.

The significance of this research lies in its potential to enhance train systems across multiple dimensions. Commuters will benefit from more reliable travel time

predictions, allowing them to plan their trips with greater confidence. Better predictability can also encourage more people to choose public transportation, which helps reduce traffic congestion and contributes to a cleaner environment by lowering vehicle emissions. These improvements can collectively reduce delays, lower operational costs, and increase system responsiveness.

For transit authorities, more accurate travel time estimates can improve system efficiency by supporting better traffic management, resource allocation, and train scheduling. This can minimize delays, reduce operational costs, and improve overall service quality. Urban planners can also use the insights gained from this model to design more efficient and resilient transportation systems. Additionally, this model can be adapted to suit the specific needs of different urban rail environments which enables more resilient and scalable transit solutions.

This study is necessary because traditional travel time models do not effectively account for real world conditions. By addressing the limitations of existing methods and exploring how deep learning techniques can enhance prediction accuracy, this research provides a more practical and adaptable solution to real-world challenges in urban transit.

Chapter 2

Review of Related Literature

2.1 Travel Time Estimation

In modern transportation systems, predicting how long it takes to travel from one location to another is important to improve travel experience. Enhancing public transportation reliability, and commuter experience. Whether it is applied to road traffic, bus routes, or rail transit systems, accurate travel time is crucial for planning, decision making, and operational efficiency. With increasing population and technological advancements, the need for intelligent and dynamic travel time estimation models has become more important. This section reviews various studies on travel time estimation, starting with its conceptual definition, followed by accuracy considerations, recent developments, data sources, challenges, and modality applications.

2.1.1 Conceptual Definition

Travel Time Estimation (TTE) is the process of determining how long does it take to travel between two points, often utilizing statistical, time-series, or more recently, machine learning models. It provides valuable insights into system efficiency, user experience, and planning. As defined in transport forecasting literature, travel time measures trip duration and reflects the overall effort by travelers. These measures are favored over simple volume-based metrics because they accurately capture the real-world experience and performance of a transportation system (of Sciences et al., 2016). In public transportation, travel time becomes a critical performance indicator used to assess factors like schedule compliance,

service reliability, and passenger satisfaction (Nesmachnow, Massobrio, Guridi, Olmedo, & Tchernykh, 2023). Evaluations of road and rail systems consistently use travel time estimation as a core metric in measuring operational quality and planning public transport improvements.

2.1.2 Considerations for TTE Accuracy

Research in Intelligent Transportation Systems (ITS) using machine learning shows a wide range of accuracy levels, primarily due to the diverse modeling approaches of the studies. Machine learning algorithms can be extensively customized to align with specific application goals or proposed solutions, contributing to this variation in performance. (Ping, 2024) explains that the efficiency of machine learning models varies depending on 6 key considerations, namely, the type of problem, dataset size, feature space, computational efficiency, interpretability, algorithm complexity and assumptions. According to (*StudySmarter*, 2025a), computational efficiency refers to the effectiveness of an algorithm in terms of the usage of resources such as time and space. On the other hand, (*Domino*, 2025b) explains that an interpretable machine learning algorithm is one where humans can clearly understand the insights derived from the data or the patterns learned by the model. These two considerations may not directly impact a model's accuracy but significantly influence the practicality and relevance of its outputs in real-world applications. With that said, factors that directly influence model accuracy include the type of problem, dataset size, feature space, and the complexity and assumptions of the chosen algorithm.

Machine learning (ML) problem types typically fall under 6 categories: regression, clustering, classification, association, anomaly detection, and reinforcement learning use cases (Yalamanchili, 2023). Travel time estimation using machine learning is considered a regression problem, as it involves predicting continuous numerical values. As such, to optimize TTE accuracy and system performance, the dataset size, its characteristics, and the chosen algorithms must be well-suited for handling regression problems effectively.

In a study by (H. Wang, Tang, Kuo, Kifer, & Li, 2019), it was found that even a simpler algorithm outperformed more complex models when supplied with large-scale trajectory data from taxi networks. Their findings highlight the significant impact of dataset size on model accuracy, particularly for problems that rely heavily on numerical and historical data. This suggests that, for TTE models, access to a large and rich dataset is crucial for capturing urban dynamics and producing more accurate predictions. Feature selection, also known as defining a model's feature space, plays a crucial role in TTE accuracy as it directly impacts

the algorithm's ability to process data, recognize patterns, and deliver precise predictions. While some algorithms perform well with high-dimensional feature spaces, others are better suited to simpler, lower-dimensional inputs. As such, careful consideration of the feature space aligned with the algorithm's complexity and computational efficiency is essential for optimizing performance and accuracy (Ping, 2024).

The assumptions an algorithm makes about a dataset significantly influence the accuracy of its predictions. As noted by (Ping, 2024), different models depend on different assumptions about the data distribution, which can impact the trends and patterns they identify and ultimately affect their predictive performance. This is especially critical in real-world applications like travel time estimation, where the algorithm must make assumptions that accurately reflect the nature of the real-life data to produce reliable results. Moreover, algorithm complexity significantly impacts accuracy, as it determines how effectively a model can manage noisy or incomplete data (Ping, 2024). Models that are not equipped to handle such imperfections are more likely to produce inaccurate predictions due to issues like biased results from on skewed predictions or false discoveries (Andersen, 2024).

2.1.3 Data Sources for TTE Applications

Travel time estimation relies on diverse data sources to capture travel patterns across road, rail, and bus networks. High-quality, granular, and relevant data are crucial for accurate estimation, as they enable models to reflect real-world and dynamic conditions (L. Zheng, Ma, & Wang, 2024; H. Wang et al., 2019). Historical data, such as past trip records, provide a foundation for understanding typical travel time and pattern, while real-time data like GPS feeds capture current traffic conditions (Yuan, Li, & Bao, 2022; L. Zheng et al., 2024). The main difference is that historical data offer stability and long-term trends, while real-time data reflect immediate and often unpredictable changes. Moreover, dynamic features, variables that change over time, such as weather, dwell times at stops, operational delays, headway (time between vehicles), and crowd congestion are increasingly integrated to improve estimation accuracy (L. Zheng et al., 2024; Zou, Yang, & Zhu, 2020). The main strength of using historical data is its availability and capacity to model long-term trends but may miss sudden disruptions, real-time data are more flexible compared to historical data but it can be noisy or incomplete (L. Zheng et al., 2024; Sanaullah, Quddus, & Enoch, 2016). When available, dynamic features enhance responsiveness to current traffic conditions but require complicated data integration and can introduce complexity (L. Zheng et al., 2024; Zou et al., 2020). Table 2.1 shows the comparative analysis

of historical data, real-time data, and dynamic features.

Table 2.1: Comparative analysis of historical data, real-time data, and dynamic features

Data	**Advantages**	**Limitations**	**Study**
Historical Data	Captures long-term trends and periodic patterns	May not reflect current disruptions or unexpected events	Semi-supervised LSTM with historical feature fusion attention for temporal sequence dynamic modeling in industrial processes (Y. Tang et al., 2023); Attention-mechanism based DiPLS-LSTM and its application in industrial process time series big data prediction (Y. Wang, Qian, & Qin, 2023)
	Readily available and easy to collect	Can become outdated if traffic patterns change	
	Useful for baseline and trend analysis	Lacks adaptability to real-time fluctuations	
Real-time Data	Reflects current conditions	Requires strong, timely data collection and integration	Geostatistics on real-time geodata streams—high-frequent dynamic autocorrelation with an extended spatiotemporal moran's I index. (Lemmerz, Herlé, & Blankenbach, 2023)
	Enables immediate response to ongoing changes	Data can be noisy, incomplete, or unavailable	
	Supports adaptive and responsive estimation	Increases computational and infrastructure demand	
Dynamic Features	Captures rapidly changing factors	Require advanced extraction and integration methods	Efficient dynamic latent variable analysis for high-dimensional time series data. (Y. Dong, Liu, & Qin, 2019); Bayesian learning for dynamic feature extraction with application in soft sensing.. (Y. Ma & Huang, 2017); Attention-mechanism based DiPLS-LSTM and its application in industrial process time series big data prediction.. (Y. Wang, Qian, & Qin, 2023)
	Enhance prediction accuracy, especially in unstable environments	Model complexity and data quality challenges	
	Allow for refined, context-aware predictions	Depend on availability and reliability of dynamic data sources	

Building upon the previous section, dynamic features are used to enhance adaptability and improve accuracy of predictions by capturing real-time conditions. These features influence outcomes by reflecting inconsistencies in system behavior and external factors. Dynamic features such as weather, dwell times, operational delay, headway, and crowd congestion play crucial roles in transportation and crowd management. Weather refers to environmental conditions like temperature, wind, and rain, and is used to predict delays and adjust operations, as it can significantly affect schedules and safety, though its unpredictability and data variation are challenges (Q. Li, Jing, & Dong, 2023; S. Yang & Qian, 2019). Dwell times, defined as the time vehicles spend at stops are operational features used for scheduling and capacity estimation. They are highly influenced by passenger flow and can improve dynamic scheduling but are sensitive to sudden surges and difficult to predict during peak times/hours (Pang, Wang, Wang, Li, & Peng, 2023). Another feature is the operational delay, these delays involve extra time from disruptions, directly impacting travel time and requiring quick response but are often unpredictable (Q. Li et al., 2023; Pang et al., 2023). Headway, the interval between vehicles on the same route is essential for maintaining service regularity and controlling crowding. While it helps reduce congestion, it demands constant monitoring and is vulnerable to disruptions (Noursalehi et al., 2021) (Pang et al., 2023). Finally, crowd congestion, defined as the density of people in a space, is used for safety management and flow prediction. With high congestion slowing movement and increasing risk, real-time detection enables better crowd control, but still faces challenges in data collection and privacy (Ali, Zhu, & Zakarya, 2022; Tripathy & Srivastava, 2020; Noursalehi et al., 2021; Q. Wang, Wan, & Yuan, 2017).

For each dynamic feature, the most common data used are as follows. Weather features typically include temperature, rainfall, wind speed, and sometimes humidity, as these variables are most strongly associated with transportation delays and congestion (J. Wu & Zhang, 2024; X. Zhang & Chen, 2019; Borsky & Unterberger, 2019). Dwell times are measured by recording the duration vehicles spend at stops or stations, often using time stamped entry and exit data from sensors or ticketing systems (AlHadidi & Rakha, 2020; Kuipers, 2024; Palmqvist, Tomii, & Ochiai, 2020). Operational delay data are usually derived from comparing scheduled versus actual arrival and departure times, capturing the extra minutes or seconds caused by disruptions or unexpected events (Artan & Şahin, 2021). This approach is widely used in both rail and air transport to monitor punctuality and assess the impact of delays on operations, as actual operational performance is measured against the schedule to identify differences in travel time and their causes (Nosedal-Sánchez, Piera-Eroles, & Lebbink, 2020). Headway is calculated using the time intervals between consecutive vehicles on the same route, this measurement is commonly tracked through GPS or automated vehi-

cle location systems, as well as other automated sensors that record the precise travel times of vehicles (Badhrudeen, Ramesh, & Vanajakshi, 2016; Elmorshedy, Smirnov, & Abdulhai, 2024; J. Du & Ma, 2024). Crowd congestion is assessed using data on the number of people in a given area or vehicle which can be collected through various methods such as sensors, video analytics, mobile device tracking, and crowd-sourced reports (S. Liu, Zhang, Ge, Li, & Long, 2025; Wong & Law, 2023; Minh, Pham-Nguyen, Tan, & Long, 2019; Bek & Monari, 2016).

In addition to the dynamic features mentioned earlier, several other features are considered in travel time estimation. These include real-time traffic speed and flow which capture the current movement and density of vehicles on roads, traffic incidents such as accidents or road works which can cause sudden disruptions, and spatio-temporal dependencies reflecting how conditions on one road or a one time can influence others nearby (Yuan et al., 2022; Jin, Yan, Li, Li, & Huang, 2023; Q. Wang, Xu, Zhang, & Li, 2021; Zhou et al., 2023; Fang, Sun, Chen, Hu, & Gao, 2024). Moreover, features like route direction, departure time, and proximity of intersections are important because they help account for variations in travel patterns and congestion at different times or specific areas in a transportation network. Studies show that spatial features such as proximity to intersections and the direction of travel influence traffic flow and congestion because they often share similar traffic patterns (Yuan et al., 2022; H. Dong et al., 2025). Temporal features including departure time, capture daily and weekly record reflecting rush hours and other recurring patterns that affect congestion levels (Yuan et al., 2022; S. Yang & Qian, 2019). Advanced machine learning models for predicting congestion and travel times frequently used features like location, driving selection, and time to improve accuracy, demonstrating that these factors are crucial for understanding and managing dynamic traffic conditions (H. Dong et al., 2025; S. Yang & Qian, 2019).

2.1.4 Recent Developments in TTE

Current travel time estimation reflects rapid advancements in machine learning, spatiotemporal data integration, and real-time traffic analytics. With increasing urbanization and the creation of sensor technologies, modern travel time estimation systems prioritize accuracy, stability, and real-time adaptability especially in dynamic and congested environments. These systems increasingly rely on diverse and merged data sources, advanced learning models, and are deployed in various real-world applications. Table 2.2 summarizes the modern travel time estimation, highlighting core developments, techniques and methods.

Table 2.2: Summary of the Current State of the Art in Travel Time Estimation

Aspect	**Key Highlights**	**References**
Data Sources and Integration	- uses GPS data and Matrix Factorization (to assign missing segments and time intervals) - solves sparsity problem - explores various sparse sensor data - loop detectors, floating-car data, bluetooth sources - concluded that the combination of floating-car and loop detector data provides best speed and travel time accuracy	Real-time travel time estimation using matrix factorization (Badrestani et al., 2019); Multi-sensor data fusion for accurate traffic speed and travel time reconstruction (Kessler, Rempe, & Bogenberger, 2021)
Advanced Models, Techniques, and Methods	- proposes STDGNN, dual graph structure capturing both node (intersections) and edge (road segments). - achieve improvements over basic methods on real-world approach - introduces PathCNN, transforming paths into image-like representations to capture both spatial and temporal patterns - proposes GNN + Resnet - extracts optimal feature points from the existing dataset using a stacked and sparse autoencoders	Spatio-temporal dual graph neural networks for travel time estimation. ACM Transactions on Spatial Algorithms and Systems (Jin, Yan, Li, Huang, & Li, 2024); Deepist: Deep image-based spatio-temporal network for travel time estimation (T.-y. Fu & Lee, 2019); A Hybrid DNN Model for Travel Time Estimation from Spatio-Temporal Features (Rajagopal, Kumar, Samui, Kaloop, & Shahdah, 2022)
Real-world Applications	- predicts travel time distributions not just point estimates - improves accuracy and dispatch decisions - provides additional travel information - hybrid markovian framework - uses Maximum Likelihood Estimation(MLE) - applied to GTFS real-time data - compares different methods such as Support Vector Regression (SVR), Randomized K-Nearest Neighbor Regression with Spherical Distance (RKNNRSD), and Incremental Polynomial Regression (IPR) - real-time travel time and speeding alerts	Uncertainty-aware probabilistic travel time prediction for on-demand ride-hailing at didi (H. Liu, Jiang, Liu, & Chen, 2023); Real-time Bus Travel Time Prediction and Reliability Quantification: A Hybrid Markov Model (Y. Sun, Spall, Wong, & Zhao, 2025); Travel time prediction in real time for GPS taxi data streams and its applications to travel safety. Human-Centric Intelligent Systems (Putatunda & Laha, 2023)

Before the start of data-driven and machine learning-driven approaches, manual travel time estimation methods served as the foundation for early traffic analysis and trip planning. These traditional approaches relied solely on straightforward calculations using basic parameters such as distance and speed, often performed by hand or with simple calculators. The most commonly used formula was Travel Time = Distance / Speed, which assumes constant speed throughout the entire route and provides a rough yet quick estimation. For scenarios requiring more precision, such as when acceleration or deceleration played a role, manual methods would incorporate mathematical calculations like the Maclaurin series to approximate travel time. By increasing the number of terms in the series, the accuracy of the estimation improved significantly (Singgam, Silalahi, Tampubolon, & Harliana, 2024). While manual methods were limited in adapting various speed, stops, and delays, they were efficient and accessible for simple, stable conditions. Despite their simplicity, these techniques laid the groundwork for more accurate travel time estimation systems.

2.1.5 Modality

Travel time estimation across different transportation modalities such as road, bus, and rail has been studied, particularly with the advancement of deep learning and spatio-temporal modeling techniques. Each modality presents different challenges and dynamics that shape the decision of predictive models and data.

Travel time estimation differs across road, bus, and train systems due to their structural and operational characteristics. For road, estimation models must take into account highly dynamic factors such as traffic congestion, intersections, and even driver behavior, studies often use deep learning or probabilistic approaches to capture the uncertainty and spatio-temporal dependencies in vehicle movement (James, 2021; D. Wang, Zhang, Cao, Li, & Zheng, 2018). On the other hand, bus travel time estimation appears to be more complex than road, as it involves not only the variability of road traffic but also additional factors such as scheduled stops, crowd level, and dwell times, advanced models often segment journeys into riding and waiting components using machine learning and historical data to improve accuracy (He, Jiang, Lam, & Tang, 2018; He, Jiang, Lam, & Sun, 2020; Dai, Ma, & Chen, 2019). In contrast with road and bus, train travel time estimation is structured by fixed tracks, scheduled timetables, and operational constraints such as track availability, network congestion, and resource allocation. Optimization models, such as mixed integer programming are commonly used to minimize difference from planned schedules while taking into account delays and network restrictions (Taslimi et al., 2022).

For road, studies have leveraged sensors, GPS, and trajectory data to model travel times under dynamic conditions. Graph-based deep learning methods, especially spatio-temporal Graph Neural Networks (GNNs) have become more popular for modeling the non-linear and complexity of road networks. For instance, (X. Zhang & Chen, 2019) proposed a GNN with temporal attention to improve accuracy in urban road travel time estimation. In bus systems, recent studies have incorporated attention mechanisms and external factors such as weather and traffic flow. Attention mechanisms, originally popularized in visual and language tasks, but in recent years it has been adapted to efficiently process complex, dynamic data, allowing models to identify the importance of different features or time steps (time intervals) which is particularly useful in transportation sector where conditions change rapidly (Guo et al., 2022). On rail transit systems, while operation takes place on fixed tracks, it still faces challenges such as dwell times, cascading delays, and congestion effect. Recent research highlights the importance of modeling these complexities using historical data and dynamic features, including passenger flow and external factors to improve travel time and delay estimation. Advanced machine learning approaches such as multi-attention mechanisms have been introduced to predict metro segment travel times by incorporating external factors (L. Zhu et al., 2023). Additionally, integrating temporal and real-time data has proven crucial for handling peak-hour variations and mitigating the impact of crowding and disruptions (L. Zhu et al., 2023; J. Tang, Xu, Luo, & Ng, 2021).

2.1.6 Challenges of TTEs

Estimating travel time accurately is crucial in intelligent transportation systems, logistics, and mobility planning. With the advancement of machine learning (ML) and deep learning techniques, travel time estimation has significantly improved, however various challenges still evident. These challenges come from the the deep and complexity of traffic systems, data limitations, computational demands, and broader concerns such as privacy across regions.

One major challenge is in capturing complex spatial-temporal dependencies. Travel time is influenced by numerous factors, including distance, road type, real-time traffic flow, speed variations, and historical patterns. Modeling how traffic conditions evolve over time and interact across different road segments is difficult, as these interactions often demonstrate non-linear and dynamic behavior. Accurately representing these relationships in machine learning models, particularly in urban environments with crowded networks is essential but rigorous (L. Yang, Jiang, & Zhang, 2022; Liao et al., 2024; Han et al., 2021).

Data quality, coverage, and unexpected events further complicate the estimation process. Accurate estimations rely on high-quality, comprehensive trajectory data collected from sensors or GPS. However, in many cases, the data may be incomplete due to low access rates, sparse sensor networks, or technical limitations (Alrukaibi, Alsaleh, & Sayed, 2019). Additionally, rare or unexpected events such as accidents, road closures, or severe weather are difficult for models to anticipate as they deviate from typical traffic patterns and are neglected in training datasets (Abdollahi, Khaleghi, & Yang, 2020).

Another issue is model generalization and transferability. Travel time models trained on one city or set of conditions often perform poorly when applied elsewhere due to differences in road layouts, traffic behavior, and temporal traffic patterns. Developing models that can adapt to different environments with small data is still a challenge. This is particularly important for scaling machine learning based systems across regions without having to start from scratch (C. Wang et al., 2022; Z. Zhang et al., 2022).

Feature representation and learning present additional difficulties. Extracting relevant, multi-dimensional features from raw trajectory and road network data is complex. Effective representation must account for spatial and temporal dependencies as well as semantic information like road intersections (Abdollahi et al., 2020; Liao et al., 2024; Han et al., 2021). Many models also struggle to integrate both Euclidean (distance-based) and non-Euclidean (graph-based) characteristics of road networks, leading to poor performance in diverse routes (Han et al., 2021).

Computational and memory efficiency is another concern. Current deep learning models for travel time estimation often demand significant resources for both training and reasoning, making them impractical for real-time applications or large-scale deployment. Balancing model complexity with performance is critical for ensuring responsiveness in dynamic traffic environments (H. Zhang et al., 2024). Furthermore, privacy and data sharing raise ethical and technical concerns. User data is sensitive and often governed by strict privacy regulations. This limits the ability to aggregate data across regions or organizations, particularly in federated learning environments where models are trained collaboratively. Ensuring data privacy while enabling effective learning remains an unresolved issue (Z. Zhang et al., 2022). Lastly, most existing methods provide deterministic travel time estimates without considering the uncertainty in traffic conditions and user behaviors (Y. Y. Y. L. Zhu Yuanshao & Yu, 2022). While machine learning has enabled promising advances in travel time estimation, overcoming these challenges is essential for developing more accurate and precise solutions.

Moreover, a majority of travel time estimation solutions overlook the significance of capturing the structural model of transportation networks, often resulting

in a lower accuracy rating. Many studies often employ a homogeneous graph design and assume all paths contribute the same to the estimation, when in reality, these segments experience different traffic conditions and speeds which influence the travel time uniquely (Fang et al., 2024). While recent studies model transportation networks as graphs, most remain too simplistic to capture the complex interconnections between nodes (Elsir & She, 2023).

2.2 Discussion of Deep Learning and Hybrid Models

2.2.1 Conceptual Definition of Deep Learning Models

With the surge of real-time data and spatio-temporal datasets, machine learning has become the ideal approach in travel time estimation. Modern approaches integrate deep learning architectures and hybrid models to capture complex traffic dynamics with significantly improved accuracy. (Pouyanfar & Iyengar, 2018) defines deep learning as a machine learning method that uses multiple layers to represent the abstractions of data through several non-linear transformations. Deep learning models often mirror the human brain by using interconnected neurons to be able to identify complex relationships between features, often using artificial neural networks as its base (Janiesch & Heinrich, 2021). As such, most state-of-the-art machine learning models for modern travel time estimation applications utilize deep learning techniques, as they are known to improve model accuracy by incorporating a larger number of features and providing a deeper understanding of how real-time and context-specific factors influence travel time.

2.2.2 Deep Learning Modelling Techniques

The baseline models discussed below are among the most commonly used techniques by researchers for predicting travel time across various modalities and feature spaces. These models are used as standalone approaches, without being combined with other modeling techniques, and have achieved an accuracy rating of 70 percent or higher.

Artificial Neural Networks (ANNs) are interconnected computing systems modeled to simulate a human brain and its neurons, rendering it particularly effective for pattern recognition and predictive tasks (Hanafy, 2020). ANNs serve as the

foundational architecture for many travel time predictive models. Recent studies build upon this baseline by incorporating additional mechanisms such as attention modules, boosting, or hybrid architectures to enhance predictive accuracy and model performance.

For instance, a predictive model built upon the foundational structures of ANNs called Wide-Deep-Recurrent (WDR) networks combine wide linear models, deep neural networks, and recurrent neural networks to effectively capture spatio-temporal dependencies. These models have demonstrated state-of-the-art performance, surpassing industry standards and supporting deployments that benefit millions of users (Z. Wang, Fu, & Ye, 2018). On the other hand, knowledge distillation techniques have also emerged as a promising approach for improving model efficiency. By transferring knowledge from a complex, high capacity teacher model to a more lightweight student model, these approaches maintain high prediction accuracy while significantly reducing computational costs (H. Zhang et al., 2024). This balance of performance and efficiency makes them well-suited for real-time applications.

Machine learning has also been adapted for applications, such as improving travel time estimates in emergency response situations. For instance, specialized models have been developed to correct travel time estimation for ambulances by identifying and adjusting the error in mapping system estimates. This refinement enhances the accuracy of emergency planning and supports better resource allocation (Torres, Trujillo, Maldonado, & Vera, 2021). Graph Neural Networks (GNNs), including graph convolutional and attention-based architectures, have demonstrated strong capabilities in modeling the influence of adjacent road segments. These models effectively capture complex spatio-temporal dependencies within traffic networks, resulting in improvements in estimation accuracy and reductions in prediction error (L. Yang et al., 2022).

First proposed by Hochreiter and Schmidhuber in 2007, the Long-Short Term Memory (LSTMs) machine learning algorithm is designed to make more accurate predictions based on time-series data by introducing an LSTM cell with the ability to remember values over a non-fixed time interval and input, output, and forget gates to retain important information and trends based on the data flowing through the cell. (Qiu & Fan, 2021) With their specialized architecture, LSTMs are among the most widely used state-of-the-art models for capturing temporal dependencies in sequential data.

Representation learning has also become a foundation in modern travel time estimation by enabling models to extract meaningful patterns from complex, multi-dimensional data. Multi-faceted and multi-semantic path representation learning frameworks segment routes into structured sequences such as GPS trajectories,

road attributes, and semantic identifiers of road segments which will be then processed through advanced sequential learning algorithms and transformer-based encoders. The merging of spatial, temporal, and external factors enhances the model's ability to capture dynamic traffic behaviors, resulting in more precise and accurate travel time estimation (Liao et al., 2024; Han et al., 2021). With that, Temporal Attributed Graph-based models have emerged as powerful tools in travel time estimation, as they enable the joint learning of node representations that incorporate both dynamic traffic conditions and the structure of the road networks. By capturing these spatio-temporal dependencies, such models allow for the prediction of full travel time rather than single-point estimates, which enhances both the accuracy and reliability of travel time estimation (Zhou et al., 2023).

Another machine learning technique that has evolved is the Multi-task learning models which combine transportation mode recommendation with travel time estimation, allowing the model to capture the mutual influence between a traveler's mode choice and the travel time. This integration enables more effective spatio-temporal representations, leading to improved prediction accuracy (S. Xu, Zhang, Cheng, & Xu, 2022). In addition, meta-learning frameworks such as MetaTTE are designed to enhance generalization across different cities and varying traffic conditions. By employing encoder-decoder architecture and training on small data samples, these frameworks are capable of adapting to diverse scenarios while maintaining consistent performance (C. Wang et al., 2022). These approaches are summarized in Table 2.3, highlighting how these machine learning methods, models, and techniques, continue to improve travel time estimation.

Table 2.3: Summary of the Machine Learning Approaches for Travel Time Estimation

Approach / Model	**Dataset Used**	**Key Factors / Features Considered**	**Application / Impact**	**Findings**
Wide-Deep-Recurrent (WDR)	Historical vehicle travel data	Spatio-temporal features, route, departure time, effective features from floating-car data	Large-scale ETA prediction for navigation	Outperforms industry solutions
Knowledge Distillation (KDTTE)	Chengdu and Porto datasets	Features learned by teacher model, trajectory-based features, spatial-temporal data	Efficient and accurate TTE	34–86% accuracy improvement in Chengdu and Porto datasets
Multi-faceted Route Representation	Chengdu and Porto datasets	GPS trajectory sequence, road segment attributes, semantic road segment IDs	Route-based TTE	Improves accuracy over baseline. Achieved best MAE, MAPE, RMSE, PCC compared to baselines. Chengdu: MAE: 87.63, MAPE: 12.59%, RMSE: 128.91, PCC: 96.65%

Continued on next page

Table 2.3 – continued from previous page

Approach / Model	Dataset Used	Key Factors / Features Considered	Application / Impact	Findings
				Porto: MAE: 94.44, MAPE: 14.68%, RMSE: 135.21, PCC: 90.76%
Multi-semantic Path Representation	Real-world taxi trajectory datasets (Chengdu and Porto)	Spatial (Euclidean), non-Euclidean, segment/intersection semantics, length, type, time, co-occurrence	Route/path-based TTE	Outperforms previous methods. Achieved best MAE, MAPE, RMSE, PCC compared to baselines. Chengdu: MAE: 109.54, MAPE: 0.24, RMSE: 267.63, PCC: 0.7980 Porto: MAE: 73.60, MAPE: 0.61, RMSE: 100.82, PCC: 0.8998
Temporal Attributed Graph	Real-world large-scale datasets (Beijing and Shenzhen)	Dynamic traffic conditions, road network structure, congestion status, spatio-temporal dependencies	Distributional TTE, reliability	Outperforms other state-of-the-art and traditional models in MAE, MAPE, RMSE. Beijing: MAE: 106.63, MAPE: 10.96, RMSE: 177.22

Continued on next page

Table 2.3 – continued from previous page

Approach / Model	Dataset Used	Key Factors / Features Considered	Application / Impact	Findings
				Shenzhen: MAE: 96.92, MAPE: 12.64, RMSE: 146.11
Multi-task Learning (MTLM)	Real datasets (not specified)	Transportation mode, spatio-temporal dependencies, mode effect, path features	Smart city, multi-modal transport	Enhanced TTE accuracy
Meta-Learning (MetaTTE)	Chengdu and Porto datasets	Spatial and temporal dependencies, trajectory data	Multi-city, dynamic traffic	Outperforms nine baselines, improves 29.35% and 25.93% over best baseline in Chengdu and Porto. Chengdu: MAE: 236.38, MAPE: 23.69, RMSE: 745.11 Porto: MAE: 62.43, MAPE: 8.83, RMSE: 196.78 Deep learning methods outperform traditional methods in MAPE.

Continued on next page

Table 2.3 – continued from previous page

Approach / Model	Dataset Used	Key Factors / Features Considered	Application / Impact	Findings
Ambulance TTE Correction	EMS logs from Red Cross Tijuana	GPS-based ground truth, mapping system estimates (Google Maps, OSRM), error between estimates	Emergency response optimization	Coverage improved from 83.9% to 100% after ML-based corrections, improving reliability.
Graph Neural Networks (GNNs)	Real taxi trajectories (not specified)	Distance, road type, speed, traffic condition, adjacent segment influence, spatio-temporal factors	Ride-hailing, navigation	13.91% estimation error
Long-Short Term Memory (LSTMs)	Real historical freeway segments datasets	Spatio-temporal dependencies, environment feature	Short-term travel time predictions for freeway corridors	Maximum estimation error of 26.76% for long-term predictions

2.2.3 Benchmark Models

Identified as benchmark or baseline models, hybrids that use similar algorithms are a good point of comparison to a newly proposed model. Benchmark models, specifically similar to the proposed hybrid model, are listed in the table below, defined with its similarities and differences as shown in Table 2.4:

Table 2.4: Benchmark/Baseline Models vs MA-GNN-LSTM

Model	**Type**	**Similarities w/ MA-GNN-LSTM**	**Differences w/ MA-GNN-LSTM**
Historical Average (HA)	Statistical	Both can be used as baseline methods for travel time estimation	HA does not learn from data; no learning process or temporal modeling; purely averages past travel times
Support Vector Regression (SVR)	Machine Learning	Can capture non-linear relationships in travel time data	SVR does not model temporal or spatial dependencies explicitly; lacks sequence modeling compared to LSTM-based models
Long Short-Term Memory (LSTM)	Deep Learning	Both use LSTM to learn temporal dependencies; performs well on sequential data	MA-GNN-LSTM incorporates GNN and multi-attention mechanisms for spatial dependencies, which vanilla LSTM lacks
DeepTTE (CNN+LSTM)	Deep Learning	Both use deep learning to capture spatial-temporal patterns; uses LSTM for temporal learning	DeepTTE uses CNN for static map/trajectory encoding, while MA-GNN-LSTM uses GNN and multi-attention spatial module
Gct-TTE)	Transformer model	Both use graph-based learning (GNN or GCN) to model spatial dependencies; Utilizes attention mechanism	Gct-TTE uses transformer encoder to capture long-range temporal dependencies while MA-GNN-LSTM enhances LSTM with a global temporal module
T.R2_LSTM_A	Deep Learning & Transfer Learning	Utilizes temporal attention mechanism, LSTM, and addresses data sparsity	T.R2_LSTM_A uses instance-based transfer learning (T.R2) to address data scarcity, while MA-GNN-LSTM uses GNN's spatial attention module (with multi-masks) to handle data sparsity

Starting from a simple or low complexity model, the Historical Average (HA) model remains a widely used baseline in travel time estimation tasks despite its simplicity. It calculates future travel times based on the mean of previously observed travel times for the same route and time period (J. Ma, Chan, Rajasegarar, & Leckie, 2022a). While it lacks the capacity to capture real-time variability, temporal dynamics, or spatial interactions, its strength lies in its computational efficiency and ease of interpretation (Fan & Gurmu, 2015). This simplicity makes it a useful lower bound for performance comparison. In contrast, MA-GNN-LSTM is a complex model combining graph neural networks, attention mechanisms, and

LSTMs. Therefore, showing clear improvement over HA helps highlight the actual value of this added complexity. The deep learning models Support Vector Regression (SVR) and Long Short-Term Memory (LSTM) also provide increasingly sophisticated approaches to travel time estimation. Support Vector Regression (SVR) remains a strong traditional machine learning benchmark due to its ability to model non-linear relationships and its robustness to noise, particularly when data is clean and features are well-engineered (Huang, Wen, Fu, Peng, & Li, 2020). Despite its merits, SVR lacks the ability to model temporal dependencies or spatial correlations inherent in sequential transportation data, as it processes data points independently. It also does not scale well to large datasets or high-dimensional feature spaces without careful feature engineering. By including SVR as a baseline in this study, we establish a classical machine learning benchmark that is simple, computationally efficient, and historically effective. Along with SVR, the LSTM deep learning model will be set as a baseline, given its strong ability to model temporal sequences through gated memory mechanisms. Similarly, our proposed model also uses LSTM to capture temporal sequences but with the integration of a global temporal module added. Comparing MA-GNN-LSTM to a standalone LSTM allows us to evaluate the contribution of the additional modules, particularly the multi-head spatial attention via graph neural networks and the multi-task learning framework.

Lastly, the higher complexity models DeepTTE proposed by (D. Wang et al., 2018), Gct-TTE proposed by (Mashurov, Chopuryan, Porvatov, Ivanov, & Semenova, 2024), and T.R2_LSTM_A proposed by Guo et al. (J. Guo et al., 2024a) will be used for model comparison. According to Li et al. (M. Li, Feng, & Wu, 2024a), DeepTTE is widely regarded as one of the most accurate models for travel time prediction using GPS trajectory data; its strong empirical performance across multiple datasets makes it an ideal benchmark for evaluating the predictive power of newer models. Likewise to MA-GNN-LSTM, DeepTTE integrates both spatial and temporal modules, specifically, a geo-convolutional layer for spatial pattern learning and an LSTM layer for temporal sequence modeling. This similarity in modular design allows for a fair and meaningful comparison between the two models in terms of how each handles spatial-temporal dependencies. DeepTTE introduces a multi-task learning framework via attribute fusion, aiming to enhance model generalization and reduce overfitting (D. Wang et al., 2018). MA-GNN-LSTM, similarly, incorporates multi-task learning, though with additional enhancements such as graph-based attention and global temporal awareness. Benchmarking against DeepTTE helps highlight the advantages of MA-GNN-LSTM's expanded architectural features. The Gct-TTE model on the other hand leverages the self-attention mechanism in transformers to model global contextual relationships between different time steps, enabling it to capture long-term dependencies more effectively than LSTM (Mashurov et al., 2024). Additionally,

transformers allow for parallel computation, improving training efficiency over the inherently sequential LSTM structure. This will be a good benchmark model to our proposed model MA-GNN-LSTM as to counter the issue in LSTM, the global temporal module was added. Finally, Guo et al.'s T.R2_LSTM_A (J. Guo et al., 2024a) represents one of the most advanced and successful frameworks for train travel time prediction to date. It was specifically designed to address (1) data sparsity and (2) complex temporal dependencies, both of which our proposed model MA-GNN-LSTM aims to address as well. T.R2_LSTM_A addresses these key challenges by employing a two-stage transfer learning strategy (T.R2), the TrAdaBoost.R2 algorithm for instance-based reweighting, and an attention-enhanced Long Short-Term Memory (LSTM) network. Similarly, our proposed MA-GNN-LSTM framework tackles these challenges by explicitly modeling spatial dependencies among stations using a multi-attention graph neural network (GNN), integrating external attributes such as weather, congestion, and operational delays, and applying both local and global temporal attention mechanisms alongside an LSTM to capture short- and long-term temporal dynamics. However, the two models differ in their methodological focus: MA-GNN-LSTM emphasizes spatial dependencies and external factors through graph-based representations and attention mechanisms, whereas T.R2_LSTM_A relies on transfer learning and instance reweighting to mitigate the effects of limited target-domain data.

2.2.4 Latest Mechanisms to Improve Model Performance

The most effective ways to improve the accuracy of TTE models involve enhancing the model's ability to represent, understand, and learn from the spatio-temporal structure of the transportation network. To achieve this, researchers typically introduce specialized modules and algorithmic modifications, which generally fall into three main categories: those that improve how the transportation network is represented, those that enhance the model's understanding of spatio-temporal and external dependencies and those that improve the model's understanding of how nodes interact and its effects.

Representation of a Transportation Networks Geographical Structure

Graph Neural Networks (GNNs) are considered one of the state-of-the-art approaches for enhancing a model's ability to represent transportation networks. Their strength lies in effectively capturing the non-Euclidean structures and geometric complexities inherent in transportation systems (Z. Zhang et al., 2024). This is one of the reasons contributing to the effectiveness of the Multi-View Transport Network Graph Construction module proposed by (J. Ma et al., 2022a)

which leverages the principles of GNNs to improve model performance.

The Multi-View Transport Network Graph Construction module, a fundamental component of the Multi-Attention Graph Travel Time Estimation (MAGTTE) model proposed by Ma et al. (2022), plays a crucial role in accurately representing the geographical structure of a bus-transit network. This is achieved through two key segments: Nodes and Node Features, and Weighted Adjacency Matrix Generation.

For Nodes and Node Features, the idea is that instead of relying solely on pre-labeled stop information, the MAGTTE model employs a Density-Based Clustering Algorithm, DBSCAN, and a node extraction algorithm to automatically identify and extract the precise locations of intersections and stops from bus trajectories with zero speed, also known as clusters. This approach is effective for capturing the distinct geographical paths of bus routes that may share common stops but follow different trajectories, thereby overcoming inaccuracies that arise from only using stop locations. The clusters identified by DBSCAN contain features such as longitude, latitude, and the number of GPS points, which directly aid in understanding dwelling times at these locations as dense clusters of stationary points indicate longer dwelling periods. Furthermore, to enrich the geographical representation of each node, the model integrates additional contextual information, such as peak/off-peak time slots and weekday/weekend status, by embedding them as node features using the Word2Vec method. This algorithm allows the model to decipher which nodes are more similar based on their temporal context.

Weighted Spatio-Temporal and External Dependencies Extraction

The Weighted Adjacency Matrix Generation module the MAGTTE model is important for accurately defining the intricate relationships and correlations between the geographically represented road segments (nodes) in a bus-transit network. MAGTTE constructs 3 weighted adjacency matrices such as the Distance Graph, Geographical Graph, and Social Function Graph which enables the model to infer patterns from fully-recorded routes to assist those with sparse or no historical data. Firstly, the Distance Graph helps the model represent the network by indicating that geographically "near neighbors" are more likely to share similar real-time traffic conditions. Secondly, the Geographical Graph states that segments of similar length often exhibit comparable travel times under similar conditions and dwelling times. Lastly, the Social Function Graphs is able to infer that nodes with similar social functions most likely have similar traffic conditions based on passenger congestion.

One of the limitations of most models, while they do incorporate spatio-temporal and external dependencies in their model, they often fail to represent how much each of these factors truly affect travel times as they would in real life due to the assumption that all nodes and their data are of the same reliability. To overcome this, the MAGTTE model uses a Spatial and Temporal Attention Modules that leverages attention mechanisms. In MAGTTE, the attention mechanism is enhanced by a multi-mask algorithm, which helps distinguish between nodes with complete data and those with sparse or unreliable information. This allows the model to assign greater attention and weight to more reliable nodes, improving prediction accuracy. The masking mechanism effectively filters out nodes that could introduce noise or unnecessary computation due to sparse data.

Soft-Masking vs. Hard-Masking

There are two kinds of masking algorithms applied to attention mechanisms, namely, hard-masking, also known as sharp-masking, or soft-masking. Hard-masking algorithms often have no attention ambiguity, meaning attention is only divided and applied onto some while the others are completely ignored (S. Guo et al., 2025). Hard-masking offers computational benefits by reducing the amount of elements the machine has to process (Lawson et al., 2018). However, with minimizing the elements the attention is divided into, this leads to potential information loss that could be pivotal in improving the model's accuracy (Indurthi, Chung, & Kim, 2019). Other papers that utilize soft-masking often achieve higher accuracy, especially in tasks that require the model to capture more complex relationships. This is due to soft-masking algorithms assigning more continuous weights among all elements, still fully utilizing their networks, which makes it harder to miss subtle correlations (Konishi et al., 2023). Although soft-masking requires more computational power to assign weights to every element or node in the network and is more prone to overfitting, to fully capture a network's most subtle relationships–especially in the context of travel time estimation models where delays affect multiple nodes throughout the network, not just its neighbors.

Node Interactions

For networks that rely on scheduled transport such as buses and trains, error accumulation is a very common issue as a delay on one node may lead to cascading delays for the following nodes that are on the scheduled trajectory of the focused vehicle. Considering that, (M. Li et al., 2024a) proposed a Multi-Task Learning module to help their model avoid this problem.

The Multi-Task Learning module within the AttentionTTE model serves as a critical component for predicting the travel time of a given trajectory while balancing both individual and collective prediction approaches. Previous studies often separated these two prediction types: individual prediction usually involves dividing a complete trajectory into multiple sub-segments, predicting the travel time for each, and then summing the results. While this method can achieve higher accuracy for individual sub-segments, it struggles to model the complex traffic environment of the entire trajectory, such as the impact of signal lights and intersections, which can lead to error accumulation and lower accuracy for longer sequences. On the other hand, collective prediction forecasts the travel time for the entire trajectory in one step. Although it can address the error accumulation issue of individual predictions, it tends to have reduced confidence and accuracy for longer trajectories due to sparse data for long-term trajectories. The Multi-Task Learning module aims to combine the strengths of both, leveraging the accuracy of individual predictions with the ability to model complex traffic states in collective predictions; therefore, improving overall accuracy.

The module's "multi-task" nature comes from its ability to learn from and balance both of these prediction methods concurrently during training. It combines the strengths of individual segment accuracy with the ability to model complex overall traffic situations, leading to improved travel time estimates for complete paths while avoiding error accumulation provided with the collective prediction's benefit. This balance is achieved by, for instance, determining the "importance" or weight of each sub-segment's contribution to the overall journey's prediction, similar to how attention mechanisms work.

2.3 Assessment Strategies

For us to evaluate the effectiveness and accuracy of travel time estimation models, especially hybrid models, a variety of assessment strategies will be employed. These strategies help determine how closely our model's output aligns with actual observed data and how efficiently it performs under operational constraints. These assessment strategies can be categorized into three primary sectors: Performance evaluation metrics, feature ablation studies, and benchmark models. Given our study's emphasis on a hybrid approach, a thorough understanding of how other hybrid travel time estimation models evaluate their data is essential. Therefore, a comparative analysis of these models will be made to gain insight into which evaluation techniques will be best utilized for our model, allowing us to select a comprehensive and vigorous set of criteria. This thorough comparison will not only identify current best practices but also point out new strategies and possible

problems with hybrid time estimation models. Ultimately, it will help choose a strong and pertinent set of standards for evaluating the predictive ability and real-world applicability of our suggested travel time estimation model.

2.3.1 Performance Evaluation Metrics

Comparing prediction performance using performance evaluation metrics is one of the most commonly employed assessment strategies among travel time estimation models to test the accuracy. (Huang, Wen, Fu, Peng, & Tang, 2020; R. Shi et al., 2021; J. Ma et al., 2022a; Tiong, Ma, & Palmqvist, 2022; M. Li et al., 2024a; C. Shi et al., 2025; J. Guo et al., 2024a) and other related studies adopt these quantitative metrics to compare the accuracy and robustness of their models against baseline or existing approaches. A table summary, Table 2.5, is provided below to showcase what evaluation metric(s) each study used and to remark on how it is used to evaluate their model.

Table 2.5: Summary of the Performance Evaluation Metrics

Model	**MAE**	**RMSE**	**R^2**	**MAPE**	**Remarks**
BO–XGBoost (Shi et al., 2021)	/	/	/	/	MAE measured average absolute prediction error in minutes; RMSE showed robustness to outliers; R^2 showed model's fit to delay patterns on train lines.
MAGTTE LSTM + Graph Neural Network (Ma et al., 2022)	X	/	X	/	RMSE compared predicted vs. actual travel times on the test set (last 5 days), penalizing severe prediction errors; MAPE used on normalized (Z-score) data for consistency.
Direct multi-output LightGBM (Tiong et al., 2022)	/	/	/	/	MAE measured the average absolute difference between predicted and actual arrival times (in minutes); RMSE evaluated robustness to severe delays. LightGBM outperformed baseline models.
CLF-Net — 3D CNN + LSTM + FCNN (Huang et al., 2020)	/	/	X	X	RMSE penalized large errors (important for severe delays); MAE provided intuitive, linear error interpretation. CLF-Net achieved lower RMSE/MAE vs. baselines.
AttentionTTE — Decision Tree + LSTM + Regression + RNN (Li et al., 2024)	/	/	X	/	MAE, RMSE, and MAPE compared across different time spans (24h, 1 week, 3 weeks); Lower MAE and RMSE proved accuracy for short-period predictions; MAPE highlighted module effectiveness.
T.R2_LSTM_A (Guo et al., 2024)	/	X	/	/	Used MAE, RMSE, and MAPE with values 2.361, 2.578, and 1.767, supassing other baseline models
Back Propagation Neural Network (BPNN) (Shi et al., 2025)	/	/	/	/	Used all 4 metrics and also performed an ablation study; achieved RMSE of 202.89 s and MAPE of 16.52%.

To better understand these evaluation metrics and how they are used to assess model performance, here is a clear description of each metric and what it does:

- **Mean Absolute Error (MAE)** — Calculates the average magnitude of the errors between predicted and actual values.

- **Root Mean Squared Error (RMSE)** — Measures the differences between values predicted by a model and the values actually observed.
- **Coefficient of Determination (R^2)** — Measures the fraction of the total variance in the dependent variable explained by the independent variable.
- **Mean Absolute Percentage Error (MAPE)** — Calculates the average absolute difference between actual and forecasted values as a percentage of the actual value.

Nowadays, many of the current studies measure the performance of their methods using these two key metrics: Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE). (Karunasingha, 2022) states that these measures are mostly utilized for prediction evaluation, model validation, model selection, model fitting (i.e., figuring out the best parameter choice for a certain model), and model comparison (among several competing models). Depending on the error distribution, both MAE and RMSE have unique benefits; RMSE is better suited for Laplace-distributed errors, whereas MAE performs better for regularly distributed errors (Hodson, 2022). As there is a heavier penalty on larger errors, RMSE is frequently used to discover and evaluate the severity of major forecast deviations or delays. For example, RMSE was used by Ma et al. (2022) and Huang et al. (2020) to evaluate the model's performance in the worst-case scenarios, demonstrating its capacity to manage excessive travel time values. When calculating for travel or arrival times, MAE is commonly used to clearly and understandably reflect the average size of forecast mistakes, which are usually quantified in minutes. In order to provide a more intuitive understanding of typical model accuracy, studies like (R. Shi et al., 2021) and (Tiong et al., 2022) employed MAE to indicate the average absolute divergence between anticipated and actual arrival times. Thus, it is common practice to evaluate regression models by integrating both measures, MAE and RMSE. This is corroborated by (Chai & Draxler, 2014) research, which asserts that depending just on one statistic only highlights one performance component and offers a single projection of a model's error characteristics. As a result, both metrics, RSME and MAE, are necessary to evaluate model performance (Chai & Draxler, 2014).

For some cases, to attain a more comprehensive analysis, Coefficient of Determination (R^2), and Weighted Mean Absolute Percentage Error (WMAPE) or Mean Absolute Percentage Error (MAPE) are also included. MAPE improves interpretability and provides customers with more lucid insights by presenting the prediction error as a percentage (Jang, 2024). Li et al. (M. Li et al., 2024a) and Ma et al. (J. Ma et al., 2022a), for instance, employed MAPE to evaluate model performance over many time periods or to standardize prediction findings in order to guarantee consistency and clarity in performance comparisons. In contrast to

error-based metrics that lack context and are more difficult to compare across models or datasets, R^2 offers a clearer, more interpretable measure of how well a regression model explains the variance in the data, as recommended by another study by (Chicco, Warrens, & Jurman, 2021). This supports the findings of Shi et al. (R. Shi et al., 2021)'s research, which assessed how effectively their model captured rail line delay patterns using R^2. Shi et al. (C. Shi et al., 2025) employed the same four evaluation metrics in their study, including during their ablation analysis. Similarly, our study, which also conducts an ablation study, will adopt these metrics as well. In light of these findings, the proposed hybrid travel time estimation model will be evaluated using four key metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), Mean Absolute Percentage Error (MAPE), and the Coefficient of Determination (R^2). This combination of metrics enables a balanced evaluation framework that considers various aspects of model performance.

2.3.2 Feature Ablation Studies

To further examine the contribution of each component within our proposed hybrid travel time estimation model, an ablation study will be carried out after the performance evaluation metrics. In the context of hybrid deep learning models for travel time estimation, ablation studies are particularly valuable for identifying which real-world variables significantly enhance model predictions. (Meyes, Lu, de Puiseau, & Meisen, 2019) employs ablation studies not only to optimize model size and computational efficiency through pruning, but also to analyze the internal structure and decision-making behavior of neural networks. By selectively removing trainable components or units, the study reveals which elements contribute most to the network's output, thereby improving transparency, interpretability, and understanding of performance-critical features (Meyes et al., 2019).

A kind of ablation study, feature ablation, is a technique that separates and evaluates the contribution of particular input variables, is discussed by the study of (Sheikholeslami et al., 2021). This approach is particularly pertinent in transport modeling, where input features frequently represent actual factors like traffic, weather, or system delays. The issue that many research fail to pinpoint the precise causes of performance improvement while putting out new models is also addressed in the study by (Sheikholeslami et al., 2021).In line with this, the current study adopts a feature-focused ablation strategy, to assess the relative importance of key real-world selected features (weather conditions, crowd congestion, dwell times, and operational delays) on the predictive accuracy of the proposed MA-GNN-LSTM model. Recent studies in travel time prediction have increasingly adopted this approach. In their thorough ablation investigation, (C. Shi

et al., 2025) evaluated a number of feature sets, including spatiotemporal, travel, weather, and driver behavior, among others, by gradually adding or removing them. Their results demonstrated that whereas spatial-temporal data offered a basic degree of prediction, it was only after taking outside influences into account that notable accuracy gains could be made (C. Shi et al., 2025). This method provided quantifiable insight into the performance impact of each added feature and emphasized the need for comprehensive input data in real-world predictive tasks. Similarly, by quantifying the performance variation across different input sets, this research will identify the features with the highest impact on error reduction and prediction stability.

Beyond input features, module ablation refers to the systematic removal or replacement of sub-modules within the model architecture. By training and evaluating the model without one of these components, researchers can measure the contribution of each module to overall performance. For example, the study by (Cao, Shan, Liu, & Yan, 2025) conducted a module ablation study to determine the importance and contribution of two specific components the image enhancement algorithm and the Adaptive Feature Fusion Module (AFFM) where removing both modules led to the highest error. Similarly, dropping the attention mechanism often results in a poorer understanding of long-range temporal dependencies (Z. Wu, Pan, Long, Jiang, & Zhang, 2019), which also used module ablation study to assess the contribution of the attention mechanism. This study will also employ the same strategy, but specifically, the 'multi-task learning module' will be subjected to ablation analysis. By isolating and removing this component, the assessment of how the accumulation of errors can affect the model's ability to generalize across multiple prediction horizons can be more accurately evaluated. This method enables a more thorough comprehension of the ways in which inter-task knowledge sharing supports both the learning process and the general resilience of the model.

2.3.3 Comparison to Selected Benchmark Model

To validate the effectiveness of the proposed hybrid model, Multi-Attention Graph Neural Network with Long Short-Term Memory (MA-GNN-LSTM), it is essential to compare its performance against existing models in train travel time prediction. By comparing our proposed MA-GNN-LSTM framework to T.R2_LSTM_A, we aim to assess the effectiveness of explicitly incorporating spatial relationships and external influencing factors, and to determine whether these enhancements offer measurable improvements over the transfer-learning-based methodology.

Chapter 3

Theoretical Framework

3.1 Graph Theory

A graph's structure is traditionally represented by a static, unweighted adjacency matrix where entries are binary (0 or 1) (X. Li & Peng, 2022). Graphs are mathematical abstractions that can represent various types of networks, including physical ones (Pirzada, 2007). Graphs can represent a wide array of data structures because it is considered non-Euclidean spaces, which distinguishes them from traditional grid-like data formats making it excel at capturing relational information between elements. A graph or a network is a collection of elements called nodes (or vertices) and the links that connect them called edges (or connections) (Khemani, Patil, Kotecha, & Tanwar, 2024). In its simplest form, a graph can be represented by a connection or adjacency matrix. This matrix captures all pairwise connections, defining the graph's topology and offering a comprehensive map of the relationships among nodes and edges. (Sporns, 2018). Nodes are typically represented as circles or ovals and edges are represented as lines or arrows. Mathematically, it is defined as G = (V, E) where V is a set of vertices or nodes and E

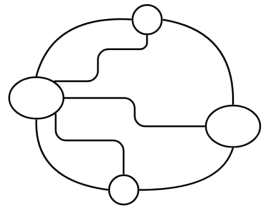

Figure 3.1: Graph Theory

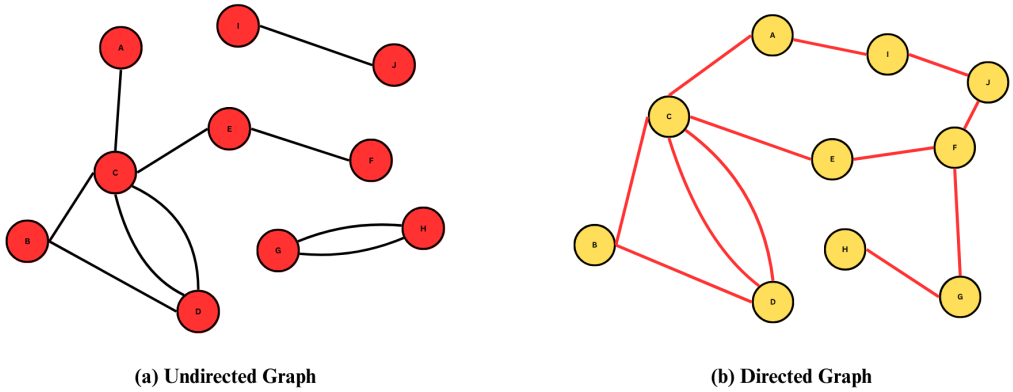

Figure 3.2: Types of Graphs

is the set of edges or connections that represent a binary relation on V. (Bondy & Murty, 2008)

Graphs can be classified as either directed or undirected. In directed graphs, edges (i, j) have a specific direction, indicating an asymmetric relationship in which a connection flows from one node to another. In Undirected graphs, the edges have no direction and they represent symmetric relations between objects. (Bondy & Murty, 2008) The difference between these graphs can be seen in Figure 3.2. Furthermore, graphs may be valued or unvalued. A valued graph assigns numerical weights to edges that represent the intensity or strength or quantity of the tie or frequency between the two nodes. The latter represent only the presence or absence of a connection, without quantifying its magnitude. (Majeed & Rauf, 2020) To better capture the nuances of complex systems, a weighted adjacency matrix is introduced, where the entry for an edge (i, j) is assigned a real value f(di, dj) based on a symmetric function of the properties of the connected nodes (X. L. Li & Yang, 2024) (X. Li & Yang, 2024). Such a matrix retains significantly more structural information than a single topological index or a binary representation (X. Li & Yang, 2024). However, in many real-world applications, these weights are not predefined but must be learned from data. The challenge is to create a module that can dynamically infer these weights to model complex interactions. Conventional Graph Convolutional Network (GCN) models often use a constant, predefined physical adjacency matrix, which makes it difficult to mine synergistic relationships between points that are not directly connected in

physical space (R. Yang, Niu, Xu, Wang, & Qiu, 2023a).

In the domain of a rail transit system, each station can be represented as a node, while the tracks connecting them serve as the edges of the graph. This graph-based representation captures the spatial and operational structure of the network. This mapping allows the graph to visually and computationally depict the system's layout and performance. (Sporns, 2018) For modeling rail travel, directed edges are essential because they represent the flow of movement from one station to another, capturing features such as one-way tracks or specific travel routes. Since travel time and delays fluctuate over time, dynamic graphs are introduced to account for these temporal changes. In dynamic graphs, the properties or structure of the graph change over time, this can involve changes like the addition or removal of nodes and edges, or updates to edge weights. This adaptability is critical for real-time traffic modeling, where factors like congestion, accidents, or track closures continuously affect travel times (Khemani et al., 2024).

Spatial-temporal graphs are a fundamental concept in analyzing systems where components exhibit both spatial relationships and dynamic temporal trends (Z. Wu et al., 2019). Spatial-temporal graphs capture both spatial layout and time-varying data. It models nodes and edges as they evolve over both time and space (Pirzada, 2007). Formally, it can be represented as a sequence or collection of graphs $G_t = (V, E_t)$, where the set of nodes V is typically fixed, but the edge set E_t and possibly node or edge attributes change over discrete or continuous time steps, capturing dynamic interactions or dependencies among entities in different spatial locations and at different times (Tan, Zhu, & Liu, 2023). A temporal graph is essentially a graph that evolves over time. Formally, it can be considered a specific type of labeled graph where the labels represent a temporal aspect. Conversely, any property observed in a graph with discrete labels can be interpreted as a temporal property if the labels are understood in a time-sensitive manner. (Michail, 2016). In spatial-temporal graphs, spatial relationships (such as physical proximity or network connectivity) are encoded in the graph structure, while temporal dynamics are captured by the evolution of the graph or by associating time-stamped events or features with nodes and edges (Y. Wang, Zhang, Xiang, & Pan, 2022). The "spatial layout" is captured by how nodes (entities) are connected by edges (relationships) (Bondy & Murty, 2008). For instance, a valued graph can have numbers attached to edges to represent the intensity, strength, quantity, or frequency of the tie between nodes, thereby combining data with spatial connections (Majeed & Rauf, 2020). The "time-varying data" aspect means the graph's properties or structure explicitly change with time. This can involve changes like adding or removing nodes and edges (Khemani et al., 2024). These graphs are a crucial evolution in graph theory and Graph Neural Networks

(GNNs) that allow for the modeling and analysis of data that changes over time while also capturing inherent spatial or relational structures (Pirzada, 2007).

3.2 Graph Neural Networks (GNNs)

Graph Neural Networks (GNNs) are a type of deep learning model designed to process and learn from data structured as graphs. They are particularly effective at capturing the intricate dependencies within graphs by employing a "message passing" mechanism between individual nodes. This capability allows GNNs to address challenges where traditional machine learning models often fall short, especially in discerning complex relationships within interconnected datasets.

A fundamental characteristic of GNNs is their preservation of graph symmetries, notably permutation invariance. They optimize transformations across all properties of a graph, encompassing nodes, edges, and the overall global context. It is important to note that while a GNN modifies the representations of each node, edge, and the global context, it does not alter the fundamental connectivity of the input graph. Consequently, the output of a GNN can be described using the same adjacency list and maintains the same count of feature vectors as the original input graph (Khemani et al., 2024).

GNNs possess a unique capacity to discern and capture complex structural information and inherent dependencies present in graph-structured data. They achieve this through a "message-passing mechanism," which aggregates information from neighboring nodes, thereby facilitating an understanding of the relationships among nodes and edges (Jegelka, 2022). The core operational principle of GNNs is this message-passing paradigm, also known as neighborhood aggregation. This foundational process enables GNNs to effectively integrate node feature information with the intrinsic graph structure by recursively transmitting neural messages along the edges of the input graph (Ying, Bourgeois, You, Zitnik, & Leskovec, 2019) (Xie, Wang, Ye, Zhang, & Fan, 2022).

By stacking multiple such layers (or iterations), GNNs can extract increasingly high-level node representations (M. Liu, Gao, & Ji, 2020). Each iteration allows a node's representation to receive messages and incorporate information from a wider, more distant neighborhood in the graph. This iterative expansion of a node's "receptive field" is important for learning complex patterns (Fey, 2019). A GNN is able to extract global information by stacking local graph convolutional layers, as the receptive field of a node grows with each layer (Z. Wu et al., 2020). However Traditional Graph Neural Networks (GNNs) can suffer from

over-smoothing or over-squashing when trying to learn representations for entities in these sparse or isolated subgraphs, especially as the number of GNN layers increases (H. Xu, Bao, & Liu, 2023).

3.3 Multi-Attention Graph Neural Networks (MA-GNNs)

Multi-Attention Graph Neural Networks (MA-GNNs) are a novel deep-learning architecture that is designed to alleviate the limitations of traditional Graph Neural Networks (GNNs) since existing GNN methods often only capture local information within the immediate neighborhood of individual entities, failing to exploit crucial global information. MA-GNN proposes a double-branch architecture that leverages multiple types of attention mechanisms to capture both global and local structural information, as well as rich entity features (H. Xu et al., 2023). MA-GNNs extend this paradigm by introducing multiple parallel attention mechanisms capable of encoding a broader range of structural and semantic relationships. Through this architectural innovation, MA-GNNs provide a more expressive and adaptable framework for modeling complex graph-structured data.

At the core of MA-GNNs lies the use of attention mechanisms, which have become a foundational tool in graph learning due to their ability to selectively focus on informative components within a graph (J. B. Lee, Rossi, Kim, Ahmed, & Koh, 2019). This allows neural networks to select discriminative features and filter out noisy information, which is particularly beneficial in complex data structures like graphs. The attention process can be generalized and decomposed into three key functions: an alignment function to determine similarity scores, a distribution function to normalize these scores into a probability distribution, and a weighted sum function to aggregate attended representations. When applied to a single sequence, it is often referred to as self-attention (C. Sun et al., 2023). This mechanism enables selective feature refinement, allowing the model to amplify relevant signals and suppress noise during the learning process.

Earlier models like the Graph Attention Network (GAT) utilize a single attention layer to weight neighboring nodes during message passing, such an approach can fall short when the graph contains multiple forms of relations or when feature heterogeneity is high. MA-GNNs enhance this capability by organizing attention mechanisms into specialized branches, each designed to handle different types of structure or data modality. This leads to three principal strategies of multi-attention learning: multi-view attention, multi-modal attention, and multi-source attention.

The most direct implementation of multi-attention in GNNs involves constructing multiple distinct graphs or "views" from the same set of nodes, where each view represents a different type of relationship. An attention mechanism is then tasked with learning how to weigh and fuse the information from these different structural representations. Multi-View Attention addresses the challenge of structural heterogeneity by constructing multiple graph views from a shared node set, with each view encoding a distinct relational aspect. Instead of a single adjacency matrix, the model is given multiple matrices, each encoding a specific relational aspect. These views are typically represented by separate adjacency matrices that reflect various forms of interaction such as geographic proximity, functional similarity, or temporal alignment. Each view is processed by a dedicated graph encoder, and a higher-level attention module is employed to fuse the resulting embeddings. This process allows the model to assign adaptive importance to each view, thereby capturing complementary and context-specific patterns in the data (F. Wu, Zheng, Zhang, Ma, & Sun, 2023).

Multi-Modal Attention is designed for scenarios where node features span multiple modalities, such as visual, textual, and numerical inputs. This strategy applies when nodes in the graph are described by features from multiple modalities. A multi-attention mechanism learns to weigh the importance of each modality, often in a personalized, user-specific manner. This is crucial for creating representations that reflect the interplay between different types of information. Instead of treating all features as homogeneous, MA-GNNs embed each modality into a shared latent space and apply attention to determine their relative importance. This mechanism learns to prioritize modalities based on their informativeness for the task at hand, facilitating the construction of unified node representations that incorporate diverse semantic cues (H. Xu et al., 2023). Such a strategy proves effective in applications where the meaning of nodes emerges from an interplay of heterogeneous attributes (Kim, Kim, Lee, Hong, & Lee, 2024).

Many real-world systems, such as Point-of-Interest (POI) networks, involve both static graph structures and dynamic user-specific sequences. Multi-attention can be used to fuse the global, structural information learned by a GNN with the personal, temporal information learned from a sequential model. Multi-Source Attention integrates static graph structures with dynamic sequential information, which is common in systems that model temporal user behavior over a spatial network. Here, MA-GNNs employ dual encoders: a GNN for capturing global, static relationships, and a sequential model (such as a Transformer or LSTM) for modeling individualized temporal patterns. The outputs of these encoders are merged via an attention-based fusion module, which learns to dynamically balance global structural context with personalized temporal dynamics (J. B. Lee et al., 2019). This facilitates the synthesis of generalized and user-specific represen-

tations in tasks like recommendation or time-series forecasting. This approach has been formalized in models such as the Attention-driven Graph-Sequence Fusion (AGSF) network, where an attention score is used to guide the merging of GCN-derived global features with sequence-derived embeddings in a shared latent space, allowing for contextually informed integration (Tao, Guo, Jie, & Liu, 2024). Similarly, the Multi-Attention Graph Neural Network for Travel Time Estimation (MAGTTE) employs attention mechanisms to combine spatial dependencies captured by graph structures with temporal patterns encoded by sequential models, facilitating accurate estimation even with limited data (J. Ma, Chan, Rajasegarar, & Leckie, 2022b).

In unifying these strategies, MA-GNNs establish a framework capable of selectively integrating multiple relational structures, diverse feature types, and complementary information sources. Rather than relying on a single attention mechanism with uniform behavior across the graph, MA-GNNs utilize specialized branches that attend to different facets of the data. This results in a more nuanced and task-sensitive model architecture. Through attention-based fusion at various levels, MA-GNNs offer a scalable and flexible solution for learning representations in complex, heterogeneous graph environments. (J. B. Lee et al., 2019)

3.4 Recurrent Neural Network (RNN)

Recurrent Neural Networks (RNNs) are a class of artificial neural networks specifically designed to process sequential data. This model achieves this by utilizing internal feedback loops, allowing them to maintain information from previous inputs in a sequence. In an RNN, neurons are interconnected in a way that allows them to send signals back to one another, enabling the network to retain information over time (Grossberg, 2013). The presence of directed cycles in their architecture allows RNNs to model temporal dependencies, making them particularly effective for identifying patterns within sequential data (Schmidt, 2019).

RNNs face two major training challenges: vanishing gradients and exploding gradients (Pascanu, Mikolov, & Bengio, 2013).These issues arise during the process of backpropagation through time, where gradients are propagated backward across many time steps. In the case of vanishing gradients, if the values involved in gradient computation are less than one, repeated multiplication causes the gradients to shrink exponentially. This results in earlier time steps contributing less and less to the model's learning, making it difficult for the RNN to capture long-term dependencies. On the other hand, exploding gradients occur when the values in the gradient computation are greater than one, causing gradients to

grow exponentially with each time step. This leads to excessively large weight updates, instability during training, and poor convergence (Schmidt, 2019) (G. Chen, 2016).

3.5 Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) (Hochreiter & Schmidhuber, 1997a) networks are a specialized type of recurrent neural network (RNN) developed to address the problem of learning long-term dependencies in sequential data, which standard RNNs struggle with due to vanishing or exploding gradients (Van Houdt, Mosquera, & Nápoles, 2020). Standard RNNs struggle with long-term dependencies because backpropagated gradients tend to either vanish or explode as they are propagated through many time steps. To address these limitations, Hochreiter and Schmidhuber proposed the Long Short-Term Memory model in 1997 (Hochreiter & Schmidhuber, 1997b).

LSTM networks enhance the traditional RNN architecture in two ways. First by introducing memory cells that store information over time, and by improving the model's ability to handle long sequences, effectively addressing the vanishing gradient issue (G. Chen, 2016). The core innovation of LSTM is the memory cell, which maintains information over long periods through a linear self-connection known as an error carousel. This fixed-weight connection ensures constant error flow, preventing both gradient vanishing and explosion during training (Staudemeyer & Morris, 2019). Each memory cell is equipped with input and output gates. This includes multiplicative units that learn to regulate access to the memory. The input gate protects the cell from irrelevant inputs, while the output gate shields downstream units from irrelevant stored content. These gates learn when to open or close, enabling the model to preserve and expose information as needed (Gers, Schmidhuber, & Cummins, 2000). The LSTM cell comprises input, output, and forget gates that regulate information flow. The gates are multiplicative and use sigmoid activation functions to control whether information should be written, read, or reset. The forget gate, in particular, was a later addition to the architecture, enabling the network to discard irrelevant memory content (Hochreiter & Schmidhuber, 1997b).

LSTM models, especially when enhanced with attention mechanisms or combined with other deep learning architectures have demonstrated superior accuracy over traditional statistical and machine learning methods (J. Guo, Wang, Tang, Zhang, & Zhuge, 2022). Attention-based LSTM frameworks can effectively capture both spatial and temporal features, leading to lower prediction errors

compared to baseline models like random forests, support vector regression, and standard LSTM (J. Guo et al., 2024b). LSTM-based models are also very good at handling irregular and nonstationary travel time patterns, which makes them suitable for real-world railway operations where data can be sparse or noisy (Duan, L.V., & Wang, 2016).

3.6 Multi-View Graph Construction Module

A Multi-View Graph Construction Module is a component in machine learning models that builds graph structures from data collected across multiple "views" or sources, such as different feature sets, sensors, or modalities (F. Wu et al., 2022). Its main goal is to capture both the unique and shared relationships present in each view, enabling more comprehensive data representation and improved downstream tasks like clustering or classification (L. Li & He, 2022). Typically, the module constructs separate graphs for each view, then fuses or integrates them into a unified or consensus graph that reflects the complementary information from all views.

Traditional graph construction methods, such as building a graph based on the k-nearest neighbor (k-NN) rule, are widely used (Z. Zhang et al., 2023) (Xiao, Du, Chen, Zhang, & Wang, 2023). However, these methods face significant challenges. Firstly, they are often sensitive to noise and redundancy present in the original features, which can lead to an inaccurate and suboptimal graph that is insufficient to depict the underlying cluster structure (Z. Li et al., 2022) (H. Wang, Yang, & Liu, 2020) (F. Wu et al., 2022). Secondly, their performance often relies on the careful tuning of hyper-parameters, such as the width of the Gaussian kernel, which is difficult to set in practice (H. Wang et al., 2020). Most importantly, these methods typically separate the process of graph construction from the subsequent feature learning, which means the resulting fixed graph may not be optimal for the specific downstream learning task (Z. Zhang et al., 2023).

The primary objective of a graph construction module is to produce an adjacency or similarity matrix that accurately reflects the relationships between data samples. This can be approached by constructing graphs for each view individually, learning a single consensus graph, or employing specialized structures like bipartite graphs. While each view may offer unique insights and contain its own biases, there is often shared or "consensus" information that can be leveraged across views. The module aims to identify and integrate this common understanding, while also exploiting the unique, complementary details each view provides (L. Li & He, 2022). Most graph-based multi-view clustering methods rely on similarity

graphs where edges represent relationships between data points (H. Wang et al., 2020). An effective construction module produces a unified graph that better reflects the underlying cluster structure than graphs derived from single views or simple fusions (Z. Li et al., 2022). Techniques such as pruning noisy edges, symmetrizing the adjacency matrix, and applying renormalization tricks further refine the graph structure. These steps ensure stability during training and improve the quality of representations learned by GNNs. By providing a more accurate and comprehensive representation of the data's intrinsic structure, the module directly enhances the performance of subsequent tasks such as multi-view clustering (L. Li & He, 2022) or semi-supervised classification (F. Wu et al., 2022).

Different multi-view learning approaches employ varied strategies for graph construction and fusion. This process typically involves two key stages: constructing view-specific graphs and then integrating or fusing them into a unified or consensus graph, which then often directly informs downstream tasks like clustering or classification. The initial step often involves view-specific graph learning, where individual graphs are built for each distinct data modality or feature set. Traditional methods might rely on pairwise similarities between data points, which can introduce biases (L. Li & He, 2022). More advanced approaches address this by employing techniques such as sparse representation to construct Similarity-Induced Graph (SIG) matrices, making the process robust to noise and outliers (H. Wang et al., 2020). Some methods, like BIGMC, introduce bipartite graphs between data points and a smaller set of representative uniform anchor points to capture consensus information within each view (L. Li & He, 2022).

3.7 Multi-Mask Attention Module

Multi-Mask Attention Modules are a type of architecture specifically designed to generate and use multiple distinct attention masks to help solve complex problems. Instead of using only one general mask or attention map, these modules divide the task into smaller parts by creating a set of masks. Each mask represents a different possible solution, interpretation, or specific function (C. Liu, Ding, Zhang, & Jiang, 2023). The main idea behind this approach is to manage the complexity and uncertainty of a task by forming a group of specialized masks. These masks work together like a "committee," and their outputs are combined or adjusted to produce a final result that is more accurate and reliable (J. Ma et al., 2022a).

This framework can be understood through three primary strategies, Parallel Mask Generation for Ambiguity Resolution, Functional Decomposition with

Operator Masks and Layer-wise Dynamic Masking for Hierarchical Processing. Parallel mask generation is the most common strategy in multi-mask attention frameworks, where a single input is used to produce multiple parallel mask hypotheses. The mechanism involves processing the input to generate several sub-queries or proposals, each capturing a different potential interpretation. Each of these sub-queries is then used to produce a corresponding, independent mask. MMNet applies this approach by generating multiple queries from a single language expression, with each query focusing on different words or attributes. This results in a series of segmentation masks, which are combined through a weighted sum to produce a final output that emphasizes the most likely interpretation while reducing randomness (Yan, He, Wan, & Liu, 2023).

Functional Decomposition with Operator Masks is a strategy that broadens the application of multi-mask techniques beyond simple segmentation tasks. In this approach, each mask does not merely define a region but corresponds to a specific function or operation that should be applied to the input. This enables the model to carry out multiple distinct modifications at the same time. The mechanism involves generating a stack of masks, where each individual mask determines both the intensity and the spatial location of a predefined operation. These operations can include adjustments to visual, structural, or contextual properties of the input. By controlling where and how each operation is applied, the model can execute a complex sequence of modifications within a single processing step (Ren & Liu, 2022).

Layer-wise dynamic masking for hierarchical processing is a strategy that extends the multi-mask approach across the different layers of a deep neural network. It is based on the idea that each layer captures varying levels of abstraction and therefore may benefit from its own tailored attention pattern. The mechanism involves dynamically generating a separate, learnable attention mask for each layer or stage in the network. These masks guide the model in regulating the flow of information and adjusting the importance of specific tokens or features at different depths of processing. By assigning unique attention behaviors to each layer, the model can better adapt to the evolving structure of the input and improve both its accuracy and efficiency in handling complex data (Barrios & Jin, 2024).

3.8 Multi-Head Attention Graph Module (Weighted Adjacency Matrix Module)

Multi-Head Attention Graph Modules have been developed as an adaptive adjacency matrix generation mechanism. This module utilizes the learning capabilities

of graph attention networks to assign dynamic edge weights that are optimized for specific tasks. The learning of these weights is achieved through a multi-head attention mechanism, which systematically determines the importance of various relationships within the graph structure (Akay, Cakir, & Arica, 2025) (G. Xu, Liu, Zhu, Liu, & Xu, 2021). The Multi-Head Attention Graph Module leverages the principles of Graph Attention Networks (GATs) to generate its adaptive weighted adjacency matrix. GATs are designed to compute attention coefficients that quantify the relevance of one node's features to another within the graph (Akay et al., 2025).

This is operational through a series of steps of generating these weights for an edge between node i and a neighboring node j. Node feature vectors undergo a shared linear transformation, followed by the application of an attention function that scores the relationship between nodes based on their transformed features (J. Zhang, Yunos, & Haron, 2023) (Reka, Karthick, Saravana Ram, & Singh, 2024). These raw scores are then normalized across a node's neighbors using a softmax function, yielding attention coefficients that serve as learned edge weights in the graph (G. Xu et al., 2021). The resulting weighted adjacency matrix encodes directionality and importance, enabling a more refined feature aggregation process in which a node updates its representation based on a weighted sum of its neighbors' features (J. Zhang et al., 2023).

Relying on a single attention mechanism can be restrictive, since it forces the model to learn only one type of relational pattern. The multi-head attention mechanism overcomes this by executing multiple independent attention operations in parallel (S. Lee & Song, 2019). To further enhance representational capacity, the multi-head extension introduces parallel attention mechanisms. Each head operates independently, learning a distinct set of attention coefficients that correspond to different types of relationships or interaction patterns in the data (Reka et al., 2024).

Each head can focus on different aspects of the relationships between nodes, such as semantic, spatial, or contextual cues (Hu, Hu, Zhang, Jin, & Xu, 2025). For example, in interpersonal relationship detection, different heads can learn to focus on age differences, expression similarities, or contextual objects independently (Akay et al., 2025). This parallel structure also allows the module to stabilize the learning process and produce a richer presentation. Averaging the outputs of multiple heads makes the learning process more stable and robust compared to a single attention mechanism (J. Zhang et al., 2023). The outputs from the multiple heads are typically aggregated and passed through a final linear transformation (S. Lee & Song, 2019). This produces a single, powerful feature representation that encapsulates the multifaceted relationships discovered by all heads. In this way, the module generates multiple adaptive weighted adjacency

matrices that collectively contribute to more effective learning and improved stability in training.

The Multi-Head Attention Graph Module is designed to be a flexible and powerful component within larger deep learning architectures. Functionally, the Multi-Head Attention Graph Module is designed to serve two primary roles within broader architectures. First, it can act as an Adjacency Matrix Generation (AMG) unit, responsible for constructing an optimized graph structure from raw node features. In some applications, the module's primary function is to explicitly generate an adaptive adjacency matrix that is then used by a standard GCN. This separates the process of structure learning from feature propagation, allowing downstream graph convolution operations to benefit from a data-driven topology tailored to the specific learning objective. The AMG module pre-analyzes the input node set to generate an adjacency matrix that is more conducive to mining synergistic information than a static, physical one. This allows a subsequent GCN to better understand the spatial or abstract domain information (R. Yang, Niu, Xu, Wang, & Qiu, 2023b).

Second, the module can be used as a feature fusion mechanism, where distinct feature groups are processed by dedicated attention heads or branches. Each branch employs a multi-head attention structure to learn relationships specific to a particular type of input feature. These representations are then integrated to form a more discriminative and generalizable model This modular architecture improves feature discrimination and reduces the risk of overfitting by allowing the model to learn targeted interactions within each feature group before fusing them (Akay et al., 2025).

By learning a weighted graph, the module is not limited by physical or predefined proximity. It can establish strong connections between distant nodes in the input sequence or graph, enabling it to capture long-distance dependencies (G. Xu et al., 2021). The concept can be extended to model higher-order connectivity patterns, such as network motifs. A motif adjacency matrix, where the weight between two nodes represents their joint participation in a specific motif can be generated. This allows the model to cluster or analyze networks based on these higher-order structures rather than just simple edges (Y. Wang, Wang, Hou, & Yang, 2023).

3.9 Spatial and Temporal Attention Modules

Spatial Attention Modules utilize a Graph Attention Network (GAT) to learn spatial dependencies among different travel segments. GAT's ability to handle dynamic graphs and inductive learning makes it suitable for city-wide TTE with limited data, allowing patterns from rich data to infer missing records. It learns correlations between node pairs and updates hidden features (J. Ma et al., 2022a). Temporal Attention Modules combine LSTM units to capture local historical travel patterns and a Transformer layer to capture global temporal dependencies. The Transformer layer provides advantages like low cumulative error and efficient training for long-term dependencies compared to traditional RNNs. This dual approach considers both immediate and long-term traffic conditions (J. Ma et al., 2022a).

The foundational idea behind attention mechanisms can be divided into two main types: spatial and temporal attention. It identifies the most informative spatial regions or features. In the context of CNNs, this can be achieved by learning weights for different pixel locations or, more commonly, for different feature channels, as different channels can learn to represent distinct spatial characteristics (S. Liu, Ma, Wu, & Li, 2020) (J. Li et al., 2020). Temporal attention addresses the question of "when" to focus on a sequential input. It assigns variable importance to different frames or time steps. One common approach is to use global pooling across spatial dimensions to generate frame-level scores that reflect temporal relevance. These scores guide the model to emphasize significant time intervals in the overall analysis (Z. Zhang, Lv, Gan, & Zhu, 2020) (S. Liu et al., 2020).

Spatio-temporal attention integrates both spatial and temporal dependencies into a unified representation. Rather than processing spatial and temporal features separately, the mechanism jointly learns attention weights across space and time to produce a feature representation that captures global contextual cues. This fused representation can be combined with high-level features, such as those from a fully-connected layer, and subsequently used to enhance recurrent models or other deep learning architectures (W. Du, Wang, & Qiao, 2018).

Designing spatio-temporal attention involves several architectural considerations that determine how models focus on relevant spatial and temporal information. One key design choice is whether to use decoupled or fused attention. Decoupled attention treats spatial and temporal features independently, allowing the model to capture the distinct properties of each dimension before integrating them. This approach avoids flattening spatio-temporal data, which can result in loss of important structural information (L. Shi, Zhang, Cheng, & Lu, 2020). In

contrast, fused attention processes spatial and temporal information jointly within a single module. Some architectures achieve this by integrating spatial and channel attention using gating mechanisms or applying attention directly to the output of convolutional LSTMs (C. Li, Wang, Niu, Yao, & Zeng, 2021) (Z. Zhang et al., 2020).

The scope of attention further influences model performance. Global attention mechanisms leverage the full input sequence to capture long-range dependencies, while local attention focuses on nearby elements for finer detail. Some modules, like the Temporal Local Context Attention (TLCA) and Proximity Frame Time-Space Attention (PFTSA), integrate both approaches to balance detailed focus with broader context (W. Du et al., 2018) (Zhao et al., 2022). Multi-scale and hierarchical attention mechanisms enhance representation by processing information at various spatial and temporal resolutions. Modules like the Pyramid Attention Module (PAM) divide feature maps into subregions for scale-specific attention, while hierarchical fusion modules combine features from different network layers to integrate multiple levels of semantic abstraction (H. Chen & Shi, 2020) (J. Fu, Gao, & Xu, 2022).

A notable strength of attention mechanisms is their modularity. Many are designed as lightweight components that can be seamlessly integrated into existing architectures without significant structural changes (J. Fu et al., 2022) (W. Du et al., 2018). To improve the effectiveness and interpretability of attention mechanisms, several advanced strategies are employed. One such strategy is semantic-aware attention, where the model is guided to attend to semantically meaningful regions. For example, attention maps can be trained with explicit supervision from category labels to align with specific class-defining features. This forces the model to associate attention weights with interpretable, task-relevant concepts (J. Fu et al., 2022). Feature sharpening techniques, like those in the Spatio-Temporal Sharpening Attention Mechanism (SSAM), improve localization accuracy by emphasizing object boundaries (Xue, Chen, Peng, Guo, & Chen, 2021). Cross-attention mechanisms extend the idea of attention by modeling relationships between different input streams. While self-attention captures dependencies within a single sequence, cross-attention identifies correlations between parallel streams, such as spatial and temporal features processed independently. This can reveal complementary patterns and enhance the model's understanding of inter-dimensional interactions. The Temporal-Spatial Cross-Attention Network (TSCA-Net), for example, uses this approach to associate temporal dynamics with spatial distributions (Yue et al., 2024).

3.10 Multi-Task Learning Module

Multi-Task Learning (MTL) is a machine learning paradigm where multiple related learning tasks are optimized simultaneously by a shared model (Y. Zhang & Yang, 2017) (Crawshaw, 2020). A multi-task learning module is employed to integrate different prediction objectives, such as estimating travel time for both the entire path and each local path (M. Li, Feng, & Wu, 2024b). This approach allows for a more comprehensive understanding of the problem by leveraging shared knowledge across related tasks. For instance, in travel time estimation, individual predictions for sub-segments can be more accurate but might fail to model global spatial correlations, while collective prediction directly estimates the entire path's travel time (M. Li et al., 2024b). Multi-task learning aims to balance this trade-off between individual segment time estimation and overall path time estimation (Z. Zheng, Ye, Zhu, Zhang, & Yu, 2023). For example, NFT-MARS uses multi-task learning to address the dual nature of NFTs by learning recommendation scores for aesthetic value and predicting price movements for financial insights (Kim et al., 2024).

The primary objective is to enhance generalization across tasks by exploiting shared information and domain-specific knowledge embedded within them (Thung & Wee, 2018) (Y. Zhang & Yang, 2017) This strategy addresses challenges associated with data sparsity, where individual tasks may lack sufficient labeled data. By integrating datasets across tasks, the model can achieve better performance and robustness (Y. Zhang & Yang, 2017) (Y. Zhang & Yang, 2022). The core assumption of MTL is that the tasks, or at least a subset of them, are related and share commonalities. By learning them jointly, the model is biased to prefer representations that are beneficial for multiple tasks, which acts as a form of inductive transfer or inductive bias (Thung & Wee, 2018). This shared representation can lead to improved data efficiency, reduced risk of overfitting, and faster learning by leveraging auxiliary information from related tasks (Crawshaw, 2020).

In deep neural networks, task sharing is shaped by architectural design. Two primary strategies include hard and soft parameter sharing (Crawshaw, 2020) (Vandenhende et al., 2022). Hard sharing employs a unified encoder for all tasks, followed by task-specific decoders. While efficient, this method may impose restrictive constraints. Soft sharing allows each task to maintain its own parameters while encouraging similarity through regularization of inter-model distances, thus offering greater flexibility at a higher parameter cost.

Architectures can also be categorized as encoder-focused or decoder-focused. Encoder-focused models restrict sharing to the encoding stage, with advanced designs like Cross-Stitch Networks and MTAN introducing mechanisms to dy-

namically share and select features via attention mechanisms (S. Liu, Johns, & Davison, 2019) (Vandenhende et al., 2022). Decoder-focused models propagate information beyond initial predictions, using inter-task predictions to refine outputs, as seen in PAD-Net (Vandenhende et al., 2022). Modular and learnable structures further extend flexibility. Modulation modules adapt shared representations for specific tasks to prevent interference (Zhao, Li, Shen, Liang, & Wu, 2018). Learnable routing architectures dynamically determine where and how to share features or parameters, enabling adaptive branching based on criteria like task affinity. Some approaches build task-specific adapters by composing modules from a shared skill set (Crawshaw, 2020) (H. Wang, Sun, Fan, & Gu, 2023).

MTL has been adapted to various learning paradigms beyond supervised learning. In multi-task unsupervised learning, shared structure across clustering tasks is leveraged to improve clustering quality (Y. Zhang & Yang, 2017). Multi-task semi-supervised learning integrates unlabeled data to enhance learning across tasks. Multi-task reinforcement learning enables a single agent to concurrently learn across multiple Markov Decision Processes by sharing policies or value functions (Y. Zhang & Yang, 2022).. Multi-task online learning addresses sequential data arrival across multiple tasks, maintaining task-specific models or constraints as new data becomes available (Y. Zhang & Yang, 2017).

Chapter 4

Methodology

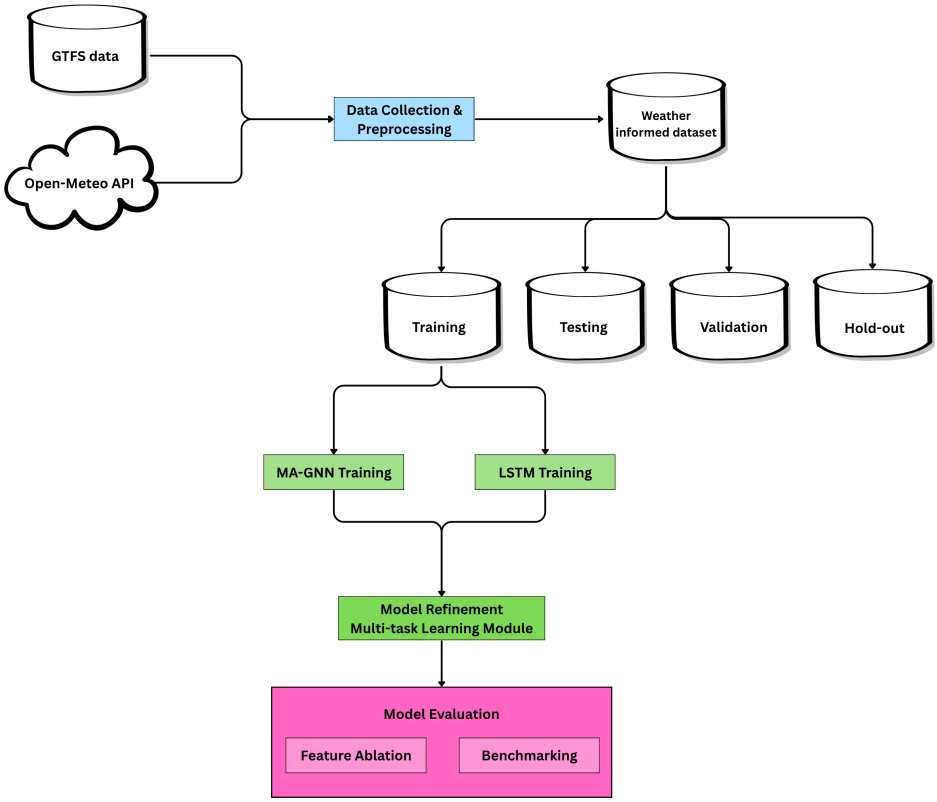

Figure 4.1: Overview Diagram of Methodology

Here, we describe the methodology for building the proposed hybrid deep learning model used for travel-time predictions, modified specifically for train transit networks. The proposed model builds off of the MAGTTE architecture, utilizing its mechanisms to enhance spatio-temporal representations and its specific robustness to situations with sparse data. To address error accumulation often encountered with solutions focused on individual segment predictions, a Multi-Task Learning module, proposed by the model AttentionTTE, is integrated into the design. This module allows the model to learn from both segment-level and overall-route level predictions simultaneously, therefore preventing the issue from happening. Figure 4.1 outlines the different phases of our methodology, covering data collection, preprocessing, model training and refinement, and evaluation.

4.1 Data Collection

This section describes the datasets used to predict travel time in the Canberra Light Rail Transit system. Similar to (Sarhani & Voß, 2024), we adopted a combination of static, real-time, and external data to capture dynamic behavior influencing transit travel time. Below, we describe the study coverage, data sources, and features used.

This study focuses on the Canberra Metro, light rail system operating in Canberra, Australia. The system's line is about 12 kilometers and consists of 14 stops, from Gungahlin place to the City Center. Figure 4.2 shows the structure of the Canberra light rail system and figure 4.3 shows a flat map of Canberra light rail, highlighting only the stations along the route.

Figure 4.2: Canberra Light Rail Map

Figure 4.3: Canberra Light Rail Stations

The GTFS static data includes information about scheduled transit operations. These datasets were used to construct a base reference of expected train movements and station geography. In our case, we collected the following data from the ACT Government Open Data portal (https://www.data.act.gov.au), (https://gtfs.org/documentation/schedule/reference/#feed_infotxt), and (https://www.transport.act.gov.au/__data/assets/pdf_file/0006/1259934/TCCS-GTFS-Implementation-Specification-v3.3-Bus-and-Light-Rail.pdf):

- stop_times.txt – contains scheduled arrival and departure times for each stop along a route.
- stops.txt – provides metadata for each stop, including stop ID, name, coordinates, and location type.

Moving on, the real-time GTFS data includes time-specific information about vehicle movements and arrival and departure times. The data collected are:

- **Trip updates** – contain actual and predicted arrival and departure times for trips, enabling comparison against scheduled times and identification of delays. We used this dataset as the foundation of our analysis, as it includes the necessary timestamps to calculate travel times, delays, and dwell time. It also allowed us to align other datasets based on trip and stop information. https://www.data.act.gov.au/Transport/Canberra-Metro-Light-Rail-Transit-Feed-Trip-Update/jxpp-4iiz/about_data
- **Vehicle position** – provides real-time coordinates and status information for each train. https://www.data.act.gov.au/Transport/Canberra-Metro-Light-Rail-Transit-Feed-Vehicle-Upd/92fy-xvmy/about_data

On the other hand, to incorporate weather data, we extracted it using the Open-Meteo API (https://open-meteo.com). Unlike other studies that used weather data of the whole location, we extracted weather data per station location. Variables collected include temperature, precipitation, windspeed, and etc., from the year 2021 to 2022.

The final dataset that would be used for training and evaluation contains a combination of trip updates, vehicle updates, weather conditions, and derived features. Below is the summary of the columns included in the processed dataset.

Table 4.1: Description of the Features of the Final Dataset

Feature	**Description**
trip_id	Unique identifier for each trip
stop_sequence	Sequence number indicating the stop order within a trip
stop_id	Unique identifier of the stop
arrivalDelay	Delay in seconds between scheduled and actual arrival time
arrivalTime	Actual recorded arrival time
depatureDelay	Delay in seconds between scheduled and actual departure time
depatureTime	Actual recorded departure time
originStopID	Unique identifier of the current location of the train
tripScheduleRelationship	Indicates the status of the scheduled trip (0=no_data, 1=scheduled, 2=skipped)
has_prev_stop	Indicates if the trip has previous stop based on stop_sequence
dwellTime_sec	Time in seconds the train spent at the stop (derived from depatureTime - arrivalTime)
is_peak_hour	Indicates if the trip falls within the peak hours
originStopName	Name of the current location
timestamp	Actual arrival time of the train on its next trip based on stop_sequence (e.g., for stop_sequence = 5, the arrivalTime of stop_sequence = 6 will be its timestamp assuming same trip_id)
vehicleID	Unique identifier of the train
vehicleLabel	Label for the vehicle
congestionLevel	Congestion status of trains (0=unknown, 1=running_smoothly, 2=stop_and_go, 3=congestion, 4=severe_congestion)
currentStatus	Status of the train (0=stopped_at, 1=in_transit_to, 2=incoming_at)

Continued on next page

Description of the Features of the Final Dataset

Feature	Description
bearing	Compass bearing or direction towards the next stop or intermediate location
destinationStopID	Unique identifier of the next stop
destinationStopName	Name of the next stop
has_vehicle	Indicates if the trip_id has vehicle record
was_skipped	Indicates if the stop was skipped by the train
travel_time_sec	Actual travel time from current stop to the next (timestamp - depatureTime)
origin_latitude	Latitude of the current location
origin_longitude	Longitude of the current location
destination_latitude	Latitude of the next location
destination_longitude	Longitude of the next location
distance_km	Total distance from current location to the next (computed using haversine)
speed_kph	Speed of the train during the trip (speed = distance / (travel time / 3600))
temperature_2m	Temperature 2 meters above ground
apparent_temperature	"feels-like" temperature
precipitation	Total daily precipitation
rain	Total rainfall
snowfall	Total snowfall
windspeed_10m	Wind speed 10 meters above ground
windgusts_10m	Wind gust speed 10 meters above ground
winddirection_10m	Dominant wind direction
is_slow_speed	Identifier if slow speed
is_long_dwell	Identifier if long dwell
is_delayed	Identifier if delayed
is_congested	Identifier if track is congested
is_slowdown	Identifier if slowdown
slowdown_lat	Slowdown latitude
slowdown_lon	Slowdown longitude

Continued on next page

Description of the Features of the Final Dataset

Feature	Description
segment	Slowdown segment (origin - destination)

4.2 Data Preprocesing

We started preprocessing by importing the needed libraries for data preprocessing. Then we loaded the required datasets: trip df.csv, vehicle df.csv, stops.txt, and stop times.txt. We also retrieved weather hourly data for 2021 and 2022 using the Open-Meteo API. This hourly granularity allows our model to capture environmental fluctuations that may affect travel time, such as sudden rain or wind changes during the day. Once everything was loaded and checked, we moved on to the next steps.

First, we used .isnull().sum() to check for missing values in each dataset. This helped us figure out which columns had the most null values and which rows to drop. In trip_df, the updateDeleted column had over 3.5 million nulls. In vehicle_df, several columns such as tripID, vehicleID, stopSequence, and others had more than a million missing values. While the other datasets had no critical missing data.

After checking for missing values, we dropped rows with important missing values and removed columns that weren't useful. In trip_df, we dropped rows where arrivalTime, depatureTime, or timestamp defaulted to 1970. We also dropped columns such as GTFSupdateID, updateDeleted, and tripUpdateSchedulerelationship. In vehicle_df, we removed GTFSRupdateID, updateDeleted, and odometer. From the stops and stop_times files, we removed columns like drop_off_type, location_type, and parent_station since they weren't relevant for our travel time modeling.

Next, we fixed the format of our data. We converted ID columns to int64, numerical ones to float64, and date fields to datetime. We also transformed categorical values numerically such as, tripScheduleRelationship as 0 ("no data"), 1 ("scheduled"), and 2 ("skipped"). congestionLevel and currentStatus were also transformed numerically.

We then added new columns to further improve the dataset. In trip_df, we created has_prev_stop to flag whether a stop had a previous stop in the same trip, using shift() on trip_id and stop_sequence. We also calculated dwellTime_sec as the time between arrivalTime and depatureTime. To identify peak hours (7–9

AM, 4:30–6 PM), we added an is_peak_hour column. Then, in vehicle_df, we created an is_weekend column by checking if the timestamp fell on a Saturday or Sunday.

After that, we started merging datasets. First, we merged the 2021 and 2022 weather data, making sure the columns and types matched. For trip and vehicle data, we converted timestamps to datetime and standardized all identifiers. We extracted the date from timestamps to create an identifier (trip_id, stop ID, stop_sequence, date) for merging. Using this, we created a lookup table from the trip data and merged it with vehicle_df, removing unmatched entries and duplicates. After that, we merged it back into the main trip dataset, so each row included matching vehicle data such as GPS and congestion level. The temporary columns were then removed.

Next, we added scheduled stop-times using stop_times.txt on trip–vehicle data. We also added a has_vehicle flag: 1 if it has vehicle data, 0 if none. We sorted by trip, date, and stop sequence, prioritizing real-time data. Then we also added fields that handle the logic behind previous and next trips such as destinationStopID and the next stop's timestamp using shift(-1). Stops without these values were dropped. Any stop missing from real-time data was marked with was_skipped = 1. We also added stop names by mapping stop_id values using stops.txt.

Now that we have the information merged together, we calculated travel time between stops using this formula: travel_time_sec = (timestamp - depatureTime).total_seconds(). We then joined coordinates for both origin and destination stops using the stops.txt. With the haversine formula, we calculated distance (distance_km) and then speed: speed_kph = distance_km / (travel_time_sec / 3600). Any infinite or invalid values were set to NaN and not dropped because we need to make sure that it doesn't affect the timestamps. Some trips skipped stops meaning they might have longer trips before they stop. The dataset was saved as merged_df_2.csv.

For weather integration, we matched station names from Open-Meteo with stop names. Since names sometimes do not align, we lowercase and clean them (removing platform numbers, applying manual fixes like 'epic and racecourse'). When there was no direct match,h, we inserted new values in missing weather values using daily averages from the weather dataset. After cleaning up temporary columns, we removed duplicates to ensure that each row was uniquely identified.

This weather dataset plays an important role in our model. The added weather details help the model learn how conditions like temperature, wind, or rainfall might affect travel times. On top of that, adding this data would enhance our use of different modules and techniques as this will have a huge impact on the

recognition of unusual behavior and detection of outlier trips or those that may be delayed or affected by external factors.

Before moving to normalization, we also added one more feature that identifies where trains tend to slow down. We defined a slowdown based on four indicators: low speed, long dwell time, significant delays or severe congestion. If any of these were met, we marked the point as a slowdown. To locate these points, we estimated their coordinates using a weighted average between each stop's origin and destination. This gave us more understanding of where and how the issues were occurring during trips, and would help us later on when we train our model to recognize patterns tied to delays.

This final dataset included travel times, delays, location info, weather data, and all derived features. Before training, we normalized the features. Models like MA-GNN and LSTM are sensitive to feature scale, so normalization helps them train more effectively and avoid bias. As (Ioffe & Szegedy, 2015) pointed out, normalization helps the model learn faster and more reliably by keeping the data going into each layer more stable.

We applied Z-score standardization to features with varying ranges or both positive and negative values such as arrivalDelay, depatureDelay, distance_km, speed_kph, temperature and wind features. However, for features that are strictly positive and bounded (dwellTime_sec, travel_time_sec), we used Min-Max scaling to map them between 0 and 1. For sparse and skewed features like precipitation_sum, rain_sum, and snowfall_sum, we applied a log transform before Min-Max scaling to reduce outlier impact. For spatial data (coordinates), we used delta encoding by measuring the distance from reference (main) stations. This helped the model focus on relative rather than absolute positions. We encoded the bearing angle between stops using sine and cosine to preserve its circular nature (0° 360°). These transformed spatial features were saved separately and merged only during model input construction to keep things organize.

4.2.1 Data Separation

To prepare for our data modeling, we divided the dataset into four parts using a 60-20-10-10 split: 60% for training, 20% for testing, 10% for validation, and the remaining 10% as a hold-out set. This structure gives the model enough data to learn from, while also providing dedicated subsets for tuning, testing, and final evaluation. Using 60% for training ensures that our model sees a wide range of data during learning. While the 20% testing allows us to evaluate performance after training, then the 10% validation helps us in possible adjustments of hyper-

parameters without the need to touch the test data. The final 10% for a holdout set, remains completely unseen throughout the development process and is only used to simulate real-world performance just before deployment.

This kind of split is commonly applied in practice, especially in deep learning and time-series processes where maintaining the order of events is critical. Several studies and industry guides recommend allocating 60-80% of data for training, 10-20% of data for testing and validation, with some suggesting a hold-out set for unbiased final evaluation. This approach helps avoid overfitting to the test set and ensures performance metrics reflect how the model would behave on future, unseen data (Nguyen, 2022; Encord, 2023; HogoNext, 2024).

For our data separation, we first merged the transformed spatial features into the main dataset using pd.concat and sorted the combined data by timestamp. We then used index-based splitting (.iloc[]) to divide the dataset into the four subsets. The training set will be used in training the models MA-GNN for spatial relationship and LSTM for temporal dependencies, then this will be tuned with the validation set, and evaluated on the test set using metrics such as MAPE, RMSE, MAE, and R^2, and finally assessed one last time with the hold-out data.

4.2.2 Consideration of Other Datasets

To ensure proper testing and refinement of our framework, we started working with transit data from Canberra, Australia. While Canberra is not a developing country like the Philippines, it offers the kind of quality, detailed open data we need such as network structure, reliable schedules, and vehicle status. We initially planned to work directly with data from the Philippines, since that is where we want this model to be useful. Unfortunately, we found that transit data in the Philippines is either very limited, not openly accessible, or do not include the features needed for our model to work. Due to this reason, Canberra became a starting point that allowed us to validate the core functionality of our model in a well documented environment.

However, our goal has always been to bring this model closer to real-world conditions similar to the Philippines. That led us to explore datasets from other developing countries across Asia, Africa, and Latin America. Among those we have explored, India (particularly Delhi) and Dubai stood out as strong alternatives with publicly available and somewhat complete datasets. Other countries such as Thailand, Colombia, Chile, and Tunisia do offer some data, but they were incomplete or inconsistent. In many places such as Vietnam, Indonesia, or Nigeria there is no openly accessible data.

If we can complete the testing in Canberra ahead of our schedule, we plan to try the model in places like Delhi or Dubai next. Their urban environments and transit challenges are closer to what we might see in the Philippines, and their datasets are promising enough to build on. This approach helps us build accurate, scalable, and eventually adaptable to the Philippine context once better local data becomes available or through future collaborations with transport agencies or even the government.

4.3 Software Design and Implementation

4.3.1 Model Overview

In (J. Ma et al., 2022b), the proposed MAGTTE model demonstrated reliable travel time predictions for bus transit systems, even under conditions of sparse data. Building on this, the present study proposes a modified version of MAGTTE, incorporating an additional module called the Multi-Task Learning which was adapted from AttentionTTE. These enhancements are intended to improve the model's ability to represent spatial interactions and contextual influences, particularly in transit systems with limited or uneven data coverage.

MAGTTE utilizes Graph Neural Networks and Long Short-Term Memory networks as a baseline model to capture spatial and temporal characteristics of bus transit systems. These networks are structurally closer to rail systems than to other forms of road-based transport, making MAGTTE a suitable foundation for rail-based applications. While automated train tracking is common in first-world countries, many railway segments, especially in underutilized or remote areas, still exhibit data sparsity. MAGTTE's state-of-the-art graph-based learning offers a robust method for handling such conditions.

However, the original MAGTTE design introduces a major limitation when applied to railway networks which is its reliance on isolated segment-level predictions which can lead to cumulative error across long trips. To address these issues, the proposed model incorporates AttentionTTE's Multi-Task Learning module, which improves generalization by jointly predicting segment-level and trip-level travel times.

4.3.2 Multi-Attention Graph Neural Network (MA-GNN) Components

Multi-View Graph Construction

The Multi-View Graph Construction Module is a core component of (J. Ma et al., 2022b)'s MAGTTE model's MA-GNN architecture. Discussed in this section is a modified Multi-Attention Graph Neural Network which is responsible for extracting the spatial dependencies between segments in a train network. It constructs multiple graphs that reflect different geo-structural relationships across the transit system. The graph construction module consists of two main subcomponents: **Node and Node Feature Extraction** and **Weighted Adjacency Matrix Generation**.

In order for the model to gain a better view of the transit system, it needs to transform the network into a graph of the railway network. To ensure this graph accurately represents the system, it is important to identify the key nodes, which correspond to the segments between stops and intersections. This is done by applying their **Density-Based Clustering Algorithm (DBSCAN) algorithm** to detect spatial clusters. These clusters help reveal frequent dwell points, or locations where trains commonly stop or slow down, which are essential for modeling delays and dwell-time-based travel time estimation. However, relying on a single method may bias the model toward a particular structural interpretation of the network. To mitigate this, multiple graph construction methods are used to evaluate the proposed hybrid model in order to test whether the model remains agnostic to the structure of the input graph.

Density-Based Clustering Algorithm

The Density-Based Clustering Algorithm (DBSCAN) is used to identify frequent stop locations and dwelling spots along the train transit network, which correspond to areas where the train is often stationary or has slowed down significantly due to scheduled stops or operational delays. Recognizing these points is critical for improving travel time estimation, as it enables the model to distinguish segments where delays are likely to occur. By assigning greater or lesser weights based on their influence on overall travel time, the model can more accurately estimate delays for trips that pass through high-impact stops compared to those that do not. This process involves two main algorithms: geo-structure discovery algorithm, which reveals the underlying spatial layout of the transit network, and nodes extraction, which defines the representative points for graph-based model-

ing. The DBSCAN algorithm takes on two parameters: **eps and minPts.**

- **eps (epsilon)**: defines how close two GPS points need to be to be considered "neighbors" or part of the same clustering group.
- **minPts (minimum number of points)**: specifies the minimum number of GPS points are required to form a cluster.

```
1   Input: t = {p1, ..., pn} in G, where speed of p = 0 or p has significantly
    decreased speed before any known stop
2   Output: the geo-structure of train routes, represented by a sequence of
    locations, including stops and intersections
3           C = {c1, ..., cn} where c is a cluster generated by DBSCAN
4
5   while t of train routes do
6       DBSCAN(P, minPts, eps)
7       return {c1, ..., cn} where c is a cluster generated by DBSCAN
8       if unique number of (c) < number of stops then
9           minPts = minPts - 1
10
11  for c in C do
12      c(long, lat, num, st) = (centerof(c.long), c.lat, number of c)
    INTERSECT stops(long, lat)
13
```

Listing 4.1: Geo-structure Discovery Algorithm

Table 4.1 Algorithm 1 describes the geo-structure discovery. To identify potential dwell areas such as intersections and stops, the algorithm first uses segments of train trajectories where the vehicle speed is zero or has significantly decreased before any known stop. In steps 3 to 6, DBSCAN is applied separately to each train route. This while loop will continue until the ***number of clusters (c)*** exceeds the ***number of known stops (st)*** to ensure all important intersections or stops have been identified. Each resulting cluster contains four key features: ***longitude (long)***, ***latitude (lat)***, and the ***number of GPS points (num) within the cluster***, and the ***stop label (st)***. These features of the clusters help determine the location and length of ***travel segments (S)*** in the graph, while the num value gives insight into how long trains typically dwell at those locations because a larger value means that there is a dense cluster of data points which may also dictate the traffic or dwelling situation at that location. To know which clusters are the known stops or stations in the network, the algorithm uses the ***stop label (st)*** which is a boolean flag. If a stop lies within a cluster, the label is set to 1; otherwise, it is set to 0. If a stop does not fall within any existing

cluster, it is added as a new cluster. ***The final set of clusters (C)***, enriched with these labels, becomes the input for node extraction in Algorithm 2.

```
    Input: C = {c1, ..., cn} in G
    Output: The node set S of transport network graph G, represented by
    selected locations {s1, ..., sn}

    for all ci in C:
        if ci(st) == 1 and c_{i+k}(st) == 1 then
            extract {ci, c_{i+1}, ..., c_{i+k}}
            for c_{i+1} to c_{i+k-1} do
                c_{i+1}(dist) = perpendicular_distance(ci, c_{i+k}, c_{i+1})

    s(long, lat) = locations of {ci, c_{i+1}, ..., c_{i+k}}

```

Listing 4.2: Nodes Extraction Algorithm

Table 4.2 Algorithm 2 describes the nodes extraction. It iterates through the sequence of clusters, previously outputted by Algorithm 1, to identify pairs of adjacent clusters that correspond to known stops. These pairs define the boundaries of potential train network segments. For each pair of known stops, the algorithm scans the clusters located in between them (line 3). If a cluster lies in between two stops or represents an intersection, it calculates the perpendicular distance from that cluster to the line connecting the two stop locations (line 4-6). This allows the model to understand the geometric relationship of intermediate points to the main transit path. The output of this process is a set of key locations, including both stop and intersection points. For each pair of adjacent stops, the module then selects the middle location between these intermediate points as the node representing that segment of the train network. This approach enables the model to distinguish between different paths that may share the same station stops but follow a different train path.

The final result is a structured set of nodes, each representing either an intersection or a stop. Each ***node (s)*** still includes all 4 features previously mentioned in algorithm 1's output with an additional feature called the embedded temporal feature vector. The temporal feature vector includes temporal features such as time slot (peak/off-peak) and day type (weekday/weekend) and is embedded using the Word2Vec algorithm to enrich each node with temporal information; therefore, enhancing multi-attention graph neural network.

Distance-Based Graph Construction

The Distance-Based Graph Construction method, adapted from GMAN's (C. Zheng, Fan, Wang, & Qi, 2020) approach, provides an alternative graph representation that focuses on the spatial proximity relationships between stations in the train network. While our primary graph construction method leverages DBSCAN to identify key spatial nodes and infer a geo-structural layout based on operational patterns, this alternative method constructs the weighted adjacency matrix purely based on the geographical proximity between identified nodes. Unlike clustering-based methods that identify functional areas or dwelling points, this approach creates a graph where the strength of connections between stations is determined purely by their geographical proximity, following the principle that nearby stations are more likely to influence each other's travel patterns. This allows us to evaluate the model's ability to learn from diffuse spatial influences, such as how events at a nearby stop might indirectly impact the travel time of a train segment.

In this construction, each unique station along the Canberra Light Rail Transit system is defined as a node $v_i \in V$, where V is the set of all N stations. This aligns with the fixed "sensor" concept in the original GMAN paper, adapted to the railway context where stations represent key, immovable points in the network. The core of this method lies in the construction of a weighted adjacency matrix $A \in \mathbb{R}^{N \times N}$, where each element $A_{i,j}$ quantifies the geographical proximity or correlation between station i and station j. This matrix reflects the inherent idea that nearby stations are more likely to share similar travel time characteristics due to their spatial relationship.

The weight $A_{i,j}$ between any two stations v_i and v_j is determined by their geographical distance, $d_{i,j}$, using a Gaussian kernel function. This ensures that the influence of one station on another diminishes rapidly with increasing distance. The formula for the adjacency matrix element $A_{i,j}$ is as follows:

$$A_{i,j} = \begin{cases} \exp\left(-\frac{d(s_i,s_j)^2}{\sigma^2}\right) & \text{if } \exp\left(-\frac{d(s_i,s_j)^2}{\sigma^2}\right) \geq \epsilon \\ 0 & \text{otherwise} \end{cases} \tag{4.1}$$

where:

- $d(s_i, s_j)$ is the Haversine distance between stations s_i and s_j.
- σ is the bandwidth parameter that controls the decay rate of influence with distance. A smaller σ results in a sharper decay, indicating that only very close stations have strong connections.

- ϵ is the sparsity threshold (0.1) that eliminates very weak connections and ensures that the graph primarily captures strong, relevant local interactions.

The parameters σ and ϵ are tuned based on the network characteristics and experimental objectives:

- σ will be set relative to the average inter-station distance in the network to ensure meaningful weight distribution.
- ϵ controls the graph sparsity, allowing for experimentation with different levels of connectivity.

Construction Process:

1. **Station Identification:** All unique stations in the Canberra Light Rail network are identified and mapped to numerical IDs, forming the set of nodes V.
2. **Distance Calculation:** For every pair of stations (v_i, v_j), their geographical distance $d_{i,j}$ is computed using the haversine formula, leveraging the latitude and longitude coordinates available in the preprocessed data.
3. **Matrix Population:** An $N \times N$ adjacency matrix A is initialized with zeros. For each pair (i, j):
   - If $d_{i,j} \leq \epsilon$, the weight $A_{i,j}$ is calculated using the Gaussian kernel: $A_{i,j} = \exp\left(-\frac{d_{i,j}^2}{\sigma^2}\right)$.
   - Otherwise, $A_{i,j}$ remains 0.
4. **Normalization:** The resulting matrix can be normalized to ensure values are within a consistent range, typically $[0, 1]$, depending on the specific GNN layer requirements. This can involve row-wise normalization (e.g., $A_{\text{norm}} = D^{-1}A$, where D is the degree matrix) or other methods to maintain numerical stability and consistent feature scaling.

```
Input: Station coordinates S = {s1, s2, ..., sn} where si = (lat_i, lon_i)
       Bandwidth parameter sigma, sparsity threshold epsilon
Output: Weighted adjacency matrix A (n x n real-valued matrix)

Initialize A with zeros of size n x n

for i = 1 to n do
    for j = 1 to n do
        if i != j then
            d_ij = calculate_haversine_distance(si, sj)
            w = exp(-d_ij^2 / sigma^2)
            if w >= epsilon then
                A[i][j] = w
            else
                A[i][j] = 0
        else
            A[i][j] = 0  // No self-loops

for i = 1 to n do
    r = sum over j of A[i][j]
    if r > 0 then
        A[i][:] = A[i][:] / r  // Row normalization

return A
```

Listing 4.3: Distance-Based Graph Construction Algorithm

Multi-Aspect Graph Construction

To further evaluate the robustness and generalizability of the proposed hybrid model, particularly its agnosticism to the specific structure of the input graph, we adopt an alternative graph construction methodology inspired by the MTMAG model (Tong, Xing, Cao, & Zhao, 2022).

For this evaluation, we define the nodes $v_i \in V$ in the graph $G = (V, E, W)$ as the distinct train segments identified by our DBSCAN and Node Extraction algorithms. The construction process involves defining two distinct weighted adjacency matrices that capture different facets of inter-segment relationships. These matrices are subsequently used to inform the initial node embeddings within the Multi-Attention Graph Neural Network (MA-GNN).

Let s_i denote a unique train segment identified from the preprocessed dataset. Each segment s_i corresponds to a node $v_i \in V$ in the graph G. For each node v_i, we construct a multi-source contextual feature vector $\mathbf{C}_{v_i}$. This vector aggregates relevant segment-level and contextual information extracted from the

preprocessed dataset (refer to Table 4.2.2). The feature vector $\mathbf{C}_{v_i}$ includes the following components:

- **Physical Characteristics:**
  - Segment length in kilometers: `distance_km`
- **Operational Metrics:**
  - Aggregated dwell time at stations (in seconds): `dwellTime_sec`
  - Average speed (in kilometers per hour): `average_speed_kph`
  - Estimated congestion level: `congestionLevel`
- **Temporal Context:**
  - Peak hour indicator: `is_peak_hour`
  - Day type (e.g., weekday/weekend/holiday): `day_type`
- **Weather Conditions:**
  - Aggregated weather features relevant to the segment's geographical area
  - Average weather conditions at the start and end locations of the segment across the relevant time period

The Contextual Feature Vector Generation Algorithm describes how to generate contextual feature vectors for each railway segment node using a preprocessed dataset.

```
Input: Preprocessed dataset D containing segment-level, temporal, and weather data
Output: Dictionary CV containing contextual feature vectors for each unique segment

Initialize empty dictionary CV
Identify all unique segments V_unique from D

for each segment si in V_unique do
    Extract all records R_si from D where segment == si
    Initialize empty feature vector C_si

    // Physical metric
    C_si["distance_km"] = average distance

    // Operational metrics
    C_si["avg_dwellTime_sec"] = average dwell time
    C_si["avg_speed_kph"] = average speed
    C_si["mode_congestionLevel"] = mode of congestion level

    // Temporal context
    C_si["is_peak_hour_ratio"] = ratio of peak hour records
    C_si["weekday_ratio"] = ratio of weekday records

    // Weather features
    C_si["avg_temperature_2m"] = average temperature at 2m
    C_si["total_precipitation"] = total precipitation
    C_si["avg_windspeed_10m"] = average windspeed at 10m

    // POI features (if available)
    C_si["poi_category_distribution"] = POI distribution

    Store C_si in CV with key si

Normalize all features in CV (e.g., Min-Max or Z-score scaling)
return CV
```

Listing 4.4: Contextual Feature Vector Generation Algorithm

Constructing the Reachability-Based Adjacency Matrix (A) This matrix models spatial correlations based on the shortest reachable time between train segments v_i and v_j, reflecting the assumption that segments closer in operational travel time may exhibit stronger interdependencies. To implement this, we define a directed graph where nodes represent train segments and edge weights correspond to average travel times between temporally adjacent segments.

For each observed trip in the training dataset, compute the travel time in seconds for every traversed segment. Let travel_time_sec(s_i) denote the average observed duration for segment s_i across all trips.

Then we construct a directed graph $G_{\text{time}} = (V, E, W)$, where:

- Each node corresponds to a unique train segment $s_i \in V$.
- A directed edge $(s_i, s_j) \in E$ exists if segment s_j immediately follows segment s_i in any recorded trip.
- The weight $w_{i,j} \in W$ is defined as the average travel time (in seconds) for segment s_j, i.e., $w_{i,j} = \text{travel_time_sec}(s_j)$.

For every pair of segments v_i and v_j in the network, compute the shortest path travel time disr_{v_i,v_j} using a shortest path algorithm (e.g., Dijkstra's algorithm). If no path exists, disr_{v_i,v_j} can be set to infinity or a very large value.

```
Input: Unique segments V_unique; average travel times avg_travel_times;
trip sequences from dataset D
Output: Matrix M_disr with shortest travel times between all segment pairs

Initialize M_disr as N x N matrix of INF, with zeros on the diagonal

Build a directed graph G_segments where nodes are segments in V_unique

for each trip in D do
    for each adjacent segment pair (s_prev, s_curr) do
        Add edge s_prev -> s_curr with weight avg_travel_times[s_curr]
        (update if necessary)

for each segment si in V_unique do
    Run Dijkstra's algorithm from si on G_segments
    Fill row i of M_disr with shortest travel times to all sj
    Set M_disr[i][j] to a large constant if no path exists

return M_disr
```

Listing 4.5: Shortest Path Travel Time Calculation Algorithm

This algorithm calculates the shortest travel time between every pair of railway segments using average travel times and observed trip data. It builds a graph from the segments and their connections, then applies Dijkstra's algorithm to compute the shortest path travel times between all pairs of segments.

After computing the shortest travel time matrix M_{disr}, we convert these values into edge weights for the reachability-based adjacency matrix A. The weight between nodes v_i and v_j is determined using a Gaussian kernel:

$$A_{v_i v_j} = \begin{cases} \exp\left(-\frac{\text{disr}^2_{v_i v_j}}{\sigma_A^2}\right) & \text{if } \exp\left(-\frac{\text{disr}^2_{v_i v_j}}{\sigma_A^2}\right) \geq \epsilon_A \\ 0 & \text{otherwise} \end{cases} \tag{4.2}$$

Here, σ_A is a bandwidth parameter controlling the decay rate, and ϵ_A is a threshold that filters out weak connections. This results in a weighted adjacency matrix that captures temporal reachability between segment nodes.

Constructing the Contextual Similarity Adjacency Matrix (B) In parallel with travel-time reachability, we also define an adjacency matrix B that reflects functional similarity between segments. We compute the cosine distance between each pair of contextual feature vectors $\mathbf{C}_{v_i}$ and $\mathbf{C}_{v_j}$:

$$\text{disf}_{v_i v_j} = 1 - \frac{\mathbf{C}_{v_i} \cdot \mathbf{C}_{v_j}}{||\mathbf{C}_{v_i}|| \cdot ||\mathbf{C}_{v_j}||} \tag{4.3}$$

This distance is transformed into edge weights using an exponential decay function similar to the one used for A:

$$B_{v_i v_j} = \begin{cases} \exp\left(-\frac{\text{disf}^2_{v_i v_j}}{\sigma_B^2}\right) & \text{if } \exp\left(-\frac{\text{disf}^2_{v_i v_j}}{\sigma_B^2}\right) \geq \epsilon_B \\ 0 & \text{otherwise} \end{cases} \tag{4.4}$$

Where σ_B is a contextual similarity bandwidth parameter and ϵ_B is a threshold for pruning weak functional relationships.

Once both A and B have been constructed, they are used to generate spatial embeddings for each node v_i. There are two primary approaches:

- **Dual GNN Streams:** Separate GNN modules are applied to A and B to generate two sets of embeddings, which are concatenated and passed through a fully connected layer.
- **Multi-View GNN:** Treat A and B as separate views and process them jointly using attention-based fusion layers within the MA-GNN.

The final output is a spatial embedding vector SE_v for each node v_i, capturing both reachability and contextual similarity.

Mixed Embedding Fusion. These spatial embeddings are combined with the historical temporal embeddings HE_t (generated as described in Section 4.1) to form the mixed embedding $\mathrm{ME}_{v,t}$:

$$\mathrm{ME}_{v,t} = \mathrm{concat}(\mathrm{SE}_v, \mathrm{HE}_t) \tag{4.5}$$

This mixed embedding is then input to the multi-attention module, allowing the model to dynamically learn spatial-temporal patterns.

The bandwidth parameters (σ_A and σ_B) and connectivity thresholds (ϵ_A and ϵ_B) are crucial for controlling the graph's structure and sparsity. The parameters governing the graph construction are:

- σ_A and σ_B: bandwidths controlling the decay of influence with respect to travel time and contextual similarity.
- ϵ_A and ϵ_B: thresholds for pruning low-weight edges and controlling graph sparsity.

These parameters are tuned using the validation set to balance connectivity and sparsity, ensuring robust learning of meaningful patterns.

Weighted Adjacency Matrix Generation

To predict train travel times, especially for segments with limited or sparse data, it is crucial to learn travel patterns from fully recorded routes while maintaining a comprehensive view of the entire transit network. This involves accurately modeling the relationships between every pair of travel segments within a graph structure. Train travel presents unique challenges compared to other vehicle types like taxis, as it is influenced by more complex and structured factors. These include fixed departure schedules, the number and location of stops which affect dwell times, predetermined paths, and the varying social functions or land use surrounding stations. These contextual elements result in distinct and independent travel patterns, even when multiple routes share segments of the same track. To effectively capture these diverse influences, the model constructs multiple weighted adjacency matrices. Each matrix forms part of a ***graph G = (S,***

E, W), where ***S*** represents the set of segments (nodes), ***E*** the edges (connections between segments), and ***W*** (the weights stored in each matrix). Overall, there will be 3 matrices representing each view, namely the distance, geographical, social-function, of the graph based on what factors the correlation values take into account. These weights represent the ***correlations (corr)*** or similarities between segments, allowing the model to understand spatial dependencies and learn more robust travel time estimations. Adjacency matrices are visualized in (J. Ma et al., 2022a)'s paper as seen in equation 4.6. Additionally, the correlation matrices may also be visualized as an adjacency table for better interpretability as seen in Figure 4.4.

$$\mathbf{A} \in \mathbb{R}^{N \times N} = \begin{bmatrix} \text{corr}_{(1,1)} & \cdots & \text{corr}_{(1,n)} \\ \text{corr}_{(2,1)} & \cdots & \text{corr}_{(2,n)} \\ \vdots & \ddots & \vdots \\ \text{corr}_{(n,1)} & \cdots & \text{corr}_{(n,n)} \end{bmatrix} \tag{4.6}$$

$$A = \in R^{NxN}$$

	S_1	S_2	S_3	S_4
S_1	0	$corr_{(2,1)}$	$corr_{(3,1)}$	$corr_{(4,1)}$
S_2	$corr_{(1,2)}$	0	$corr_{(3,2)}$	$corr_{(4,2)}$
S_3	$corr_{(1,3)}$	$corr_{(2,3)}$	0	$corr_{(4,3)}$
S_4	$corr_{(1,4)}$	$corr_{(2,4)}$	$corr_{(3,4)}$	0

where S_i x S_j is $corr_{(i,j)}$

Figure 4.4: A sample adjacency matrix between segments in the train line.

A Distance Graph is constructed to quantify the relationship between travel segments based on their spatial proximity. The distance graph states that nearby travel segments are more strongly related than distant ones, as they tend to share similar real-time traffic conditions. For instance, if traffic congestion occurs on one road segment, its effects, such as reduced speeds, often propagate to surrounding areas. To determine the ***weights (W)***, we utilize reciprocal distance between nodes (travel segments), where W = $distance^{-1}$. This ensures that the closer the travel segments are, the higher their assigned weights will be. Moreover, to standardize these relationships, the weights of this distance graph are normalized to a range between 0 and 1.

The Geographical Graph is constructed to capture the inherent idea that road segments with similar geo-structures frequently exhibit similar travel patterns. This graph is further broken down into two distinct sub-factors, each represented by a sub-matrix: ***Road Length*** (G_{gl}) and ***Dwelling Times*** (G_{gd}). To quantify these relationships, the method employs a **cosine similarity function** to determine the weights, meaning that based on the similarity in road length and a node's clustering, the weights will range from -1 to 1, with a value of 1 indicating a stronger, more similar relationship. For ***Road Length*** (G_{gl}), ***the weights (W= corr(road length))*** are based on the correlation of road lengths between segments. A higher weight is assigned when segments have similar road lengths, as they are likely to experience similar travel times under comparable traffic con-

ditions. Meanwhile, for ***Dwelling Times*** (G_{gd}), ***the weights (W= corr***(c_i, c_j)***)*** are derived from the correlation between the sets of clusters associated with different segments. This leverages ***num (number of GPS points)*** extracted for each cluster, which helps identify the waiting times at stops and intersections. A higher weight in this context indicates a greater number of GPS instances where a train or bus is dwelling, suggesting a higher probability of traffic impact and, consequently, a stronger relationship in travel time patterns.

The Social Functional Graph is designed to represent relationships where regions or stations with similar social functions, such as commercial or residential areas, tend to share similar travel patterns. This similarity can significantly influence traffic flow and passenger demand within the network. To quantify these relationships, the ***weights (W)*** in this graph are determined based on the correlation of social functions ***(corr***(f_i, f_j)***)*** between travel segments. These social functions are typically derived from the density and categories of Point of Interest (POI) data associated with each segment. The method utilizes a cosine similarity function to calculate these correlations, ensuring that a higher weight is assigned to segments exhibiting greater similarity in their social functions.

Multi-Attention Neural Network

MAGTTE's proposed Multi-Attention Graph Neural Network architecture is made up of two key modules: a spatial attention module and a temporal attention module. These modules are dedicated to discovering spatial dependencies and temporal influences, respectively, which are then combined using global and local attention mechanisms to produce travel time predictions that reflect how factors affect these estimations realistically. The travel time sequence is denoted by time series $\boldsymbol{T} = \{t_1, \ldots, t_n\} = \{x_1, \ldots, x_n\}$ which essentially represents the travel times for each segment of the network.

Referenced by the MAGTTE model, the Graph Attention Network's base component is its Graph Attention Layer which is denoted by the ***variable (l)***. This layer allows the module to learn the correlations between node pairs and update their hidden features to enhance its model accuracy. The node feature in ***layer (l)*** in ***time interval (t)*** is denoted as:

$$h_1^t \in \mathbb{R}^{d^{(l)}} \tag{4.7}$$

where $\mathbf{d(l)}$ is the length of the feature of travel segment s_i at layer l and $\mathbf{h}_i^t$ is the input travel time record of segment s_i and the embedding temporal

information in the first layer.

An integral part of the attention mechanism is the attention coefficients assigned to each travel segment or node. These coefficients quantify the correlation of nodes to each other and help the model understand how strongly related the segments are. The attention coefficient of s_i and s_j is represented as:

$$e_{ij}^t = a(Wh_i^t, Wh_j^t) \tag{4.8}$$

where ***W*** is the learnable parameter of ***layer (l)*** and ***a(..)*** is the function that calculates the correlations or attention coefficients.

The Spatial Attention Module also uses the **LeakyReLU Activation Function** to train the feedforward neural network. Moreover, the softmax function was used to normalize the output of each layer to a [0,1] range. The complete formula for utilizing these two functions to calculate the normalized, final and weighted attention coefficient is represented as:

$$a_{ij}^t = \text{softmax}\left(\text{LeakyReLU}\left(a(Wh_i^t, Wh_j^t)\right)\right) \tag{4.9}$$

To gain a more comprehensive and robust understanding of diverse travel patterns and influences among trains, the spatial attention module is extended to incorporate multiple attention heads with each of these heads operating with its own unique set of learnable parameters. Instead of a single attention calculation, K independent and parallel attention heads are employed and concatenated and is represented as:

$$h_i' = \|_{k=1}^{K} \delta \left(\sum_{j=1}^{N} a_{ij}^t W h_i^t \right) \tag{4.10}$$

where || represents concatenation.

An attention mask matrix effectively increases the prediction accuracy for train trips located in the suburbs or those under design, which typically suffer from sparse or non-existent historical records. When a target train segment has complete historical data, its own records provide reliable data for travel time prediction. However, segments with significant data sparsity or no records often have historical data that is mostly zero or null values. These introduce "dense noise" that can decrease prediction accuracy. By adding masks, the model is

directed to focus its attention on neighboring nodes that possess fully-recorded data; thus, providing more accurate prediction of travel times for these challenging train trips. MAGTTE utilizes a soft-masking algorithm that assigns a continuous value for all nodes while masking or diminishing the weights of those who are considered "less relevant" to the estimate.

Mask matrix (M) for a certain ***node*** ***(s_i)*** is used to determine whether the node (s_i) can be aggregated as a **support feature** for node (s_j). The mask determines which nodes to avoid based on their geo-distance in which if the geo-distance of another node is greater than **attribute (y)** then it will not function as a good support feature for the node. If the calculated attention coefficient is greater than y, then the ***mask element*** $m_t^{(l)}$ is set to 1. This ensures that nodes that have a positive correlation to the node have a positive and larger attention coefficient and vice versa for the nodes with no reliable data and are therefore, negatively correlated to the focused node. The masks can be represented as:

$$\mathbf{m}_t^{(l)} = \begin{cases} 1, & \text{if } a(Wh_{ti}, Wh_{tj}) > \gamma \\ -1, & \text{if } a(Wh_{ti}, Wh_{tj}) < \gamma \end{cases} \tag{4.11}$$

where m_t is a $N \times N$ matrix, similar to attention matrix A from the weighted adjacency matrix module.

In each layer, its outputs include a normalized N x N attention matrix A with mask matrix M. Thus, when element-wise multiplied to the mask matrix, the output (X) of layer (l) has the following formulation: $Xl'= (A \cdot M)X'$ where $A \cdot M$ is the element-wise product of A and M.

Each of these outputs from each Masked-Attention Head Layer is concatenated to form the spatial vector for a node as seen in Figure 4.6.

In summary, each node contains a spatial embedding vector with learnable values based on the weights of each attention matrix per geographical, distance, or social-function view.

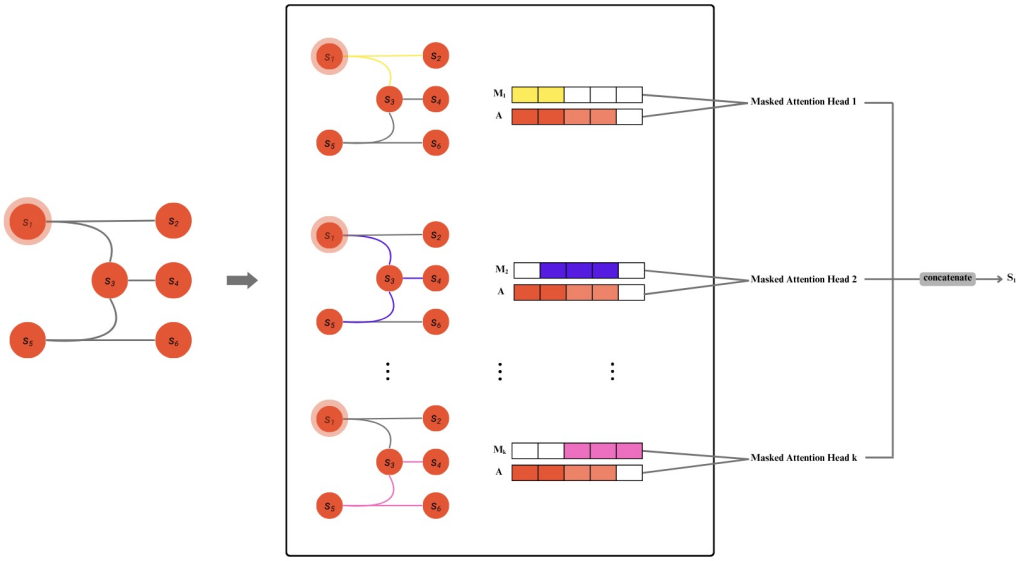

Figure 4.5: An Illustration of the Concatenation of Mask Matrices

4.3.3 Long-Short Term Memory(LSTM) Components

After receiving the Spatial-Temporal Dataset (Trajectories) output from the spatial-temporal module, the Long Short-Term Memory (LSTM) will be used to capture the short-term historical travel patterns and local temporal dependencies from train trajectories. The LSTM process will include four main components: gathering historical trajectories, passing them through the LSTM layer, and integrating the global temporal attention.

Gathering Historical Trajectories

To begin, the historical train trajectory data (the output of the spatial attention module) is collected and structured as input to the LSTM. These trajectories consist of sequences of observations recorded along the train's route, which is represented as:

$$\mathbf{t}_t = \{t_1, t_2, \ldots, t_n\}.$$

This sequence is then concatenated with the temporal features—operational

delay (od_t) and weather conditions (w_t)—and redefined as:

$$\mathbf{x} = \{x_1, x_2, \ldots, x_n\},$$

where each observation point x_t represents the spatially-aware hidden feature at time t along with the concatenated operational delay and weather features. That is,

$$x_t = t_t + od_t + w_t.$$

In this formulation, each x_t incorporates the spatial dependencies derived from the MA-GNN, as well as contextual features related to operational delays and weather. These enriched representations are then used to learn short-term temporal dependencies of travel times across consecutive time steps.

LSTM Layer

The Long Short-Term Memory (LSTM) is a recurrent neural network (RNN) specifically designed to model sequential data and capture both short- and long-term dependencies in the input sequence. It achieves this by maintaining an *internal cell state* that acts as a memory and three gating mechanisms to regulate the flow of information at each time step.

Figure 1 illustrates the overall process of how the LSTM operates. After the historical train trajectories are preprocessed, they are sequentially fed into the LSTM layer, which functions as a crucial component of the temporal module in the MA-GNN-LSTM framework. The LSTM will then process the sequence one time step at a time. Initially, at time step **t**, the LSTM receives the input vector x_1 = **{stop_lon, stop_lan, timestamp}** and processes it by updating its internal state (the cell) C_t and computing the output h_t. The output h_t is then passed alongside the next current input x_t, allowing it to remember the previous steps of the sequence. Following this, for every input, it receives the output of the previous step h_{t-1}, the input of the new step x_t, and the previous cell state, C_{t-1}.

The LSTM '**state**' here is a cell that acts as the persistent memory of the network, as shown in Figure 2, operating using three gates: forget gate (f_t), input gate (i_t), and output gate (o_t). At each step, the input flows through these gates sequentially. The first gate the input encounters is the forget gate f_t =

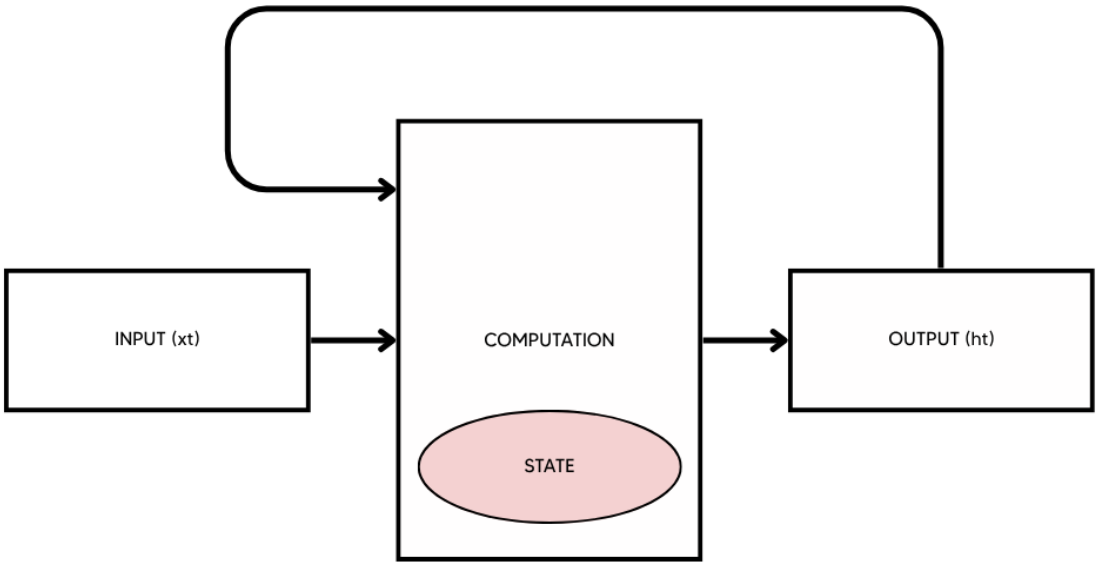

Figure 4.6: Overview of LSTM

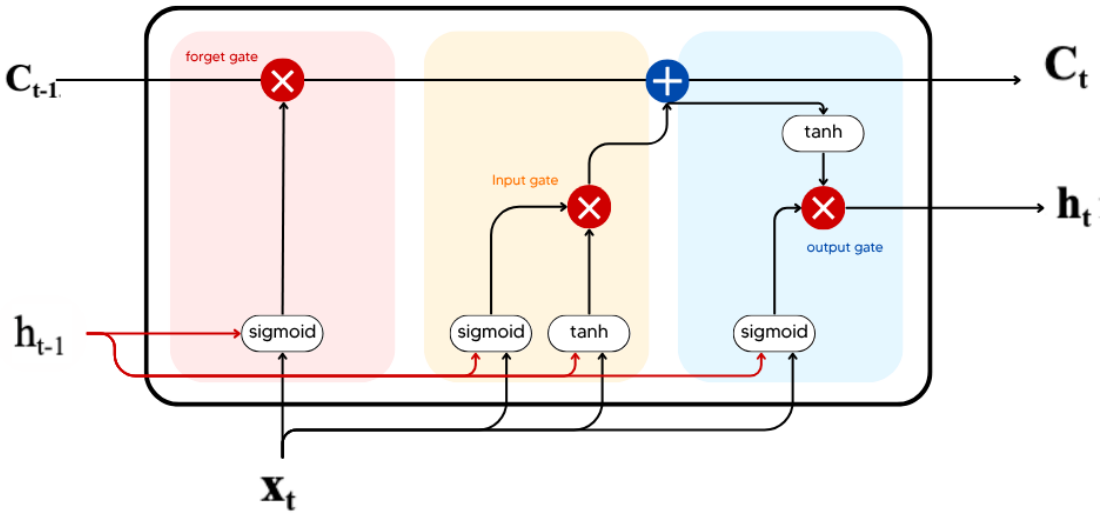

Figure 4.7: LSTM cell

$\sigma(W_{fx}X_t + W_{fh}h_{t-1} + b_f)$, which will determine how much of the previous cell state C_{t-1} to retain and which should be forgotten.

- W_{fx} is the weight matrix connecting the input X_t to the forget gate.
- W_{fh} is the weight matrix connecting the hidden state h_{t-1} to the forget gate.
- b_f is the bias vector of the forget gate.

The sigmoid function σ squashes its input into the range [0,1], to control the flow of information, where a value of $f_t = 0$ means forgetting the part of the memory and $f_t = 1$ means retaining most of C_{t-1}.

After determining what to forget, the LSTM decides what new information to add or update to the cell state using the input gate $i_t = \sigma(W_{ix}X_t + W_{ih}h_{t-1} + b_i)$, where W_{ix}, W_ih, and b_i are weights and bias associated with the input gate. At the same time, the LSTM generates a candidate state cell $\hat{C}_t = tanh(W_{cx}X_t + W_{ch}h_{t-1} + b_c)$ that represents the new memory content of the cell. The input gate i_t and the candidate state $\hat{C}_t$ will work together to determine how much of this new information to incorporate into the cell state. The updated cell Ct from this will then be computed by combining the retained memory from the previous state Ct-1 and the new candidate state $_t$, resulting in the formula: $C_t = f_tC_{t-1} + i_t\hat{C}_t$. Lastly, the output gate $o_t = (W_{ox}X_t + W_{oh}h_t - 1 + b_o)$ determines which information stored in the cell state should be outputted as the hidden state h_t. The output h_t will then serve as both the output at time step t and as input h_{t-1} for the next time step. This LSTM layer will then output a sequence of hidden states h_t, which summarizes the learned short-term and medium-term temporal patterns from the input data.

After processing the full sequence $x = x_1, x_2, \ldots, x_t$, the LSTM produces a sequence of hidden state vectors $h = h_1, h_2, \ldots, h_t$, where each hidden state h_t at time step t encodes the LSTM's understanding of the sequence up to that point, effectively summarizing the trajectory history for short-term dependencies. These h_t vectors are then extracted and aggregated as intermediate representations of the temporal dynamics of the trajectory data.

Global Temporal Attention

The LSTM focuses on local short-term temporal patterns, but to capture long-range temporal dependencies, global temporal attention is needed. Therefore, global temporal attention will be used for the model to consider all time steps at

once, learning which parts of the sequence (whether near or far) are most relevant for predicting the target. After the historical trajectories are processed through the LSTM layer and produce a sequence of hidden states $h = h_1, h_2, \ldots, h_t$, these hidden states are fed into the global temporal attention module to compute a globally-informed temporal embedding.

The input for this will be $X = h_1, h_2, \ldots, h_t \in R^{Txd}$ where T is the number of time steps and d is the dimension of the hidden state at each time step. The global attention mechanism will compute for each timestep a weighted sum of the representations of all other time steps, where the weights reflect the relative importance of each time step for predicting future travel times. The input sequence X is linearly projected into three different vector spaces : the query (Q), key (K), and value (V):

$$Q = X_i W^Q, \quad K = X_i W^K, \quad V = X_i W^V \tag{4.12}$$

where W^Q, W^K, W^V are learnable weight matrices.

For each time step t, a query (Q) vector is computed and compared to all key (K) vectors in the sequence to measure how much attention t should pay to each other step t. The weighted sum of the corresponding value (V) vectors then gives the new representation at t, incorporating information from across the whole sequence. Next, the similarity between each query and all keys is computed using a scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{4.13}$$

Here, the softmax function normalizes the scores so they sum to 1, producing a set of weights that indicate the relative importance of each historical time step in predicting the current output. Finally, the weighted sum of the value vectors V produces a global temporal embedding vector, which encapsulates long-range temporal dependencies from the entire sequence. The output of the global temporal attention module is $\hat{X} = \hat{h_1}, \hat{h_2}, ;\hat{h_t}$ where each $\hat{h_t}$ is a globally-informed representation of the sequence at time t, incorporating information from all other time steps. Now the vector captures both local (short-term) and global (long-term) temporal patterns observed in the data.

4.3.4 Multi-Task Learning Module Components

The Multi-Task Learning (MTL) module offers key advantages for travel time estimation models, particularly in the context of train systems, where accurate predictions are more complex due to fixed routes, scheduled stops, and limited flexibility in rerouting. MTL enhances model performance by jointly optimizing both segment-level and trip-level travel time predictions. In this way, its design allows the model to capture detailed local variations such as delays between consecutive stations while also learning global patterns that influence the overall journey time, such as systemic congestion or consistent dwell-time behaviors. This is especially beneficial for trains, where delays in one segment can have a compounding effect across the entire route due to tightly coupled operations and sequential scheduling leading to error accumulation for predictions.

While the concept is based on the original implementation in AttentionTTE, which separately handles individual and collective predictions, this paper will be using a modified approach in which it integrates MTL directly into the MAGTTE architecture. Instead of introducing additional model components to use AttentionTTE's algorithms, this method reuses the spatial and temporal representations already learned through MAGTTE's multi-attention mechanisms. These matrices and vectors, which encapsulate complex spatial dependencies and time-series patterns across train segments, are utilized both for predicting segment-level travel times and for inferring the overall trip duration through aggregation techniques such as mean or attention pooling. This approach not only reduces computational overhead and model complexity but also ensures that the predictions are made from a consistent representation space. In train systems where data sparsity in certain segments and the cascading impact of delays are common, this shared learning structure improves generalization, reduces error accumulation, and produces more reliable and system-aware estimations.

The MAGTTE model inherently produces individual segment travel time predictions as part of its time series output. This sequence, represented as

$$T = \{t_1, t_2, \ldots, t_n\} \tag{4.14}$$

where each (t_i) corresponds to the predicted travel time for segment (i), fulfills the first objective of the Multi-Task Learning Module, providing accurate local-level estimations for each segment in the route.

To estimate the total travel time for the entire trip, attention pooling is employed. This approach enables the model to generate a comprehensive travel time

estimate while preserving accuracy by assigning meaningful attention weights to each segment. These weights ensure that the contribution of each segment is considered based on its learned significance, rather than treating all segments equally.

Unlike a simple summation of individual scalar segment predictions, attention pooling leverages the **hidden vectors** instead which is denoted as

$$H = [h_1, h_2, \ldots, h_n] \in \mathbb{R}^{n \times d_H} \tag{4.15}$$

These vectors encode the spatial and temporal relationships learned by MAGTTE's GAT and LSTM modules and contain rich contextual information crucial to understanding the overall trip dynamics.

Using these hidden representations and a **learnable query vector** $q \in R^{dq}$, the model applies a softmax-based attention mechanism to compute attention coefficients for each segment. This mechanism scores each segment based on the relevance of its hidden vector (h_i) to the learnable query vector (q), using the formula:

$$a_i = \frac{\exp(q^\top h_i)}{\sum_{j=1}^{n} \exp(q^\top h_j)} \tag{4.16}$$

Next, a contextualized global vector representing the entire trip is calculated by taking a weighted sum of all segment hidden vectors which is denoted as:

$$h_{\text{global}} = \sum_{i=1}^{n} a_i h_i \tag{4.17}$$

Finally, the overall predicted travel time for the trip, referred to as the global prediction, is obtained by applying a linear transformation to this pooled representation denoted as:

$$T_{\text{global}} = W_g^\top h_{\text{global}} + b_g \tag{4.18}$$

where $w_g \in R^d$ is the learnable weight vector and b_g is the bias term.

In the Multi-Task Learning setup applied to the MAGTTE model, the total loss is composed of two components: the local loss, which evaluates the accuracy of the segment-level predictions, and the global loss, which assesses the model's

prediction of the total trip duration. These two losses ensure that the model accurately captures both fine-grained travel time estimations for individual segments and an overall understanding of the full trip.

The segment-level predictions are given by the time series output $T = \{t_1, t_2, \ldots, t_n\}$, where (t_i) is the predicted travel time for segment (i). These predictions are directly compared to the ground truth segment travel times $\hat{T} = \{\hat{t_1}, \hat{t_2}, ..., \hat{t_n}\}$. Moreover, the local loss is calculated using the Mean Squared Error (MSE) using the formula:

$$L_{\text{local}} = \frac{1}{n} \sum_{i=1}^{n} (\hat{t}_i - t_i)^2 \tag{4.19}$$

This loss quantifies how well the model predicts the duration of each individual segment. Minimizing this loss helps the model improve its accuracy on a segment-by-segment basis, which is especially important in train systems where delays can vary by station.

While the global travel time prediction follows similar suit where predicted travel times are compared to the ground-truth global travel times, it uses a different MSE formula which is detailed as:

$$L_{\text{global}} = \left(\hat{T}_{\text{global}} - T_{\text{global}}\right)^2 \tag{4.20}$$

This loss evaluates the model's overall estimation for the whole trip, helping it learn how local conditions influence the total travel time in combination, effectively eliminating the error accumulation issue when using individual segments for prediction.

4.3.5 Model Training and Refinement

To optimize the model for accurate travel time estimation, a multi-task loss function is employed by combining the segment-level and trip-level prediction errors. The total loss is computed as a weighted sum of the local and global losses which is denoted as:

$$L_{\text{total}} = \lambda_1 L_{\text{local}} + \lambda_2 L_{\text{global}} \tag{4.21}$$

where λ_1 and λ_2 are hyperparameters that control the balance between the importance of local segment accuracy and overall trip prediction performance.

To minimize this total loss, both MAGTTE and AttentionTTE use the Adam optimizer to train their models. Similarly, this model will adapt the same optimizer technique. The adam optimizer adapts the learning rate for each parameter based on the first and second moments of the gradients. During each training iteration, the total loss is back propagated through the network, and gradients are computed with respect to all learnable parameters—such as those in the graph attention layers, LSTM units, temporal attention mechanisms, and linear projection heads. Then, these gradients are used to update the model parameters in a direction that reduces the overall error.

This iterative optimization process allows the model to refine its understanding of both short-range segment-level dynamics and long-range trip-wide dependencies. By learning both tasks simultaneously, the model becomes more robust and generalizable, improving its ability to make accurate and consistent travel time predictions under varying traffic patterns, network structures, and operational scenarios.

4.4 Model Validation

To assess the effectiveness and accuracy of the proposed model, this is the assessment strategies framework that will be used:

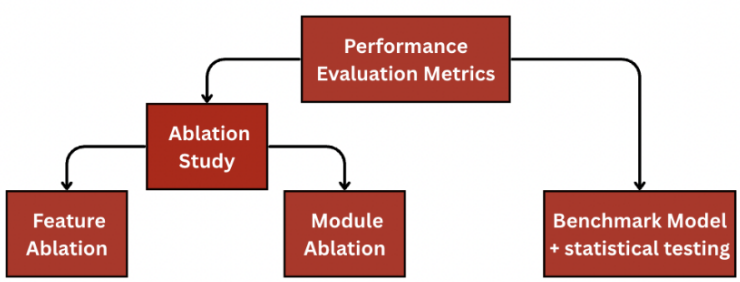

Figure 4.8: Model Validation Framework.

Here, the performance evaluation metrics will be used as a measurement for ablation studies and benchmark model(s) with statistical tests. The ablation study will be divided into two types: Feature Ablation and Module Ablation

which will be discussed in detail further later on. The outline of the following assessment strategies will be applied: (1) Feature Ablation, (2) Module Ablation, (3) Benchmark Modeling with statistical tests.

4.4.1 Performance Evaluation Metrics

As mentioned earlier, these are the evaluation metrics to measure the performance of our proposed model: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), Mean Absolute Percentage Error (MAPE), and Coefficient of Determination (R^2). The mathematical formulations are shown below:

- **Root Mean Squared Error (RMSE)**: Measures the average magnitude of the error between the model's predicted travel times and the actual (observed) travel times. It is defined as:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

- **Mean Absolute Error (MAE)**: Measures the average absolute difference between predicted and actual values.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- **Mean Absolute Percentage Error (MAPE)**: Calculates the average error in the travel time predictions as a percentage of the actual time.

$$\text{MAPE} = \frac{100\%}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|$$

- **Coefficient of Determination (R^2)**: Measures how well the model's predicted travel times explain the variance in the actual travel times.

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

4.4.2 Feature Ablation Study

Firstly, a feature ablation study will be conducted to evaluate how the individual external attributes contribute to the overall model performance. Three model variants will be trained and evaluated, where each variant excludes one of the external features, alongside the original proposed model that includes all external features. Model A will be the full model with all the external factors accounted for, Model B will exclude operational delay (od_t), and Model C will exclude weather conditions (w_t). The configurations of the model variants are summarized in Table 4.2 below.

Table 4.2: Feature Ablation Study Configurations

Model	**Operational Delay** (od_t)	**Weather Conditions** (w_t)
Model A	✓	✓
Model B	×	✓
Model C	✓	×

Let's say $X = X_{operationaldelay}, X_{weather}, X_{otherfeatures}]$, where they add up to d dimensions. Every $X_{variant}$ contains the features associated with the external attribute (e.g., feature 'weathercode' 'rain' is in $X_{weather}$. For each variant, a new X will be constructed by dropping the corresponding columns (or features) that are related to the external attributes.

- Model Variant $A \rightarrow X_A = [X_{operationaldelay}, X_{weather}, X_{otherfeatures}]$
- Model Variant $B \rightarrow X_B = [X_{weather}, X_{otherfeatures}]$
- Model Variant $C \rightarrow X_C = [X_{operationaldelay}, X_{otherfeatures}]$

For each variant, the model is trained and tested on identical datasets and evaluated using the same performance metrics (which will be discussed below). The change in predictive performance for each variant relative to the baseline (Variant A) will reflect which external attribute contributes the most to the predictive accuracy. The change in MAE, RMSE, R^2, and MAPE will be observed at each stage to identify the most impactful predictors and enhance model interpretability.

4.4.3 Module Ablation Study

Another evaluation strategy employed is the *module ablation study*, wherein specific components of the model are systematically removed to assess their individual contributions. In this case, the *multi-task learning module* will be excluded from the framework. The multi-task learning module is responsible for addressing error accumulation during sequential predictions. By removing it, we aim to analyze how the absence of multi-task supervision affects the model's stability and performance over time. The results of this study will help determine whether the module contributes to improving the proposed model's predictive accuracy. Comparative experiments will be conducted between the full model and its ablated version, using the same evaluation metrics - Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), Mean Absolute Percentage Error (MAPE), and the Coefficient of Determination (R^2) - to quantify the impact of the module's removal.

4.4.4 Benchmark Models with Statistical Tests

To demonstrate the effectiveness of the proposed MA-GNN-LSTM model, it will be compared against five benchmark models, namely: Historical Average (HA), Support Vector Regression (SVR), Long Short-Term Memory (LSTM), DeepTTE, Gct-TTE, and T.R2_LSTM_A. From basic statistical techniques to deep learning strategies designed especially for spatiotemporal sequence prediction, these models were chosen to span a broad spectrum of complexity and modeling paradigms.

Table 4.3: Comparison of benchmark models based on learning paradigms

Model	**Machine Learning**	**Deep Learning**	**Transfer Learning**
Historical Average (HA)			
Support Vector Regression (SVR)	✓		
LSTM only		✓	
DeepTTE (CNN+LSTM)		✓	
Gct-TTE		✓	
T.R2_LSTM_A		✓	✓
MA-GNN-LSTM	✓	✓	

- **Historical Average (HA)** (He et al., 2019) – A simple heuristic baseline based on the average travel time within fixed time slots.
- **Support Vector Regression (SVR)** (Sharmila et al., 2019) – A traditional regression model using a radial basis function (RBF) kernel.

- **LSTM** (Hochreiter and Schmidhuber, 1997) – A deep recurrent neural network designed to capture temporal patterns in sequential data.
- **DeepTTE** (Wang et al., 2018) – A spatial-temporal model that combines geo-convolutional layers with LSTM for travel time prediction.
- **Gct-TTE** (Mashurov et al., 2024) – A deep learning model that combines Graph Convolutional Networks (GCNs) for capturing spatial dependencies and Transformer encoders for modeling long-range temporal relationships
- **T.R2_LSTM_A** (Guo et al., 2024) – A temporal-aware LSTM variant that incorporates attention mechanisms to improve sequence-level modeling in travel time estimation.

As shown in the table above, the benchmark models vary from simpler to more complex travel time estimation models. Historical Average serves as a simple or naive baseline to determine whether more complex models actually improve over simple temporal aggregation. The more medium complexity models such as Support Vector Regression (SVR) and Long Short-Term Memory (LSTM) offer additional insight into the advantages of traditional machine learning and basic deep learning techniques. SVR models nonlinear relationships using kernel functions, but it lacks inherent temporal sequence modeling. LSTM, on the other hand, is specifically designed to handle sequential data and can capture long-range temporal dependencies, making it a strong deep learning baseline for time series tasks. Beyond these, more sophisticated models like DeepTTE, Gct-TTE, and T.R2_LSTM_A incorporate both spatial and temporal features. DeepTTE combines convolutional layers (for spatial context) with LSTM (for temporal dynamics), explicitly modeling trajectory paths over road networks. GCT-TTE enhances spatio-temporal learning by integrating graph convolutional networks with transformer encoders, enabling it to capture complex spatial structures and long-range temporal dependencies through global self-attention mechanisms. Meanwhile, T.R2_LSTM_A further advances temporal modeling by introducing attention-like mechanisms and time-awareness into the sequence structure, allowing it to handle finer temporal dependencies at the trip level. This comparison aims to demonstrate that MA-GNN-LSTM achieves competitive performance relative to current up-to-date train travel time estimation models, while providing additional benefits such as explicit spatial modeling and incorporating more external attributes.

To ensure the reliability and statistical validity of our comparisons, each model will be trained and evaluated on the same dataset under identical preprocessing, training conditions, and evaluation metrics. To account for random initialization and training variability, each model will run over 30 times independently. There will be two non-parametric statistical tests used to prove that the proposed model

is statistically significant and meaningful: (1) Wilcon Signed Rank Test and (2) Kruskal-Wallis H Test. A dedicated holdout set will be reserved exclusively for final evaluation, ensuring unbiased assessment after model tuning.

(A) Wilcon Signed Rank Test

The non-parametric alternative to the paired t-test is the Wilcoxon Signed-Ranked Test. It will be used to compare the two related groups (paired samples) without assuming the differences follow a normal distribution. To determine if the performance difference between two models is statistically significant, a paired t-test will be used. Each baseline model will be compared against the proposed model, MA-GNN-LSTM, using the evaluation metrics. The results of the paired t-tests will reveal whether the observed differences in evaluation metrics are statistically meaningful, rather than due to random fluctuations in model training. The procedure for computing the Wilcoxon Signed-Rank statistic W is as follows:

1. **Compute paired differences**
   For each of the n experimental runs, calculate the difference between the two models' results:

$$D_i = \text{Metric}_{\text{Proposed}i} - \text{Metric}_{\text{Baseline}i}$$

2. **Remove zero differences**
   Any pair where $D_i = 0$ (exact tie) is excluded from further analysis. The remaining sample size is denoted n'.

3. **Rank the absolute differences**
   Take the absolute value of each difference, $|D_i|$, and assign ranks from 1 (smallest absolute difference) to n' (largest). If ties occur, assign the average rank that those tied values would occupy.

4. **Reassign signs to ranks**
   Return the sign of the original D_i to its corresponding rank:

   - Positive $D_i \rightarrow$ positive rank
   - Negative $D_i \rightarrow$ negative rank

5. **Calculate the Wilcoxon statistic**
   Sum all the positive ranks to obtain W^+. The test statistic W is defined as the smaller of W^+ and W^- (sum of negative ranks).

6. **Determine significance**
   The W value is then compared to critical values from the Wilcoxon distribution (for small n') or evaluated using a normal approximation (for large

n') to obtain the p-value. If $p < 0.05$, the difference between the two models is considered statistically significant.

In this study, the Wilcoxon Signed-Rank Test will be conducted separately for each evaluation metric, using the results from the 30 independent runs per model, ensuring a robust and assumption-free statistical comparison.

(B)The Kruskal-Wallis H Test

The Kruskal–Wallis H test is a non-parametric test that is particularly suitable in scenarios where the assumptions of normality or homogeneity of variances may not hold. It serves as the non-parametric alternative to the one-way analysis of variance (ANOVA), and is used to assess whether there are statistically significant differences between the median values of more than two independent groups. In this study, the Kruskal–Wallis test will be used to compare the distributions of error metrics (such as MAE, RMSE, and MAPE) across the different travel time estimation models: Historical Average (HA), Support Vector Regression (SVR), LSTM, DeepTTE, T.R2_LSTM_A, and the proposed MA-GNN-LSTM.

We begin by combining all model performance scores (e.g., MAE or RMSE) into a single dataset and sorting the values in ascending order. Ranks are then assigned to each value; in the case of tied scores, the average rank is used. For each model group j, the sum of ranks is computed as:

$$T_j = \sum_{i=1}^{n_j} R_{ij}$$

where R_{ij} is the rank of the i-th observation in model group j, and n_j is the number of observations in group j.

The Kruskal–Wallis H statistic is then calculated using the formula:

$$H = \frac{12}{n(n+1)} \sum_{j=1}^{k} \frac{T_j^2}{n_j} - 3(n+1)$$

where:

- n is the total number of observations across all groups,
- k is the number of groups (i.e., models),

- T_j is the sum of ranks for group j,
- n_j is the number of observations in group j.

The resulting H-statistic is then compared to a chi-squared distribution with $k-1$ degrees of freedom to obtain the p-value.

Decision Rule:

- If $p < \alpha$ (e.g., 0.05), we reject the null hypothesis and conclude that there is a statistically significant difference in performance among the models.
- If $p \geq \alpha$, we fail to reject the null hypothesis, suggesting there is insufficient evidence to declare that the models differ significantly.

A significant result indicates that there is a meaningful difference in performance among the models, suggesting that at least one model consistently outperforms or underperforms the others. This strengthens the comparative evaluation by providing statistical evidence that the proposed MA-GNN-LSTM model performs better or is more robust than the other benchmark models included in the study. On the other hand, if it underperforms, the evaluation will still yield valuable insights by identifying potential reasons for its shortcomings. This enables us to propose targeted mitigation strategies to improve the model. By doing these assessment strategies, we can not only confirm if a model performs better but also provides a deeper understanding of why the model behaves as it does and how it can be enhanced.

Appendix A

Research Ethics Documents

College of Computer Studies

RESEARCH / PROJECT ETHICS REVIEW FORM
For Thesis / Capstone / Dissertation Proposals
(*ver.Oct2024c*)

Names of Student Researcher(s):
Joehanna Llarenas Cansino,
Harmony Claire Cheng Dy,
Christa Ysabel Talusan Hernandez,
Justine Nicole Chya Uy

Research/Thesis/Capstone Title: Right on Track: A Hybrid Approach to Travel Time Estimation

College: College of Computer Studies

Department: Department of Software Technology

Course: Bachelor of Science in Computer Science major in Software Technology

Expected Duration of the Project: from: Term 3 of AY 24-25 to: Term 3 of AY 25-26

PROTOCOL:
(*To be filled up by proponents; Use the Ethics Checklists as guides in determining areas for ethical concern/consideration. Attach and properly reference the accomplished ethics checklists and supporting documents.)*

		Reference Document (include page number)
Target participants	Not Applicable	
Data to be used / collected	1. Real-time transit trip data from the Canberra Metro Light Rail GTFS-RT feed (both trip and vehicle), downloaded as a CSV. 2. Real-time weather data collected through an automated weather API (Open-Meteo) 3. Light Rail information (Stops and stop times) 4. Data includes fields such as trip_id, stopID, stop_sequence, arrival/departure times, arrival/departure delays, vehicleID, congestionLevel, currentStatus, coordinates, and etc. 5. Weather data using the Open-Meteo API, including variables like temperature, precipitation, wind speed, and weather conditions, based on corresponding timestamps and locations.	1-5. Proposal Manuscript, pp. 53-58

Brief procedure	1. Download trip and vehicle data from the official open data portal in CSV format 2. Download light rail information from https://www.transport.act.gov.au/google transit/google_transit_lr .zip in zip format 3. Use an API to collect corresponding weather data based on timestamps and locations (per station) Preprocess, merge, and normalize the datasets 4. Perform machine learning analysis to estimate or predict travel time. 5. Evaluate and interpret the results.	1-3. Proposal Manuscript, pp. 53-60 4. Proposal Manuscript, pp. 62-88 5. Proposal Manuscript, pp. 88-96
Potential risks	There are no potential risks as the study does not involve human subjects, personal data, or sensitive information. All data used is publicly available and anonymized	
Applicable Terms of Use or Licensing Agreement of Data, Models and/or Tools	1. The Canberra Metro Light Rail data is published under the ACT Government Open Data License, which permits reuse for research and public interest. 2. Open-Meteo provides free access for non-commercial use with no authentication required and is permitted under their terms for academic purposes.	
Steps to safeguard the participants and/or data	1. No human participant data is involved. 2. All data collected will be stored in secure, access-controlled environments. 3. API usage will follow Open-Meteo's acceptable use policies. 4. No personal, sensitive, or identifiable information is collected or stored.	
Other concerns and considerations	1. The research will remain strictly non-commercial. All data sources will be properly cited and used in accordance with their licensing terms. 2. Any visualizations or reports will use aggregated, non-sensitive data.	

PANEL'S ASSESSMENT/COMMENTS:

The study has no ethical concerns.

PANEL'S DECISION:

- ☑ **Approved** (*Proponent/s may proceed with the study/data collection*)
- ☐ **Modifications Required** (*Proponent/s should address ethical issues, follow the panel's recommendations for required modifications, and resubmit Ethics Review Form for approval before proceeding with the study or before starting with data collection*)
- ☐ **For Full Review** (*Proponent/s should submit the proposal to the University Research Ethics Review Committee (RERC) and obtain a research ethics clearance before proceeding with the study or before starting with data collection*)

JUSTIFICATION:
The study design has addressed all ethical considerations.

Briane Paul V. Samson, PhD
Name and Signature of Adviser/Mentor
Date: 7 Aug 2025

Briane Paul V. Samson, PhD
Name and Signature of Panel Member
Date: 7 Aug 2025

Jazzie Jao
Name and Signature of Panel Member
Date:

Abien Agarap
Name and Signature of Panel Member
Date: 8 Aug 2025

DE LA SALLE UNIVERSITY
General Research Ethics Checklist

This checklist is to ensure that the research conducted by the faculty members and students of De La Salle University is carried out according to the guiding principles outlined in the Code of Research Ethics of the University. The investigator is advised to refer to the De La Salle University Code of Research Ethics and Guide to Responsible Conduct of Research before completing this checklist. Statements pertinent to ethical issues in research should be addressed below. The checklist will help the researcher/s and advisers/readers/evaluators determine whether procedures should be undertaken during the course of the research to maintain ethical standards. The University's Guide to the Responsible Conduct of Research provides details on these appropriate procedures.

Researcher Details	
Students	Joehanna Llarenas Cansino, Harmony Claire Cheng Dy, Christa Ysabel Talusan Hernandez, Justine Nicole Chya Uy
Thesis Adviser	Briane Paul V. Samson
Department/College	Department of Software Technology / College of Computer Studies
Proposed Title of the Research	Right on Track: A Hybrid Approach to Travel Time Estimation
Term(s) and academic year in which research project is to be undertaken	Term 3 of AY 24-25 to Term 3 of AY 25-26

This checklist must be completed AFTER the* De La Salle University Code of Ethics *has been read and BEFORE gathering data.

Questions	Yes	No
1. Does your research involve human participants (this includes new data gathered or using pre-existing data)? If your answer is **yes**, please answer **Checklist A (Human Participants).** **Please specify if the kind of research you will be conducting falls under any of the following Human Participants sub-categories:**		✔
1.A. Will you be conducting Action Research in an existing business, company, or school? If your answer is **yes**, please answer **Checklist F (Action Research).**		✔
1.B. Does your research involve online communities (this includes culling data from social media platforms, online forums and blogs)? If your answer is **yes**, please answer **Checklist G (Internet Research).**	✔	

1.C. Does your research involve human participants who are situated in a community and may necessitate permission to acquire access to them? If your answer is **yes**, please answer **Checklist H (Community Research).**		✔
2. Will your research make use of documents which are not in the public domain and, thus, require permission for use from the custodian of such documents? **If YES, please provide certification that permission from the custodian of the data was sought and granted.**		✔
3. Will your research make use of secondary data (e.g., surveys, inventories, plans, official documents, etc.) from an institution, organization, or agency, which are not in the public domain and, thus, require permission for use from the custodian of such documents? **If YES, please provide certification that permission to use the data was sought from the institution, organization, or agency and approval was granted.**		✔
4. Does your research involve animals (non-human subjects)? If your answer is **yes**, please answer **Checklist B (Animal Subjects).**		✔
5. Does your research involve Wildlife? If your answer is **yes**, please answer **Checklist C (Wildlife)**.		✔
6. Does your research involve microorganisms that are infectious, disease causing or harmful to health? If your answer is **yes**, please answer **Checklist D (Infectious Agents).**		✔
7. Does your research involve toxic/chemicals/ substances/materials? If your answer is **yes**, please answer **Checklist E (Toxic Agents).**		✔

Research with Ethical Issues to address:

If you have a YES answer to any of the above categories, you will be required to complete a detailed checklist for that particular category. A YES answer does not mean the disapproval of your research proposal. By providing you with a more detailed checklist, we ensure that the ethical concerns are identified so these can be addressed in adherence to the University Code of Ethics.

Declaration of Conflict of Interest

[✔] 1. I do not have a conflict of interest in any form (personal, financial, proprietary, or professional) with the sponsor/grant-giving organization, the study, the co-investigators/personnel, or the site.

[ ] 2. I do have a conflict of interest, specifically:

[ ] A. I have a personal/family or professional interest in the results of the study (family members who are co-proponents or personnel in the study, membership in relevant professional associations/organizations).
Please describe the personal/family or professional interest:

[ ] B. I have proprietary interest vested in this proposal (with the intent to apply for a patent, trademark, copyright, or license)
Please describe proprietary interest:

[ ] C. I have significant financial interest vested in this proposal (remuneration that exceeds P250,000.00 each year or equity interest in the form of stock, stock options or other ownership interests).
Please describe financial interest:

DE LA SALLE UNIVERSITY

Checklist G
Research Ethics Checklist for Investigators conducting Internet Research

***This checklist must be completed AFTER the* De La Salle University Code of Research Ethics and Guide to Responsible Conduct of Research *has been read* and *BEFORE gathering data*. The University Code of Research Ethics is available at http://www.dlsu.edu.ph/offices/urco/forms/URCO-Code-of-Research-Ethics_August2011.pdf**

NOTE: This checklist is completed after the research proponent fills out the General Checklist Form and Checklist A.

Only answer this Checklist if you are conducting Internet Research (research on online communities).

Which of the following online data will you be using in your research? Check all that apply:

☐Social Media Platform (e.g. Twitter, Facebook, 9gag)
☐Blogs & Forum including Comments (e.g. BoredPanda, Reddit)
☐E-mails & Chats
☐Video Blogs (e.g. YouTube)
☐Collaborative (e.g. Wikipedia)
✓ Websites (e.g. News, Media, Company, E-Commerce, Government, etc.)
☐Online Recruitment Platform (e.g. Mechanical Turk, Freelancer, E-lance, O-Desk)
✓ Open Source Websites (e.g. GitHub)
☐Others:_____

Where will you source your data?

☐New
✓ Existing
☐Combination

What type of data will be collected?

✓ Text
☐Audio
☐Video/Film
☐Photo
☐Metadata (e.g. Profile, Geographic Location, Tags)

☐ Presentations (e.g. downloaded PowerPoint or Keynote presentations)
☐ Contents of an application such as input, output, log files for analysis software, simulation software, schemas
☐ Correspondence, including electronic mail

What is the period coverage of data collection? (indicate in years and months)

2021-01-01 to 2022-11-25

How many participants will you collect data from?

Not Applicable

What are all the websites you will source your data from? Please list all URLs:

1. https://www.data.act.gov.au/Transport/Canberra-Metro-Light-Rail-Transit-Feed-Trip-Update/jxpp-4iiz/about_data
2. https://www.data.act.gov.au/Transport/Canberra-Metro-Light-Rail-Transit-Feed-Vehicle-Upd/92fy-xvmy
3. https://www.transport.act.gov.au/__data/assets/pdf_file/0006/1259934/TCCS-GTFS-Implementation-Specification-v3.3-Bus-and-Light-Rail.pdf
4. https://gtfs.org/documentation/schedule/reference/
5. Open-Meteo.com
6. https://www.transport.act.gov.au/contact-us/information-for-developers

How will informed consent be obtained? Please check.

☐Electronic Information Sheet with "check box" for consent
☐E-mail with name
☐Consent implied through submitting information
☐Written Consent
☐Audio-recorded Consent
✓ Others: _____

How will the participants obtain a copy of the informed consent form? Please check.

☐Hard copy

☐Online copy (If online copies will be given, the users should be able to click on the Print button incorporated in the information page, or an e-mail should be sent to the participant containing the agreement form.)

If a participant wishes to leave the study and wishes to withdraw the submitted information, describe the steps a participant must take to withdraw from the study in the research proposal and in the informed consent form.

	Yes	No	Not Applicable
1. Is the data you are planning to gather publicly available? **If NO…attach a letter of support from the website or server owner/moderator indicating approval to use this for data gathering**	✓		
2. Will the participants be compensated for participating? **If YES…indicate the type of compensation to be provided and provide information on how appropriate and just compensation (proportionate to the contribution in the research, research budget, and local conditions) will be provided.**		✓	
3. Will you have minors as participants in your study? Minors are individuals under the age of 18 years old. **If YES…Obtain parental/guardian consent and participant assent to participate in your study. Attach the parental consent and assent forms to your proposal. The consent forms should indicate the measures you will undertake to ensure confidentiality and protect the participants. The electronic versions of the consent forms are acceptable.**		✓	
4. Will data collection involve students?		✓	

If YES… let the participant sign an informed consent form that assures that his/her academic status will not be affected by participation or non-participation in the study.			
5. Will data collection involve persons who belong to a vulnerable group (PWDs, minorities, abuse victims, students, etc.) **If YES… ensure that the participants are briefed about the study and the data gathering protocols, and will be provided special assistance should they require it. Please submit a corresponding detailed research protocol.**		✓	
	Yes	**No**	**Not Applicable**
6. Will data be collected using an automated system? (e.g. hardware, software, or meta search engines) **If YES… provide details on the automation system (web crawler, etc.)**	✓ The main dataset is obtained as a CSV file from the publicly available Canberra Metro Light Rail feed. Real-time weather data will be collected using an automated system via a weather API (Open-Meteo.com)		
7. Will there be information about the study that will be deliberately withheld from the participants?		✓	

If YES… what information will be withheld, what are the benefits for doing so, and how will debriefing be done?			
8. Will your data be stored after this study? **If YES, please indicate in the research proposal how long data will be stored, and how it will be stored. Include this information in the informed consent form and provide a plan for data confidentiality.**			✓
9. Will the data be made available for future research? **If YES… indicate in the informed consent form that the results of the study will be used for future studies. Specify that the identity of the participant will be anonymized in future studies. Provide participants with the option to allow the results to be used for further studies and the option to withdraw their results from future studies.**			✓
10. Will you anonymize the participants in your data? **If YES…provide details on how you will assure the confidentiality and anonymity of the participants.**			✓
11. Will the results of this study have a commercial value? If yes, do you intend to apply for a patent for the output of this research? Please check: _____ Yes _____ No		✓	

Appendix B

Turnitin Summary Report

PAPER NAME

THESIS-1-DOCU.pdf

AUTHOR

-

WORD COUNT

25200 Words

CHARACTER COUNT

143975 Characters

PAGE COUNT

79 Pages

FILE SIZE

871.4KB

SUBMISSION DATE

Jul 7, 2025 8:35 PM GMT+8

REPORT DATE

Jul 7, 2025 8:36 PM GMT+8

14% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

- 8% Internet database
- 10% Publications database
- Crossref database
- Crossref Posted Content database
- 7% Submitted Works database

Excluded from Similarity Report

- Bibliographic material
- Quoted material
- Manually excluded sources

References

(2025a).

(2025b).

Abdollahi, M., Khaleghi, T., & Yang, K. (2020). An integrated feature learning approach using deep learning for travel time prediction. *Expert Systems with Applications*, *139*, 112864.

Akay, S., Cakir, D., & Arica, N. (2025). Interpersonal relationship detection using multi-head graph attention networks with multi-feature fusion. *IEEE Access*, *13*, 90945-90961. doi: 10.1109/ACCESS.2025.3571099

AlHadidi, T., & Rakha, H. A. (2020). Estimating transit vehicle dwell times at bus stops. *arXiv preprint arXiv:2104.09266*.

Ali, A., Zhu, Y., & Zakarya, M. (2022). Exploiting dynamic spatio-temporal graph convolutional neural networks for citywide traffic flows prediction. *Neural networks*, *145*, 233–247.

Alrukaibi, F., Alsaleh, R., & Sayed, T. (2019). Applying machine learning and statistical approaches for travel time estimation in partial network coverage. *Sustainability*, *11*(14), 3822.

Andersen, G. (2024, Feb). *Machine learning engineering: Challenges in handling noisy and incomplete data.* Retrieved from `https://moldstud.com/articles/p-machine-learning-engineering-challenges-in-handling-noisy-and-incomplete-data`

Artan, M. Ş., & Şahin, İ. (2021). Exploring patterns of train delay evolution and timetable robustness. *IEEE Transactions on Intelligent Transportation Systems*, *23*(8), 11205–11214.

Badhrudeen, M., Ramesh, V., & Vanajakshi, L. (2016). Headway analysis using automated sensor data under indian traffic conditions. *Transportation Research Procedia*, *17*, 331–339.

Badrestani, E., Bahrak, B., Elahi, A., Faramarzi, A., Golshanrad, P., Monsefi, A. K., ... Zirak, A. (2019). Real-time travel time estimation using matrix factorization. *arXiv preprint arXiv:1912.00455*.

Bao, X., Li, Y., Li, J., Shi, R., & Ding, X. (2021). Prediction of train arrival delay using hybrid elm-pso approach. *Journal of Advanced Transportation*,

2021(1), 7763126.

Barrios, W., & Jin, S. (2024). Multi-layer learnable attention mask for multimodal tasks. *arXiv preprint arXiv:2406.02761*.

Bek, S., & Monari, E. (2016). The crowd congestion level—a new measure for risk assessment in video-based crowd monitoring. In *2016 ieee global conference on signal and information processing (globalsip)* (pp. 1212–1217).

Bondy, J. A., & Murty, U. S. R. (2008). *Graph theory*. Springer Publishing Company, Incorporated.

Borsky, S., & Unterberger, C. (2019). Bad weather and flight delays: The impact of sudden and slow onset weather events. *Economics of transportation*, *18*, 10–26.

Buchanan, M. (2019). The benefits of public transport. *Nature Physics*, *15*(9), 876–876.

Cao, Q., Shan, Z., Liu, C., & Yan, M. (2025). Attention-based feature fusion empowered encoder-decoder framework for nighttime traffic perception from high-altitude surveillance system. *IEEE Transactions on Intelligent Transportation Systems*.

Chai, T., & Draxler, R. R. (2014). Root mean square error (rmse) or mean absolute error (mae)?–arguments against avoiding rmse in the literature. *Geoscientific Model Development*, *7*(3), 1247–1250.

Chen, G. (2016). A gentle tutorial of recurrent neural network with error backpropagation. *arXiv preprint arXiv:1610.02583*.

Chen, H., & Shi, Z. (2020). A spatial-temporal attention-based method and a new dataset for remote sensing image change detection. *Remote Sensing*, *12*(10). Retrieved from `https://www.mdpi.com/2072-4292/12/10/1662` doi: 10.3390/rs12101662

Chicco, D., Warrens, M. J., & Jurman, G. (2021). The coefficient of determination r-squared is more informative than smape, mae, mape, mse and rmse in regression analysis evaluation. *PeerJ Computer Science*, *7*, e623.

Crawshaw, M. (2020). *Multi-task learning with deep neural networks: A survey.* Retrieved from `https://arxiv.org/abs/2009.09796`

Dai, Z., Ma, X., & Chen, X. (2019). Bus travel time modelling using gps probe and smart card data: A probabilistic approach considering link travel time and station dwell time. *Journal of Intelligent Transportation Systems*, *23*(2), 175–190.

Dong, H., Wang, N., Zeng, Y., Zhu, J., Li, X., Qiu, H., & Yang, X. (2025). Data-driven prediction of intersection traffic congestion. In *International conference on smart transportation and city engineering (stce 2024)* (Vol. 13575, pp. 970–974).

Dong, Y., Liu, Y., & Qin, S. J. (2019). Efficient dynamic latent variable analysis for high-dimensional time series data. *IEEE Transactions on Industrial Informatics*, *16*(6), 4068–4076.

Du, J., & Ma, W. (2024). Maximin headway control of automated vehicles for system optimal dynamic traffic assignment in general networks. *Transportation Research Part E: Logistics and Transportation Review*, *188*, 103628.

Du, W., Wang, Y., & Qiao, Y. (2018). Recurrent spatial-temporal attention network for action recognition in videos. *IEEE Transactions on Image Processing*, *27*(3), 1347-1360. doi: 10.1109/TIP.2017.2778563

Duan, Y., L.V., Y., & Wang, F.-Y. (2016). Travel time prediction with lstm neural network. In *2016 ieee 19th international conference on intelligent transportation systems (itsc)* (p. 1053-1058). doi: 10.1109/ITSC.2016.7795686

Elmorshedy, L., Smirnov, I., & Abdulhai, B. (2024). Freeway congestion management on multiple consecutive bottlenecks with rl-based headway control of autonomous vehicles. *IET Intelligent Transport Systems*, *18*(6), 1137–1163.

Elsir, A. K., Alfateh M. Tag, & She, Y. (2023). Triplet-contrastive periodical siamese graph networks for travel time estimation. *IEEE Transactions on Intelligent Transportation Systems*.

Encord. (2023). *How to split your dataset into training, validation, and test sets.* Retrieved from `https://encord.com/blog/train-val-test-split/`

Endo, K., Edelenbos, J., & Gianoli, A. (2024). Finance arrangements and governance modes toward sustainable infrastructure: the case of urban railway projects in manila, the philippines. *Discover Sustainability*, *5*(1), 235.

Fajardo, J., Hernandez, B. L., Passi, J., Vigonte, F., & Abante, M. V. (2023). Demographic analysis, patron satisfaction, and suggestions within the light rail transit (lrt) line 2 service: A scholarly-based service enhancement matrix (imrad). *Patron Satisfaction, and Suggestions within the Light Rail Transit (Lrt) Line*, *2*.

Fan, W., & Gurmu, Z. (2015). Dynamic travel time prediction models for buses using only gps data. *International Journal of Transportation Science and Technology*, *4*(4), 353–366.

Fang, Z.-Q., Sun, Q.-C., Chen, L., Hu, D.-L., & Gao, Y.-J. (2024). Spatio-temporal learning for route-based travel time estimation. *Journal of Computer Science and Technology*, *39*(5), 1107–1122.

Fey, M. (2019). Just jump: Dynamic neighborhood aggregation in graph neural networks. *arXiv preprint arXiv:1904.04849*.

Fu, J., Gao, J., & Xu, C. (2022). Learning semantic-aware spatial-temporal attention for interpretable action recognition. *IEEE Transactions on Circuits and Systems for Video Technology*, *32*(8), 5213-5224. doi: 10.1109/TCSVT.2021.3137023

Fu, Q., Ding, S., Zhang, T., Wang, R., Hu, P., & Pu, C. (2024). Short-term train arrival delay prediction: a data-driven approach. *Railway Sciences*, *3*(4), 514–529.

Fu, T.-y., & Lee, W.-C. (2019). Deepist: Deep image-based spatio-temporal network for travel time estimation. In *Proceedings of the 28th acm international*

conference on information and knowledge management (pp. 69–78).

Gers, F. A., Schmidhuber, J., & Cummins, F. (2000). Learning to forget: Continual prediction with lstm. *Neural Computation*, *12*(10), 2451-2471. doi: 10.1162/089976600300015015

Grossberg, S. (2013). Recurrent neural networks. *Scholarpedia*, *8*(2), 1888.

Guo, J., Wang, W., Guo, J., D'Ariano, A., Bosi, T., & Zhang, Y. (2024a). An instance-based transfer learning model with attention mechanism for freight train travel time prediction in the china–europe railway express. *Expert Systems with Applications*, *251*, 123989.

Guo, J., Wang, W., Guo, J., D'Ariano, A., Bosi, T., & Zhang, Y. (2024b). An instance-based transfer learning model with attention mechanism for freight train travel time prediction in the china–europe railway express. *Expert Systems with Applications*, *251*, 123989. Retrieved from `https://www.sciencedirect.com/science/article/pii/S0957417424008558` doi: https://doi.org/10.1016/j.eswa.2024.123989

Guo, J., Wang, W., Tang, Y., Zhang, Y., & Zhuge, H. (2022). A cnn-bi$_l stmparallelnetworkapproachfortraint$ doi: https://doi.org/10.1016/j.knosys.2022.109796

Guo, S., Shi, L., Jiang, X., Lv, P., Liu, Q., Hu, Y., ... Xu, M. (2025). An efficient ungrouped mask method with two learnable parameters for 3d object detection. *IEEE Transactions on Multimedia*, *27*, 1003-1017. doi: 10.1109/TMM.2024.3349912

Han, L., Du, B., Lin, J., Sun, L., Li, X., & Peng, Y. (2021). Multi-semantic path representation learning for travel time estimation. *IEEE Transactions on Intelligent Transportation Systems*, *23*(8), 13108–13117.

Hanafy, G. M. M. M. . A. E. G. M. A., Y. A. (2020). A comparison between adaptive neural networks algorithms for estimating vehicle travel time. *15th International Conference on Computer Engineering and Systems (ICCES)*.

He, P., Jiang, G., Lam, S.-K., & Sun, Y. (2020). Learning heterogeneous traffic patterns for travel time prediction of bus journeys. *Information Sciences*, *512*, 1394–1406.

He, P., Jiang, G., Lam, S.-K., & Tang, D. (2018). Travel-time prediction of bus journey with multiple bus trips. *IEEE Transactions on Intelligent Transportation Systems*, *20*(11), 4192–4205.

Hochreiter, S., & Schmidhuber, J. (1997a). Long short-term memory. *Neural computation*, *9*(8), 1735–1780.

Hochreiter, S., & Schmidhuber, J. (1997b). Long short-term memory. *Neural Computation*, *9*(8), 1735-1780. doi: 10.1162/neco.1997.9.8.1735

Hodson, T. O. (2022). Root mean square error (rmse) or mean absolute error (mae): When to use them or not. *Geoscientific Model Development Discussions*, 1–10.

HogoNext. (2024). *How to correctly split your artificial neural network data.* Retrieved from `https://hogonext.com/how-to-correctly-split-your-ann-data/`

Hu, S., Hu, J., Zhang, X., Jin, S., & Xu, X. (2025, 02). Drug target affinity prediction based on multi-scale gated power graph and multi-head linear attention mechanism. *PLOS ONE*, *20*(2), 1-19. Retrieved from `https://doi.org/10.1371/journal.pone.0315718` doi: 10.1371/journal.pone.0315718

Huang, P., Wen, C., Fu, L., Peng, Q., & Li, Z. (2020). A hybrid model to improve the train running time prediction ability during high-speed railway disruptions. *Safety Science*, *122*, 104510.

Huang, P., Wen, C., Fu, L., Peng, Q., & Tang, Y. (2020). A deep learning approach for multi-attribute data: A study of train delay prediction in railway systems. *Information Sciences*, *516*, 234–253.

Indurthi, S. R., Chung, I., & Kim, S. (2019). Look harder: A neural machine translation model with hard attention. , 3037–3043.

Ioffe, S., & Szegedy, C. (2015). *Batch normalization: Accelerating deep network training by reducing internal covariate shift.*

James, J. (2021). Citywide estimation of travel time distributions with bayesian deep graph learning. *IEEE Transactions on Knowledge and Data Engineering*, *35*(3), 2366–2378.

Jang, J. (2024). Travel time prediction using machine learning algorithms: Focusing on k-nn, lstm, and transformer. *The Open Transportation Journal*, *18*(1).

Janiesch, P. Z., Christian, & Heinrich, K. (2021). Machine learning and deep learning. *Electronic markets*.

Jegelka, S. (2022). Theory of graph neural networks: Representation and learning. In *The international congress of mathematicians* (pp. 1–23).

Jin, G., Yan, H., Li, F., Huang, J., & Li, Y. (2024). Spatio-temporal dual graph neural networks for travel time estimation. *ACM Transactions on Spatial Algorithms and Systems*, *10*(3), 1–22.

Jin, G., Yan, H., Li, F., Li, Y., & Huang, J. (2023). Dual graph convolution architecture search for travel time estimation. *ACM Transactions on Intelligent Systems and Technology*, *14*(4), 1–23.

Karunasingha, D. S. K. (2022). Root mean square error or mean absolute error? use their ratio as well. *Information Sciences*, *585*, 609–629.

Kessler, L., Rempe, F., & Bogenberger, K. (2021). Multi-sensor data fusion for accurate traffic speed and travel time reconstruction. *Frontiers in Future Transportation*, *2*, 766951.

Khemani, B., Patil, S., Kotecha, K., & Tanwar, S. (2024). A review of graph neural networks: concepts, architectures, techniques, challenges, datasets, applications, and future directions. *Journal of Big Data*, *11*(1), 18.

Kim, Y., Kim, S., Lee, Y., Hong, J., & Lee, Y. (2024). Multi-attention recommender system for non-fungible tokens. *Engineering Applications of Artificial Intelligence*, *137*, 109179. Retrieved from `https://www.sciencedirect.com/science/article/pii/S095219762401337X` doi: https://doi.org/10.1016/j.engappai.2024.109179

Konishi, T., Kurokawa, M., Ono, C., Ke, Z., Kim, G., & Liu, B. (2023, 23–29 Jul). Parameter-level soft-masking for continual learning. , *202*, 17492–17505. Retrieved from `https://proceedings.mlr.press/v202/konishi23a.html`

Kuipers, R. A. (2024). Understanding dwell times using automatic passenger count data: A quantile regression approach. *Journal of Rail Transport Planning & Management*, *29*, 100431.

Lapamonpinyo, P., Derrible, S., & Corman, F. (2022). Real-time passenger train delay prediction using machine learning: A case study with amtrak passenger train routes. *IEEE Open Journal of Intelligent Transportation Systems*, *3*, 539–550.

Lawson, D., Chiu, C.-C., Tucker, G., Raffel, C., Swersky, K., & Jaitly, N. (2018). Learning hard alignments with variational inference. , 5799–5803.

Lee, J. B., Rossi, R. A., Kim, S., Ahmed, N. K., & Koh, E. (2019). Attention models in graphs: A survey. *ACM Transactions on Knowledge Discovery from Data (TKDD)*, *13*(6), 1–25.

Lee, S., & Song, B. C. (2019). *Graph-based knowledge distillation by multi-head attention network.* Retrieved from `https://arxiv.org/abs/1907.02226`

Lee, W.-H., Yen, L.-H., & Chou, C.-M. (2016). A delay root cause discovery and timetable adjustment model for enhancing the punctuality of railway services. *Transportation Research Part C: Emerging Technologies*, *73*, 49–64.

Lemmerz, T., Herlé, S., & Blankenbach, J. (2023). Geostatistics on real-time geodata streams—high-frequent dynamic autocorrelation with an extended spatiotemporal moran's i index. *ISPRS International Journal of Geo-Information*, *12*(9), 350.

Li, C., Wang, J., Niu, Z., Yao, J., & Zeng, X. (2021, 04). A spatial-temporal gated attention module for molecular property prediction based on molecular geometry. *Briefings in Bioinformatics*, *22*(5), bbab078. Retrieved from `https://doi.org/10.1093/bib/bbab078` doi: 10.1093/bib/bbab078

Li, J., Liu, X., Zhang, W., Zhang, M., Song, J., & Sebe, N. (2020). Spatio-temporal attention networks for action recognition and detection. *IEEE Transactions on Multimedia*, *22*(11), 2990-3001. doi: 10.1109/TMM.2020.2965434

Li, L., & He, H. (2022). Bipartite graph based multi-view clustering. *IEEE Transactions on Knowledge and Data Engineering*, *34*(7), 3111-3125. doi: 10.1109/TKDE.2020.3021649

Li, M., Feng, Y., & Wu, X. (2024a). Attentiontte: a deep learning model for estimated time of arrival. *Frontiers in Artificial Intelligence*, *7*, 1258086. doi: 10.3389/frai.2024.1258086

Li, M., Feng, Y., & Wu, X. (2024b). Attentiontte: a deep learning model for estimated time of arrival. *Frontiers in Artificial Intelligence*, *Volume 7 - 2024*. Retrieved from `https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2024.1258086` doi: 10.3389/frai.2024.1258086

Li, Q., Jing, R., & Dong, Z. S. (2023). Flight delay prediction with priority information of weather and non-weather features. *IEEE Transactions on Intelligent Transportation Systems*, *24*(7), 7149–7165.

Li, X., & Peng, D. (2022). Extremal problems for graphical function-indices and f-weighted adjacency matrix. *Discrete Mathematics Letters*. Retrieved from `http://cfc.nankai.edu.cn/_upload/article/files/64/96/0f291f2a4669a8f0d8ed8fe74459/5a30af58-35c5-4501-b7d6-1dd8b4672ae2.pdf` (In memory and honor of Professor Nenad Trinajstić)

Li, X., & Yang, N. (2024). Unified approach for spectral properties of weighted adjacency matrices for graphs with degree-based edge-weights. *Linear Algebra and its Applications*, *696*, 46–67.

Li, X. L., & Yang, N. (2024). Spectral properties and energy of weighted adjacency matrices for graphs with degree-based edge-weight functions. *Acta Mathematica Sinica, English Series*, *40*(12), 3027–3042.

Li, Z., Tang, C., Liu, X., Zheng, X., Zhang, W., & Zhu, E. (2022). Consensus graph learning for multi-view clustering. *IEEE Transactions on Multimedia*, *24*, 2461-2472. doi: 10.1109/TMM.2021.3081930

Liao, T., Han, L., Xu, Y., Zhu, T., Sun, L., & Du, B. (2024). Multi-faceted route representation learning for travel time estimation. *IEEE Transactions on Intelligent Transportation Systems*.

Liu, C., Ding, H., Zhang, Y., & Jiang, X. (2023). Multi-modal mutual attention and iterative interaction for referring image segmentation. *IEEE Transactions on Image Processing*, *32*, 3054–3065.

Liu, H., Jiang, W., Liu, S., & Chen, X. (2023). Uncertainty-aware probabilistic travel time prediction for on-demand ride-hailing at didi. In *Proceedings of the 29th acm sigkdd conference on knowledge discovery and data mining* (pp. 4516–4526).

Liu, M., Gao, H., & Ji, S. (2020). Towards deeper graph neural networks. In *Proceedings of the 26th acm sigkdd international conference on knowledge discovery & data mining* (pp. 338–348).

Liu, S., Johns, E., & Davison, A. J. (2019, June). End-to-end multi-task learning with attention. In *Proceedings of the ieee/cvf conference on computer vision and*

pattern recognition (cvpr).

Liu, S., Ma, X., Wu, H., & Li, Y. (2020). An end to end framework with adaptive spatio-temporal attention module for human action recognition. *IEEE Access*, *8*, 47220-47231. doi: 10.1109/ACCESS.2020.2979549

Liu, S., Zhang, L., Ge, J., Li, W., & Long, Y. (2025). Framework for timely perception of spatiotemporal crowd congestion risk in public spaces based on video pedestrian tracking and geographic mapping. *GIScience & Remote Sensing*, *62*(1), 2480416.

Ma, J., Chan, J., Rajasegarar, S., & Leckie, C. (2022a). Multi-attention graph neural networks for city-wide bus travel time estimation using limited data. *Expert Systems with Applications*, *202*, 117057.

Ma, J., Chan, J., Rajasegarar, S., & Leckie, C. (2022b). Multi-attention graph neural networks for city-wide bus travel time estimation using limited data. *Expert Systems with Applications*, *202*, 117057. Retrieved from `https://www.sciencedirect.com/science/article/pii/S0957417422004717` doi: https://doi.org/10.1016/j.eswa.2022.117057

Ma, Y., & Huang, B. (2017). Bayesian learning for dynamic feature extraction with application in soft sensing. *IEEE Transactions on Industrial Electronics*, *64*(9), 7171–7180.

Majeed, A., & Rauf, I. (2020). Graph theory: A comprehensive survey about graph theory applications in computer science and social networks. *Inventions*, *5*(1), 10.

Mashurov, V., Chopuryan, V., Porvatov, V., Ivanov, A., & Semenova, N. (2024). Gct-tte: graph convolutional transformer for travel time estimation. *Journal of Big Data*, *11*(1), 15.

Meyes, R., Lu, M., de Puiseau, C. W., & Meisen, T. (2019). Ablation studies in artificial neural networks. *arXiv preprint arXiv:1901.08644*.

Michail, O. (2016). An introduction to temporal graphs: An algorithmic perspective. *Internet Mathematics*, *12*(4), 239–280.

Mijares, A. C., Regmi, M. B., & Yai, T. (2014). Enhancing the sustainability and inclusiveness of the metro manila's urban transportation systems: Proposed

fare and policy reforms. *Transport and Communications Bulletin for Asia and the Pacific*, *84*(2), 28–40.

Miller, P., de Barros, A. G., Kattan, L., & Wirasinghe, S. (2016). Public transportation and sustainability: A review. *KSCE Journal of Civil Engineering*, *20*(3), 1076–1083.

Minh, Q. T., Pham-Nguyen, H.-N., Tan, H. M., & Long, N. X. (2019). Traffic congestion estimation based on crowd-sourced data. In *2019 international conference on advanced computing and applications (acomp)* (pp. 119–126).

Nesmachnow, S., Massobrio, R., Guridi, S., Olmedo, S., & Tchernykh, A. (2023). Big data analysis for travel time characterization in public transportation systems. *Sustainability*, *15*(19), 14561.

Nguyen, T. (2022). Best practices for data splitting in machine learning projects. *Journal of Data Science Applications*, *5*(2), 45–58.

Nosedal-Sánchez, J., Piera-Eroles, M. A., & Lebbink, D. (2020). Identification and empirical characterisation of flight arrival variation and the impact on departure punctuality. *International Journal of Services and Operations Management*, *35*(3), 307–320.

Noursalehi, P., Koutsopoulos, H. N., & Zhao, J. (2021). Dynamic origin-destination prediction in urban rail systems: A multi-resolution spatio-temporal deep learning approach. *IEEE Transactions on Intelligent Transportation Systems*, *23*(6), 5106–5115.

of Sciences, N. A., Medicine, on Engineering, D., Sciences, P., Aeronautics, Board, S. E., ... to Reduce Commercial Aviation Carbon Emissions, E. S. (2016). *Commercial aircraft propulsion and energy systems research: reducing global carbon emissions*. National Academies Press.

Palmqvist, C.-W., Tomii, N., & Ochiai, Y. (2020). Explaining dwell time delays with passenger counts for some commuter trains in stockholm and tokyo. *Journal of Rail Transport Planning & Management*, *14*, 100189.

Pang, Z., Wang, L., Wang, S., Li, L., & Peng, Q. (2023). Dynamic train dwell time forecasting: a hybrid approach to address the influence of passenger flow fluctuations. *Railway Engineering Science*, *31*(4), 351–369.

Pascanu, R., Mikolov, T., & Bengio, Y. (2013, 17–19 Jun). On the difficulty of training recurrent neural networks. In S. Dasgupta & D. McAllester (Eds.), *Proceedings of the 30th international conference on machine learning* (Vol. 28, pp. 1310–1318). Atlanta, Georgia, USA: PMLR. Retrieved from `https://proceedings.mlr.press/v28/pascanu13.html`

Ping, D. (2024). *The machine learning solutions architect handbook: Practical strategies and best practices on the ml lifecycle, system design, mlops, and generative ai*. Packt Publishing Ltd.

Pirzada, S. (2007). Applications of graph theory. In *Pamm: Proceedings in applied mathematics and mechanics* (Vol. 7, pp. 2070013–2070013).

Pouyanfar, S. S. Y. Y. H. T. Y. T. M. P. R. M.-L. S. S.-C. C., Samira, & Iyengar, S. S. (2018). A survey on deep learning: Algorithms, techniques, and applications. *ACM COmputing Sirveys*.

Putatunda, S., & Laha, A. K. (2023). Travel time prediction in real time for gps taxi data streams and its applications to travel safety. *Human-Centric Intelligent Systems*, *3*(3), 381–401.

Qiu, B., & Fan, W. (2021). Machine learning based short-term travel time prediction: Numerical results and comparative analyses. *Sustainability*, *13*(13), 7454.

Rajagopal, B. G., Kumar, M., Samui, P., Kaloop, M. R., & Shahdah, U. E. (2022). A hybrid dnn model for travel time estimation from spatio-temporal features. *Sustainability*, *14*(21), 14049.

Reka, R., Karthick, R., Saravana Ram, R., & Singh, G. (2024). Multi head self-attention gated graph convolutional network based multi-attack intrusion detection in manet. *Computers Security*, *136*, 103526. Retrieved from `https://www.sciencedirect.com/science/article/pii/S0167404823004364` doi: https://doi.org/10.1016/j.cose.2023.103526

Ren, X., & Liu, Y. (2022). Semantic-guided multi-mask image harmonization. In S. Avidan, G. Brostow, M. Cissé, G. M. Farinella, & T. Hassner (Eds.), *Computer vision – eccv 2022* (pp. 564–579). Cham: Springer Nature Switzerland.

Sanaullah, I., Quddus, M., & Enoch, M. (2016). Developing travel time estimation methods using sparse gps data. *Journal of Intelligent Transportation Systems*,

20(6), 532–544.

Sarhani, M., & Voß, S. (2024). Prediction of rail transit delays with machine learning: How to exploit open data sources. *Multimodal Transportation*, *3*(2), 100120.

Schilde, M., Doerner, K., & Hartl, R. (2014). Integrating stochastic time-dependent travel speed in solution methods for the dynamic dial-a-ride problem. *European Journal of Operational Research*, *238*(1), 18-30. Retrieved from `https://www.sciencedirect.com/science/article/pii/S0377221714002197` doi: https://doi.org/10.1016/j.ejor.2014.03.005

Schmidt, R. M. (2019). Recurrent neural networks (rnns): A gentle introduction and overview. *arXiv preprint arXiv:1912.05911*.

Sheikholeslami, S., Meister, M., Wang, T., Payberah, A. H., Vlassov, V., & Dowling, J. (2021). Autoablation: Automated parallel ablation studies for deep learning. In *Proceedings of the 1st workshop on machine learning and systems* (pp. 55–61).

Shi, C., Zou, W., Wang, Y., Zhu, Z., Chen, T., Zhang, Y., & Wang, N. (2025). Enhancing travel time prediction for intelligent transportation systems: A high-resolution origin–destination-based approach with multi-dimensional features. *Sustainability*, *17*(5), 2111.

Shi, L., Zhang, Y., Cheng, J., & Lu, H. (2020, November). Decoupled spatial-temporal attention network for skeleton-based action-gesture recognition. In *Proceedings of the asian conference on computer vision (accv).*

Shi, R., Xu, X., Li, J., & Li, Y. (2021). Prediction and analysis of train arrival delay based on xgboost and bayesian optimization. *Applied Soft Computing*, *109*, 107538.

Singgam, P., Silalahi, A., Tampubolon, J. D., & Harliana, P. (2024). Analysis of travel time approximation on trips using maclaurin series. *AURELIA: Jurnal Penelitian dan Pengabdian Masyarakat Indonesia*, *4*(1), 1222–1225.

Spanninger, T., Trivella, A., Büchel, B., & Corman, F. (2022). A review of train delay prediction approaches. *Journal of Rail Transport Planning & Management*, *22*, 100312.

Sporns, O. (2018). Graph theory methods: applications in brain networks. *Dialogues in clinical neuroscience*, *20*(2), 111–121.

Staudemeyer, R. C., & Morris, E. R. (2019). *Understanding lstm – a tutorial into long short-term memory recurrent neural networks.* Retrieved from `https://arxiv.org/abs/1909.09586`

Sun, C., Li, C., Lin, X., Zheng, T., Meng, F., Rui, X., & Wang, Z. (2023). Attention-based graph neural networks: a survey. *Artificial intelligence review*, *56*(Suppl 2), 2263–2310.

Sun, Y., Spall, J., Wong, W., & Zhao, X. (2025). Real-time bus travel time prediction and reliability quantification: A hybrid markov model. *arXiv preprint arXiv:2503.05907*.

Tan, Z., Zhu, Y., & Liu, B. (2023). Learning spatial-temporal feature with graph product. *Signal Processing*, *210*, 109062.

Tang, J., Xu, L., Luo, C., & Ng, T. S. A. (2021). Multi-disruption resilience assessment of rail transit systems with optimized commuter flows. *Reliab. Eng. Syst. Saf.*, *214*(Oct), 107715.

Tang, Y., Wang, Y., Liu, C., Yuan, X., Wang, K., & Yang, C. (2023). Semi-supervised lstm with historical feature fusion attention for temporal sequence dynamic modeling in industrial processes. *Engineering Applications of Artificial Intelligence*, *117*, 105547.

Tao, R., Guo, W., Jie, M., & Liu, X. (2024). Attention-driven graph-sequence fusion network for next point-of-interest recommendation. In *2024 international conference on image processing, computer vision and machine learning (icicml)* (p. 1958-1964). doi: 10.1109/ICICML63543.2024.10957767

Taslimi, B., Sarijaloo, F. B., Liu, H., & Pardalos, P. M. (2022). A novel mixed integer programming model for freight train travel time estimation. *European Journal of Operational Research*, *300*(2), 676–688.

Thung, K.-H., & Wee, C.-Y. (2018). A brief review on multi-task learning. *Multimedia Tools and Applications*, *77*(17), 29705–29725. doi: 10.1007/s11042-018-6463-x

Tiong, K., Ma, Z., & Palmqvist, C. W. (2022). Real-time train arrival time

prediction at multiple stations and arbitrary times. In *2022 ieee 25th international conference on intelligent transportation systems (itsc)* (pp. 793–798).

Tong, J., Xing, Y., Cao, Z., & Zhao, D. (2022). Multi-task multi-attention graph neural network for mobile crowd sensing data reconstruction and prediction. In *2022 ieee 8th international conference on cloud computing and intelligent systems (ccis)* (pp. 654–661).

Torres, N., Trujillo, L., Maldonado, Y., & Vera, C. (2021). Correction of the travel time estimation for ambulances of the red cross tijuana using machine learning. *Computers in Biology and Medicine*, *137*, 104798.

Tripathy, S. K., & Srivastava, R. (2020). A real-time two-input stream multi-column multi-stage convolution neural network (tis-mcms-cnn) for efficient crowd congestion-level analysis. *Multimedia systems*, *26*(5), 585–605.

Vandenhende, S., Georgoulis, S., Van Gansbeke, W., Proesmans, M., Dai, D., & Van Gool, L. (2022). Multi-task learning for dense prediction tasks: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *44*(7), 3614-3633. doi: 10.1109/TPAMI.2021.3054719

Van Houdt, G., Mosquera, C., & Nápoles, G. (2020). A review on the long short-term memory model. *Artificial Intelligence Review*, *53*(8), 5929–5955.

Wang, C., Zhao, F., Zhang, H., Luo, H., Qin, Y., & Fang, Y. (2022). Fine-grained trajectory-based travel time estimation for multi-city scenarios based on deep meta-learning. *IEEE Transactions on Intelligent Transportation Systems*, *23*(9), 15716–15728.

Wang, D., Zhang, J., Cao, W., Li, J., & Zheng, Y. (2018). When will you arrive? estimating travel time based on deep neural networks. In *Proceedings of the aaai conference on artificial intelligence* (Vol. 32).

Wang, H., Sun, T., Fan, C., & Gu, J. (2023). *Customizable combination of parameter-efficient modules for multi-task learning.* Retrieved from `https://arxiv.org/abs/2312.03248`

Wang, H., Tang, X., Kuo, Y.-H., Kifer, D., & Li, Z. (2019). A simple baseline for travel time estimation using large-scale trip data. *ACM Transactions on Intelligent Systems and Technology (TIST)*, *10*(2), 1–22.

Wang, H., Yang, Y., & Liu, B. (2020). Gmc: Graph-based multi-view clustering. *IEEE Transactions on Knowledge and Data Engineering*, *32*(6), 1116-1129. doi: 10.1109/TKDE.2019.2903810

Wang, Q., Wan, J., & Yuan, Y. (2017). Deep metric learning for crowdedness regression. *IEEE Transactions on Circuits and Systems for Video Technology*, *28*(10), 2633–2643.

Wang, Q., Xu, C., Zhang, W., & Li, J. (2021). Graphtte: Travel time estimation based on attention-spatiotemporal graphs. *IEEE Signal Processing Letters*, *28*, 239–243.

Wang, Y., Qian, C., & Qin, S. J. (2023). Attention-mechanism based dipls-lstm and its application in industrial process time series big data prediction. *Computers & Chemical Engineering*, *176*, 108296.

Wang, Y., Wang, G., Hou, X., & Yang, F. (2023). Motif adjacency matrix and spectral clustering of directed weighted networks. *AIMS Math*, *8*(6), 13797–13814.

Wang, Y., Zhang, C., Xiang, S., & Pan, C. (2022). Subgraph-aware graph structure revision for spatial–temporal graph modeling. *Neural Networks*, *154*, 190–202.

Wang, Z., Fu, K., & Ye, J. (2018). Learning to estimate the travel time. In *Proceedings of the 24th acm sigkdd international conference on knowledge discovery & data mining* (pp. 858–866).

Welch, T. F., & Widita, A. (2019). Big data in public transportation: a review of sources and methods. *Transport reviews*, *39*(6), 795–818.

Wong, V. W., & Law, K. H. (2023). Fusion of cctv video and spatial information for automated crowd congestion monitoring in public urban spaces. *Algorithms*, *16*(3), 154.

Wu, F., Jing, X.-Y., Wei, P., Lan, C., Ji, Y., Jiang, G.-P., & Huang, Q. (2022). Semi-supervised multi-view graph convolutional networks with application to webpage classification. *Information Sciences*, *591*, 142-154. Retrieved from `https://www.sciencedirect.com/science/article/pii/S0020025522000160` doi: https://doi.org/10.1016/j.ins.2022.01.013

Wu, F., Zheng, C., Zhang, C., Ma, J., & Sun, K. (2023). Multi-view multi-attention graph neural network for traffic flow forecasting. *Applied Sciences*, *13*(2), 711.

Wu, J., Du, B., Wu, Q., Shen, J., Zhou, L., Cai, C., ... Zhou, Q. (2021). A hybrid lstm-cps approach for long-term prediction of train delays in multivariate time series. *Future Transportation*, *1*(3), 765–776.

Wu, J., & Zhang, C. (2024). The role of temperature and rainfall in traffic congestion: evidence from 98 chinese cities. *Weather, Climate, and Society*, *16*(3), 481–495.

Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., & Yu, P. S. (2020). A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems*, *32*(1), 4–24.

Wu, Z., Pan, S., Long, G., Jiang, J., & Zhang, C. (2019). Graph wavenet for deep spatial-temporal graph modeling. *arXiv preprint arXiv:1906.00121*.

Xiao, S., Du, S., Chen, Z., Zhang, Y., & Wang, S. (2023). Dual fusion-propagation graph neural network for multi-view clustering. *IEEE Transactions on Multimedia*, *25*, 9203-9215. doi: 10.1109/TMM.2023.3248173

Xie, Z., Wang, M., Ye, Z., Zhang, Z., & Fan, R. (2022). Graphiler: Optimizing graph neural networks with message passing data flow graph. *Proceedings of Machine Learning and Systems*, *4*, 515–528.

Xu, G., Liu, P., Zhu, Z., Liu, J., & Xu, F. (2021). Attention-enhanced graph convolutional networks for aspect-based sentiment classification with multi-head attention. *Applied Sciences*, *11*(8). Retrieved from `https://www.mdpi.com/2076-3417/11/8/3640` doi: 10.3390/app11083640

Xu, H., Bao, J., & Liu, W. (2023). Double-branch multi-attention based graph neural network for knowledge graph completion. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: Long papers)* (pp. 15257–15271).

Xu, S., Zhang, R., Cheng, W., & Xu, J. (2022). Mtlm: a multi-task learning model for travel time estimation. *GeoInformatica*, *26*(2), 379–395.

Xue, M., Chen, M., Peng, D., Guo, Y., & Chen, H. (2021). One spatio-temporal

sharpening attention mechanism for light-weight yolo models based on sharpening spatial attention. *Sensors*, *21*(23). Retrieved from https://www.mdpi.com/1424-8220/21/23/7949 doi: 10.3390/s21237949

Yalamanchili, C. m. (2023, Feb). *A comprehensive study of machine learning models - types, examples, and use cases - consensus.* Retrieved from https://consensus.app/papers/a-comprehensive-study-of-machine-learning-models-types-yalamanchili/ffd493ae39a75cffac98e1dff0e3057e/?extracted-answer=Different%2Btypes%2Bof%2Bmachine%2Blearning%2Bmodels%2C%2Bsuch%2Bas%2Bregression%2C%2Bclassification%2C%2Bclustering%2C%2Bassociation%2C%2Banomaly%2Bdetection%2C%2Band%2Breinforcement%2Blearning%2C%2Bcan%2Bsolve%2Bdistinct%2Bproblems.&q=what%2Bare%2Bthe%2Bdifferent%2Bproblem%2Btypes%2Bfor%2Bmachine%2Blearning%2Bmodels%3F

Yan, Y., He, X., Wan, W., & Liu, J. (2023). *Mmnet: Multi-mask network for referring image segmentation.* Retrieved from https://arxiv.org/abs/2305.14969

Yang, L., Jiang, S., & Zhang, F. (2022). Multitask learning with graph neural network for travel time estimation. *Computational Intelligence and Neuroscience*, *2022*(1), 6622734.

Yang, R., Niu, J., Xu, Y., Wang, Y., & Qiu, L. (2023a). Action recognition based on gcn with adjacency matrix generation module and time domain attention mechanism. *Symmetry*, *15*(10), 1954.

Yang, R., Niu, J., Xu, Y., Wang, Y., & Qiu, L. (2023b). Action recognition based on gcn with adjacency matrix generation module and time domain attention mechanism. *Symmetry*, *15*(10). Retrieved from https://www.mdpi.com/2073-8994/15/10/1954 doi: 10.3390/sym15101954

Yang, S., & Qian, S. (2019). Understanding and predicting travel time with spatio-temporal features of network traffic flow, weather and incidents. *IEEE Intelligent Transportation Systems Magazine*, *11*(3), 12–28.

Ying, Z., Bourgeois, D., You, J., Zitnik, M., & Leskovec, J. (2019). Gnnexplainer: Generating explanations for graph neural networks. *Advances in neural information processing systems*, *32*.

Yuan, H., Li, G., & Bao, Z. (2022). Route travel time estimation on a road network revisited: Heterogeneity, proximity, periodicity and dynamicity. *Proceedings of*

the VLDB Endowment, *16*(3), 393–405.

Yue, X., et al. (2024). Temporal-spatial cross attention network for recognizing imagined characters. *Scientific Reports*, *14*. Retrieved from `https://www.nature.com/articles/s41598-024-59263-5` doi: 10.1038/s41598-024-59263-5

Zeng, W., Fu, C.-W., Arisona, S. M., Erath, A., & Qu, H. (2014). Visualizing mobility of public transportation system. *IEEE transactions on visualization and computer graphics*, *20*(12), 1833–1842.

Zhang, H., Zhao, F., Wang, C., Luo, H., Xiong, H., & Fang, Y. (2024). Knowledge distillation for travel time estimation. *IEEE Transactions on Intelligent Transportation Systems*.

Zhang, J., Yunos, Z. M., & Haron, H. (2023). Parallel multi-head graph attention network (pmgat) model for human-object interaction detection. *IEEE Access*, *11*, 131708-131725. doi: 10.1109/ACCESS.2023.3335193

Zhang, X., & Chen, M. (2019). Quantifying the impact of weather events on travel time and reliability. *Journal of advanced transportation*, *2019*(1), 8203081.

Zhang, Y., & Yang, Q. (2017, 09). An overview of multi-task learning. *National Science Review*, *5*(1), 30-43. Retrieved from `https://doi.org/10.1093/nsr/nwx105` doi: 10.1093/nsr/nwx105

Zhang, Y., & Yang, Q. (2022). A survey on multi-task learning. *IEEE Transactions on Knowledge and Data Engineering*, *34*(12), 5586-5609. doi: 10.1109/TKDE.2021.3070203

Zhang, Z., Bu, J., Ester, M., Zhang, J., Li, Z., Yao, C., ... Wang, C. (2023). Hierarchical multi-view graph pooling with structure learning. *IEEE Transactions on Knowledge and Data Engineering*, *35*(1), 545-559. doi: 10.1109/TKDE.2021.3090664

Zhang, Z., Li, S., Zhou, J., Wang, J., Angirekula, A., Zhang, A., & Zhao, L. (2024). Non-euclidean spatial graph neural network. In *Proceedings of the 2024 siam international conference on data mining (sdm)* (pp. 154–162).

Zhang, Z., Lv, Z., Gan, C., & Zhu, Q. (2020). Human action recognition using convolutional lstm and fully-connected lstm with different attentions. *Neurocomput-*

ing, *410*, 304-316. Retrieved from `https://www.sciencedirect.com/science/article/pii/S0925231220310031` doi: https://doi.org/10.1016/j.neucom.2020.06.032

Zhang, Z., Wang, H., Fan, Z., Chen, J., Song, X., & Shibasaki, R. (2022). Gof-tte: Generative online federated learning framework for travel time estimation. *IEEE Internet of Things Journal*, *9*(23), 24107–24121.

Zhao, X., Li, H., Shen, X., Liang, X., & Wu, Y. (2018, September). A modulation module for multi-task learning with applications in image retrieval. In *Proceedings of the european conference on computer vision (eccv).*

Zhao, X., Wu, Z., Tan, S., Fan, D.-J., Li, Z., Wan, X., & Li, G. (2022). Semi-supervised spatial temporal attention network for video polyp segmentation. In L. Wang, Q. Dou, P. T. Fletcher, S. Speidel, & S. Li (Eds.), *Medical image computing and computer assisted intervention – miccai 2022* (pp. 456–466). Cham: Springer Nature Switzerland.

Zheng, C., Fan, X., Wang, C., & Qi, J. (2020). Gman: A graph multi-attention network for traffic prediction. In *Proceedings of the aaai conference on artificial intelligence* (Vol. 34, pp. 1234–1241).

Zheng, L., Ma, H., & Wang, Z. (2024). Travel time estimation for urban arterials based on the multi-source data. *Sustainability*, *16*(17), 7845.

Zheng, Z., Ye, Y., Zhu, Y., Zhang, S., & Yu, J. J. (2023). Data-driven methods for travel time estimation: A survey. In *2023 ieee 26th international conference on intelligent transportation systems (itsc)* (p. 1292-1299). doi: 10.1109/ITSC57777.2023.10422502

Zhou, W., Xiao, X., Gong, Y.-J., Chen, J., Fang, J., Tan, N., ... others (2023). Travel time distribution estimation by learning representations over temporal attributed graphs. *IEEE Transactions on Intelligent Transportation Systems*, *24*(5), 5069–5081.

Zhu, L., Chen, C., Wang, H., Yu, F. R., & Tang, T. (2023). Machine learning in urban rail transit systems: A survey. *IEEE Transactions on Intelligent Transportation Systems*, *25*(3), 2182–2207.

Zhu, Y. Y. Y. L., Yuanshao, & Yu, J. J. (2022). Cross-area travel time uncertainty estimation from trajectory data: A federated learning approach. *IEEE*

Transactions on Intelligent Transportation Systems.

Zou, Z., Yang, H., & Zhu, A.-X. (2020). Estimation of travel time based on ensemble method with multi-modality perspective urban big data. *IEEE Access*, *8*, 24819–24828.